

Displaying the Dataset's Description

Let's look at the dataset's description. The `DESCR` information includes:

- Number of Instances—this dataset contains 20,640 samples.
- Number of Attributes—there are 8 features (attributes) per sample.
- Attribute Information—feature descriptions.
- Missing Attribute Values—none are missing in this dataset.

According to the description, the target variable in this dataset is the *median house value*—this is the value we'll be trying to predict via multiple linear regression.

[lick here to view code image](#)

```
n [3]: print(california.DESCR)
.. _california_housing_dataset:

California Housing dataset
-----

**Data Set Characteristics:**

:Number of Instances: 20640

:Number of Attributes: 8 numeric, predictive attributes and
the target

:Attribute Information:
  - MedInc           median income in block
  - HouseAge         median house age in block
  - AveRooms         average number of rooms
  - AveBedrms        average number of bedrooms
  - Population       block population
  - AveOccup         average house occupancy
  - Latitude         house block latitude
  - Longitude        house block longitude

:Missing Attribute Values: None

This dataset was obtained from the StatLib repository.
http://lib.stat.cmu.edu/datasets/

The target variable is the median house value for California districts.

This dataset was derived from the 1990 U.S. census, using one row per ce
```

```
It can be downloaded/loaded using the  
:func:`sklearn.datasets.fetch_california_housing` function.
```

```
.. topic:: References
```

- Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions, Statistics and Probability Letters, 33 (1997) 291-297

gain, the Bunch object's data and target attributes are NumPy arrays containing the 20,640 samples and their target values respectively. We can confirm the number of samples (rows) and features (columns) by looking at the data array's shape attribute, which shows that there are 20,640 rows and 8 columns:

[lick here to view code image](#)

```
In [4]: california.data.shape  
Out[4]: (20640, 8)
```

Similarly, you can see that the number of target values—that is, the median house values—matches the number of samples by looking at the target array's shape:

[lick here to view code image](#)

```
In [5]: california.target.shape  
Out[5]: (20640,)
```

The Bunch's **feature_names attribute** contains the names that correspond to each column in the data array:

[lick here to view code image](#)

```
In [6]: california.feature_names  
Out[6]:  
['MedInc',  
 'HouseAge',  
 'AveRooms',  
 'AveBedrms',  
 'Population',  
 'AveOccup',  
 'Latitude',  
 'Longitude']
```

14.5.2 Exploring the Data with Pandas

Let's use a pandas `DataFrame` to explore the data further. We'll also use the `DataFrame` with Seaborn in the next section to visualize some of the data. First, let's import pandas and set some options:

[lick here to view code image](#)

```
In [7]: import pandas as pd

In [8]: pd.set_option('precision', 4)

In [9]: pd.set_option('max_columns', 9)

In [10]: pd.set_option('display.width', None)
```

In the preceding `set_option` calls:

- `'precision'` is the maximum number of digits to display to the right of each decimal point.
- `'max_columns'` is the maximum number of columns to display when you output the `DataFrame`'s string representation. By default, if pandas cannot fit all of the columns left-to-right, it cuts out columns in the middle and displays an ellipsis () instead. The `'max_columns'` setting enables pandas to show all the columns using multiple rows of output. As you'll see momentarily, we'll have nine columns in the `DataFrame`—the eight dataset features in `california.data` and an additional column for the target median house values (`california.target`).
- `'display.width'` specifies the width in characters of your Command Prompt (Windows), Terminal (macOS/Linux) or shell (Linux). The value `None` tells pandas to auto-detect the display width when formatting string representations of `Series` and `DataFrames`.

Next, let's create a `DataFrame` from the Bunch's `data`, `target` and `feature_names` arrays. The first snippet below creates the initial `DataFrame` using the data in `california.data` and with the column names specified by `california.feature_names`. The second statement adds a column for the median house values stored in `california.target`:

[lick here to view code image](#)

```
In [11]: california_df = pd.DataFrame(california.data,
...:                                  columns=california.feature_names)
...:
In [12]: california_df['MedHouseValue'] = pd.Series(california.target)
```

We can peek at some of the data using the `head` function. Notice that pandas displays the `DataFrame`'s first six columns, then skips a line of output and displays the remaining columns. The `\` to the right of the column head "AveOccup" indicates that there are more columns displayed below. You'll see the `\` only if the window in which IPython is running is too narrow to display all the columns left-to-right:

[lick here to view code image](#)

```
In [13]: california_df.head()
Out[13]:
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	\
0	8.3252	41.0	6.9841	1.0238	322.0	2.5556	
1	8.3014	21.0	6.2381	0.9719	2401.0	2.1098	
2	7.2574	52.0	8.2881	1.0734	496.0	2.8023	
3	5.6431	52.0	5.8174	1.0731	558.0	2.5479	
4	3.8462	52.0	6.2819	1.0811	565.0	2.1815	

	Latitude	Longitude	MedHouseValue
0	37.88	-122.23	4.526
1	37.86	-122.22	3.585
2	37.85	-122.24	3.521
3	37.85	-122.25	3.413
4	37.85	-122.25	3.422

Let's get a sense of the data in each column by calculating the `DataFrame`'s summary statistics. Note that the median income and house values (again, measured in hundreds of thousands) are from 1990 and are significantly higher today:

[lick here to view code image](#)

```
In [14]: california_df.describe()
Out[14]:
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	\
count	20640.0000	20640.0000	20640.0000	20640.0000	20640.0000	
mean	3.8707	28.6395	5.4290	1.0967	1425.4767	
std	1.8998	12.5856	2.4742	0.4739	1132.4621	
min	0.4999	1.0000	0.8462	0.3333	3.0000	
25%	2.5634	18.0000	4.4407	1.0061	787.0000	
50%	3.5348	29.0000	5.2291	1.0488	1166.0000	

75%	4.7432	37.0000	6.0524	1.0995	1725.0000
max	15.0001	52.0000	141.9091	34.0667	35682.0000

	AveOccup	Latitude	Longitude	MedHouseValue
count	20640.0000	20640.0000	20640.0000	20640.0000
mean	3.0707	35.6319	-119.5697	2.0686
std	10.3860	2.1360	2.0035	1.1540
min	0.6923	32.5400	-124.3500	0.1500
25%	2.4297	33.9300	-121.8000	1.1960
50%	2.8181	34.2600	-118.4900	1.7970
75%	3.2823	37.7100	-118.0100	2.6472
max	1243.3333	41.9500	-114.3100	5.0000

14.5.3 Visualizing the Features

It's helpful to visualize your data by plotting the target value against *each* feature—in this case, to see how the median home value relates to each feature. To make our visualizations clearer, let's use `DataFrame` method **sample** to randomly select 10% of the 20,640 samples for graphing purposes:

[lick here to view code image](#)

```
In [15]: sample_df = california_df.sample(frac=0.1, random_state=17)
```

The keyword argument `frac` specifies the fraction of the data to select (0.1 for 10%), and the keyword argument `random_state` enables you to seed the random number generator. The integer seed value (17), which we chose arbitrarily, is crucial for *reproducibility*. Each time you use the *same* seed value, method `sample` selects the *same* random subset of the `DataFrame`'s rows. Then, when we graph the data, you should get the *same* results.

Next, we'll use Matplotlib and Seaborn to display scatter plots of each of the eight features. Both libraries can display scatter plots. Seaborn's are more attractive and require less code, so we'll use Seaborn to create the following scatter plots. First, we import both libraries and use Seaborn function `set` to scale each diagram's fonts to two times their default size:

[lick here to view code image](#)

```
In [16]: import matplotlib.pyplot as plt
```

```
In [17]: import seaborn as sns
```

```
In [18]: sns.set(font_scale=2)

In [19]: sns.set_style('whitegrid')
```

The following snippet displays the scatter plots.⁸ Each shows one feature along the x -axis and the median home value (`california.target`) along the y -axis, so we can see how each feature and the median house values relate to one another. We display each scatter plot in a separate window. The windows are displayed in the order the features were listed in snippet [6] with the most recently displayed window in the foreground:

⁸ When you execute this code in IPython, each window will be displayed in front of the previous one. As you close each, you'll see the one behind it.

[lick here to view code image](#)

```
In [20]: for feature in california.feature_names:
...:     plt.figure(figsize=(16, 9))
...:     sns.scatterplot(data=sample_df, x=feature,
...:                     y='MedHouseValue', hue='MedHouseValue',
...:                     palette='cool', legend=False)
...:
```

For each feature name, the snippet first creates a 16-inch-by-9-inch Matplotlib Figure—we're plotting many data points, so we chose to use a larger window. If this window is larger than your screen, Matplotlib fits the Figure to the screen. Seaborn uses the current Figure to display the scatter plot. If you do not create a Figure first, Seaborn will create one. We created the Figure first here so we could display a large window for a scatter plot containing over 2000 points.

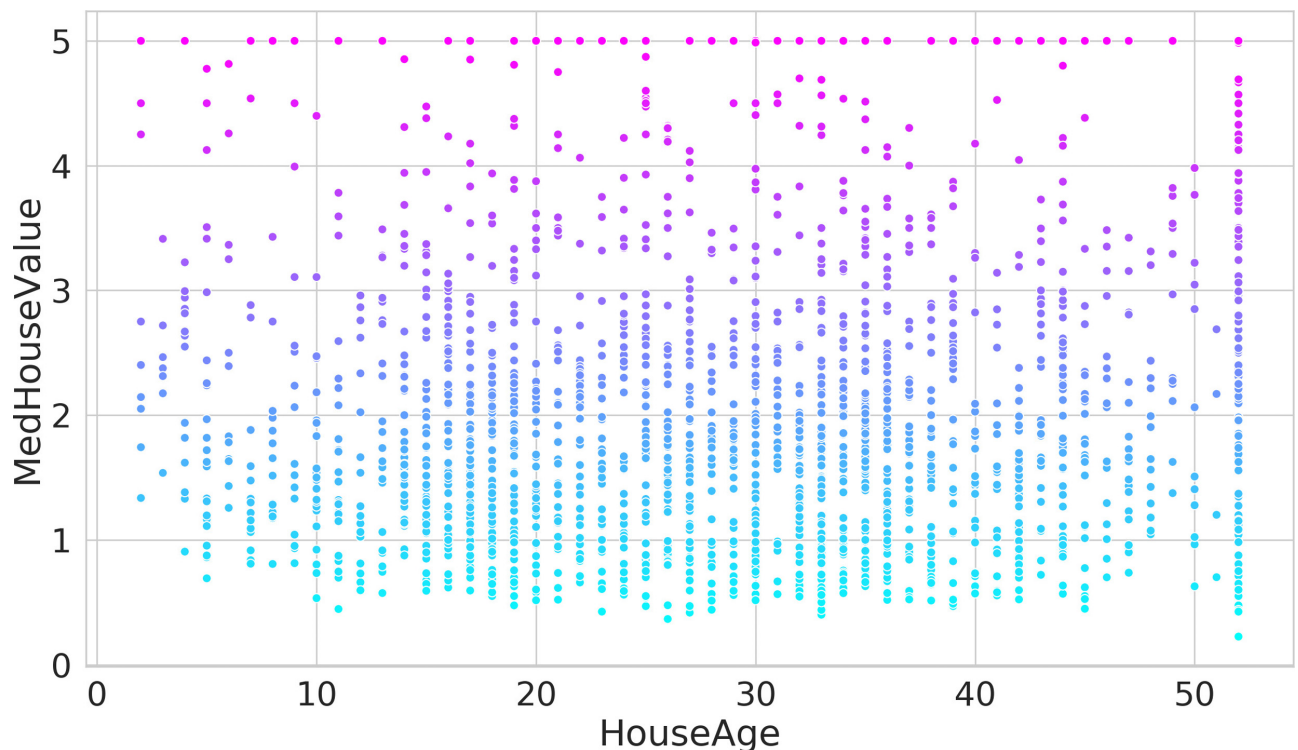
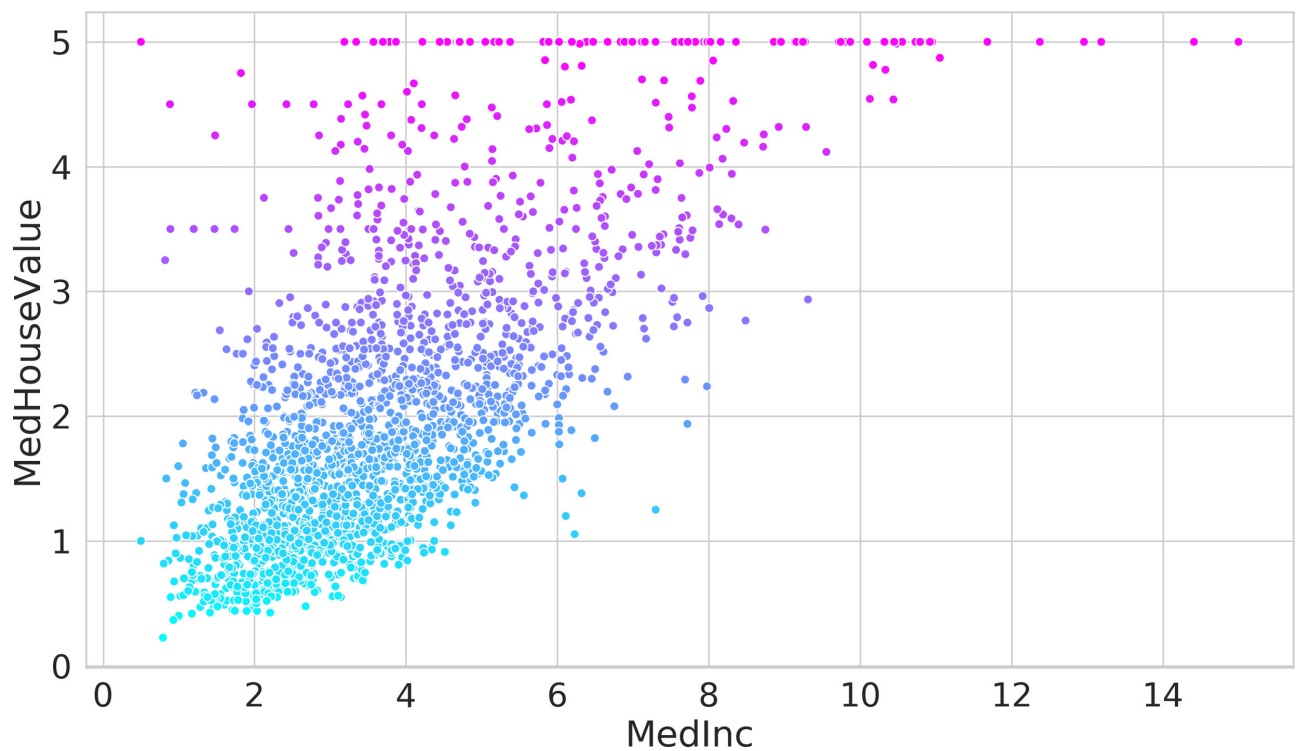
Next, the snippet creates a Seaborn `scatterplot` in which the x -axis shows the current feature, the y -axis shows the 'MedHouseValue' (*median house values*), and the 'MedHouseValue' determines the dot colors (*hue*). Some interesting things to notice in these graphs:

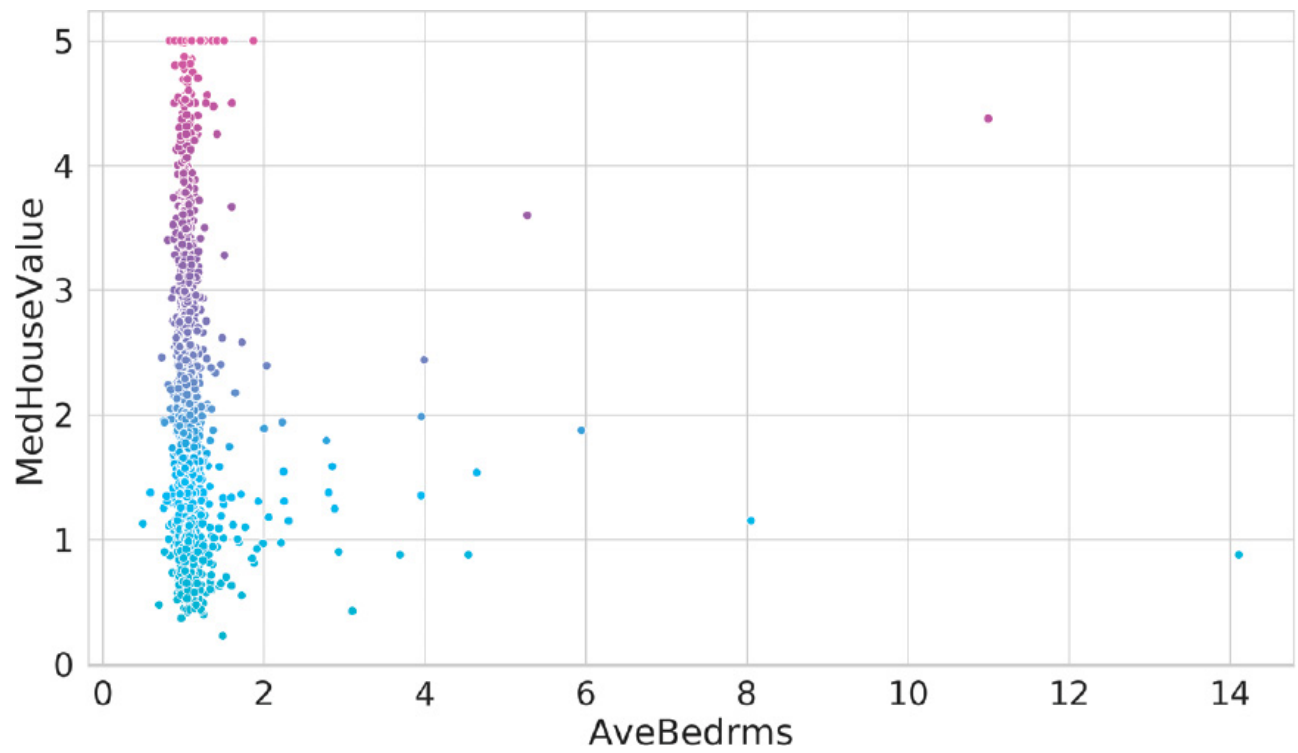
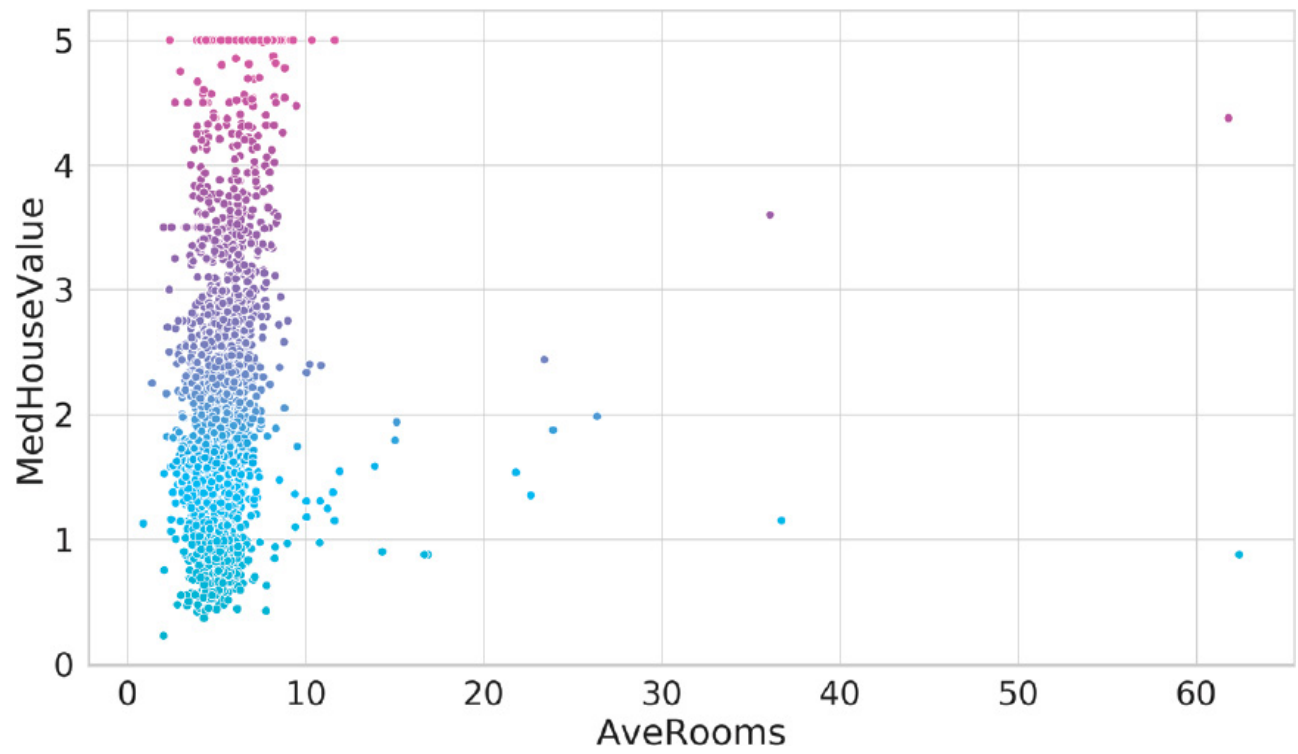
- The graphs showing the latitude and longitude each have two areas of especially significant density. If you search online for the latitude and longitude values where those dense areas appear, you'll see that these represent the greater Los Angeles and greater San Francisco areas where house prices tend to be higher.
- In each graph, there is a horizontal line of dots at the y -axis value 5, which

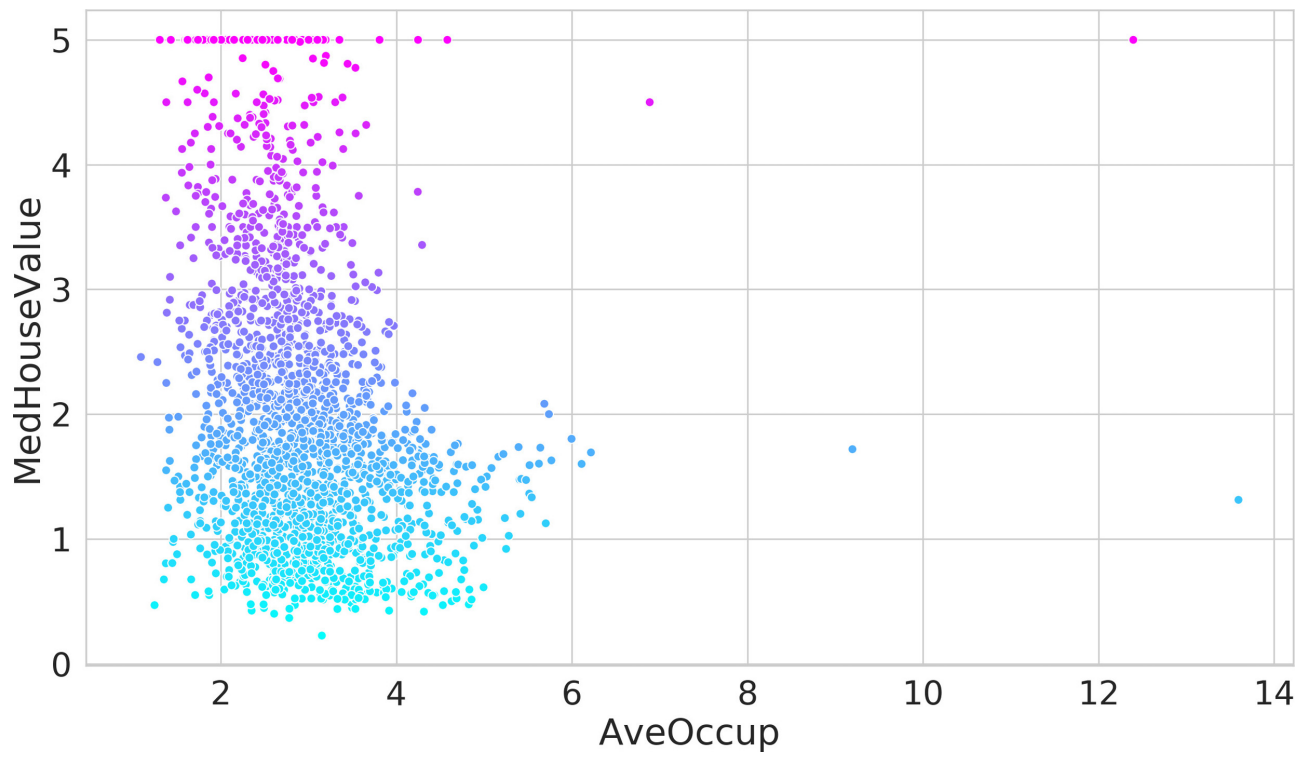
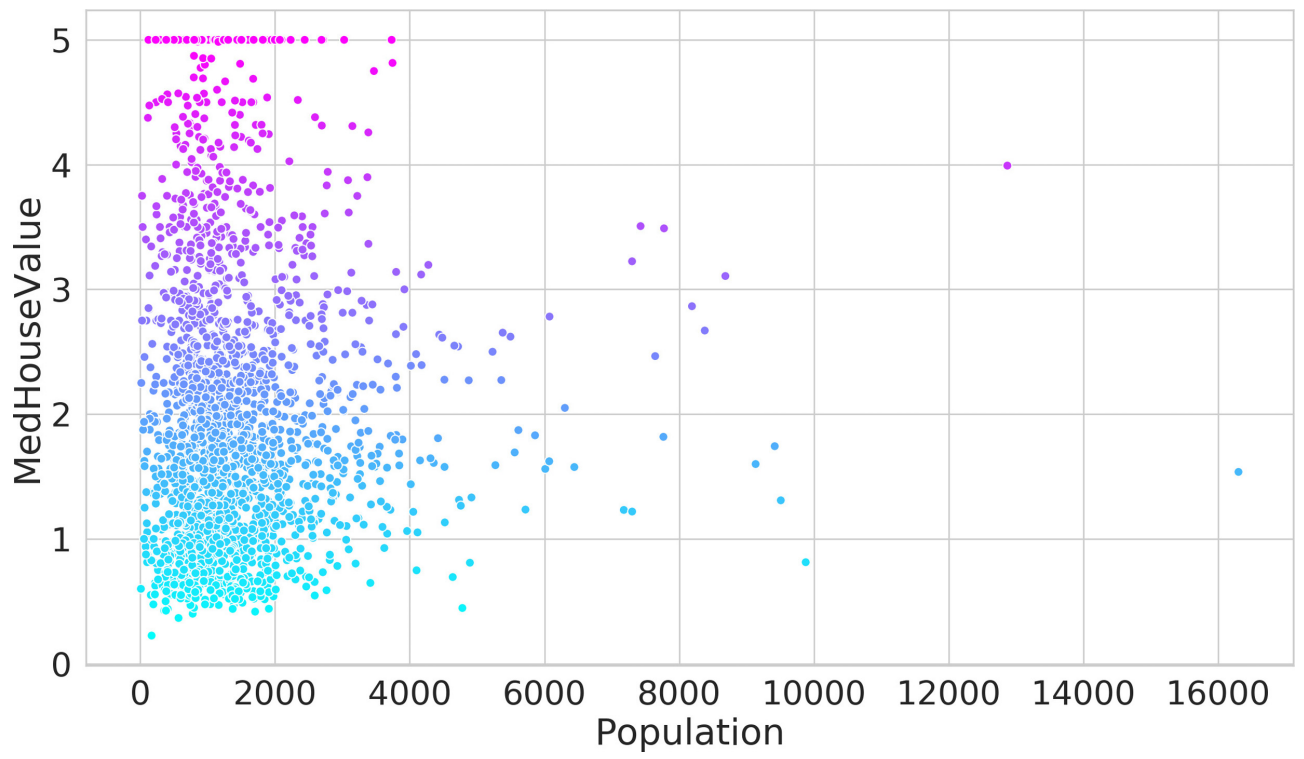
represents the median house value \$500,000. The highest home value that could be chosen on the 1990 census form was “\$500,000 or more.”⁹ So any block group with a median house value over \$500,000 is listed in the dataset as 5. Being able to spot characteristics like this is a compelling reason to do data exploration and visualization.

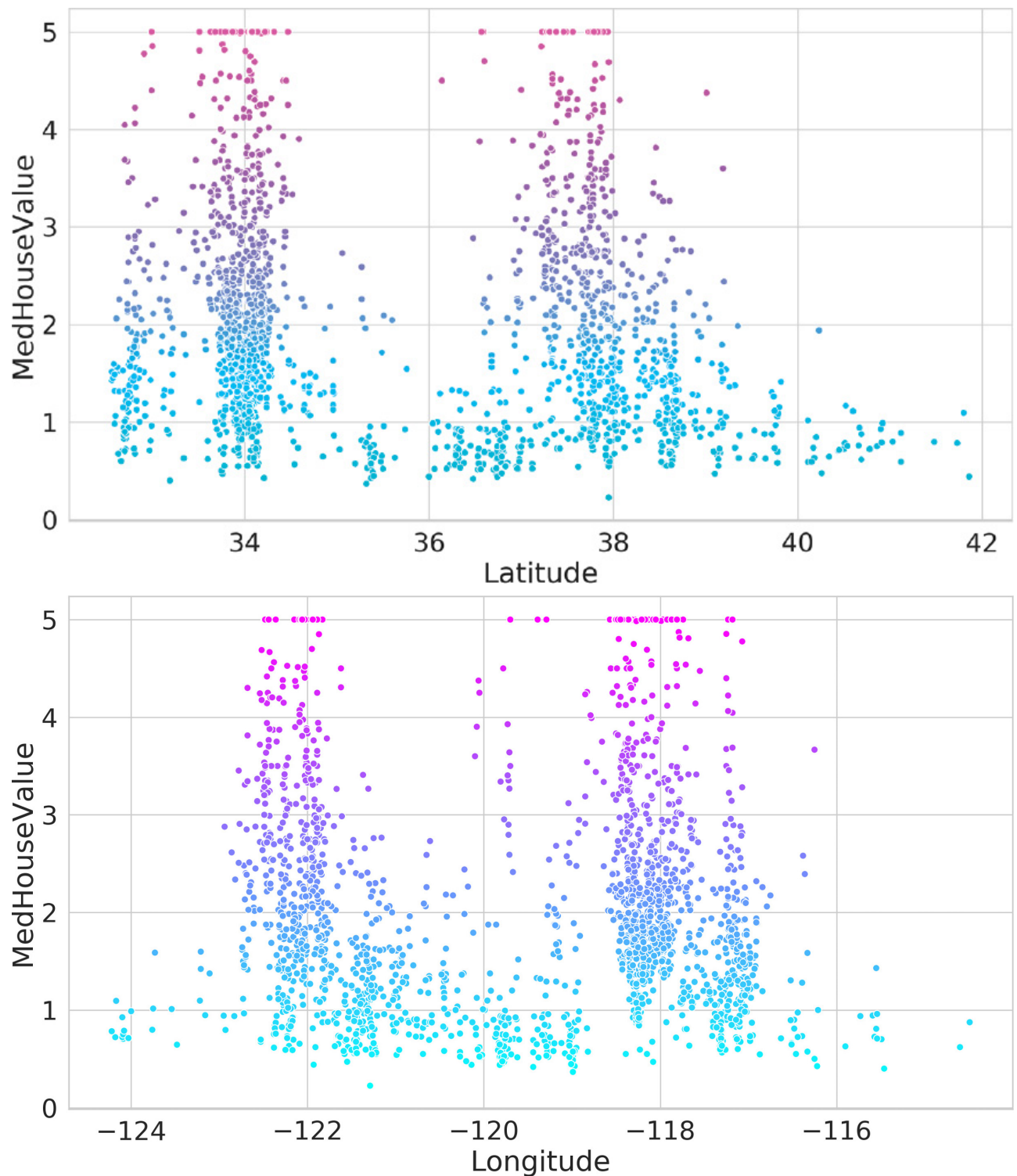
⁹ <https://www.census.gov/prod/1/90dec/cph4/appdxe.pdf>.

- In the `HouseAge` graph, there is a vertical line of dots at the `x`-axis value 52. The highest home age that could be chosen on the 1990 census form was 52, so any block group with a median house age over 52 is listed in the dataset as 52.









14.5.4 Splitting the Data for Training and Testing

Once again, to prepare for training and testing the model, let's break the data into training and testing sets using the `train_test_split` function then check their sizes:

[lick here to view code image](#)

```
In [21]: from sklearn.model_selection import train_test_split

In [22]: X_train, X_test, y_train, y_test = train_test_split(
...:     california.data, california.target, random_state=11)
```



```
...:

In [23]: X_train.shape
Out[23]: (15480, 8)

In [24]: X_test.shape
Out[24]: (5160, 8)
```

We used `train_test_split`'s keyword argument `random_state` to seed the random number generator for reproducibility.

14.5.5 Training the Model

Next, we'll train the model. By default, a `LinearRegression` estimator uses *all* the features in the dataset's `data` array to perform a multiple linear regression. An error occurs if any of the features are *categorical* rather than numeric. If a dataset contains categorical data, you either must preprocess the categorical features into numerical ones (which you'll do in the next chapter) or must exclude the categorical features from the training process. A benefit of working with scikit-learn's bundled datasets is that they're already in the correct format for machine learning using scikit-learn's models.

As you saw in the previous two snippets, `X_train` and `X_test` each contain 8 columns—one per feature. Let's create a `LinearRegression` estimator and invoke its `fit` method to train the estimator using `X_train` and `y_train`:

[lick here to view code image](#)

```
In [25]: from sklearn.linear_model import LinearRegression

In [26]: linear_regression = LinearRegression()

In [27]: linear_regression.fit(X=X_train, y=y_train)
Out[27]:
LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None,
                 normalize=False)
```

Multiple linear regression produces separate coefficients for each feature (stored in `coef_`) in the dataset and one intercept (stored in `intercept_`):

[lick here to view code image](#)

```
In [28]: for i, name in enumerate(california.feature_names):
...:     print(f'{name:>10}:    {linear_regression.coef_[i]}')
...:
```

```

MedInc: 0.4377030215382206
HouseAge: 0.009216834565797713
AveRooms: -0.10732526637360985
AveBedrms: 0.611713307391811
Population: -5.756822009298454e-06
AveOccup: -0.0033845664657163703
Latitude: -0.419481860964907
Longitude: -0.4337713349874016

In [29]: linear_regression.intercept_
Out[29]: -36.88295065605547

```

For positive coefficients, the median house value *increases* as the feature value *increases*. For negative coefficients, the median house value *decreases* as the feature value *increases*. Note that the population coefficient has a *negative exponent* ($e-06$), so the coefficient's value is actually -0.000005756822009298454 . This is close to zero, so a block group's population apparently has little effect the median house value.

You can use these values with the following equation to make predictions:

$$y = m_1x_1 + m_2x_2 + \dots m_nx_n + b$$

where

- $m_1, m_2, , m_n$ are the feature coefficients,
- b is the intercept,
- $x_1, x_2, , x_n$ are the feature values (that is, the values of the independent variables), and
- y is the predicted value (that is, the dependent variable).

14.5.6 Testing the Model

Now, let's test the model by calling the estimator's `predict` method with the test samples as an argument. As we've done in each of the previous examples, we store the array of predictions in `predicted` and the array of expected values in `expected`:

[lick here to view code image](#)

```

In [30]: predicted = linear_regression.predict(X_test)

```



```
In [31]: expected = y_test
```

Let's look at the first five predictions and their corresponding expected values:

[lick here to view code image](#)

```
In [32]: predicted[:5]
Out[32]: array([1.25396876, 2.34693107, 2.03794745, 1.8701254 , 2.536083

n [33]: expected[:5]
Out[33]: array([0.762, 1.732, 1.125, 1.37 , 1.856])
```

With classification, we saw that the predictions were distinct classes that matched existing classes in the dataset. With regression, it's tough to get exact predictions, because you have continuous outputs. Every possible value of x_1, x_2, x_n in the calculation

$$y = m_1x_1 + m_2x_2 + \dots m_nx_n + b$$

predicts a value.

14.5.7 Visualizing the Expected vs. Predicted Prices

Let's look at the expected vs. predicted median house values for the test data. First, let's create a DataFrame containing columns for the expected and predicted values:

[lick here to view code image](#)

```
In [34]: df = pd.DataFrame()

In [35]: df['Expected'] = pd.Series(expected)

In [36]: df['Predicted'] = pd.Series(predicted)
```

Now let's plot the data as a scatter plot with the expected (target) prices along the x-axis and the predicted prices along the y-axis:

[lick here to view code image](#)

```
In [37]: figure = plt.figure(figsize=(9, 9))

In [38]: axes = sns.scatterplot(data=df, x='Expected', y='Predicted',
...:                             hue='Predicted', palette='cool', legend=False)
```

...:

Next, let's set the x - and y -axes' limits to use the same scale along both axes:

[lick here to view code image](#)

```
In [39]: start = min(expected.min(), predicted.min())

In [40]: end = max(expected.max(), predicted.max())

In [41]: axes.set_xlim(start, end)
Out[41]: (-0.6830978604144491, 7.155719818496834)

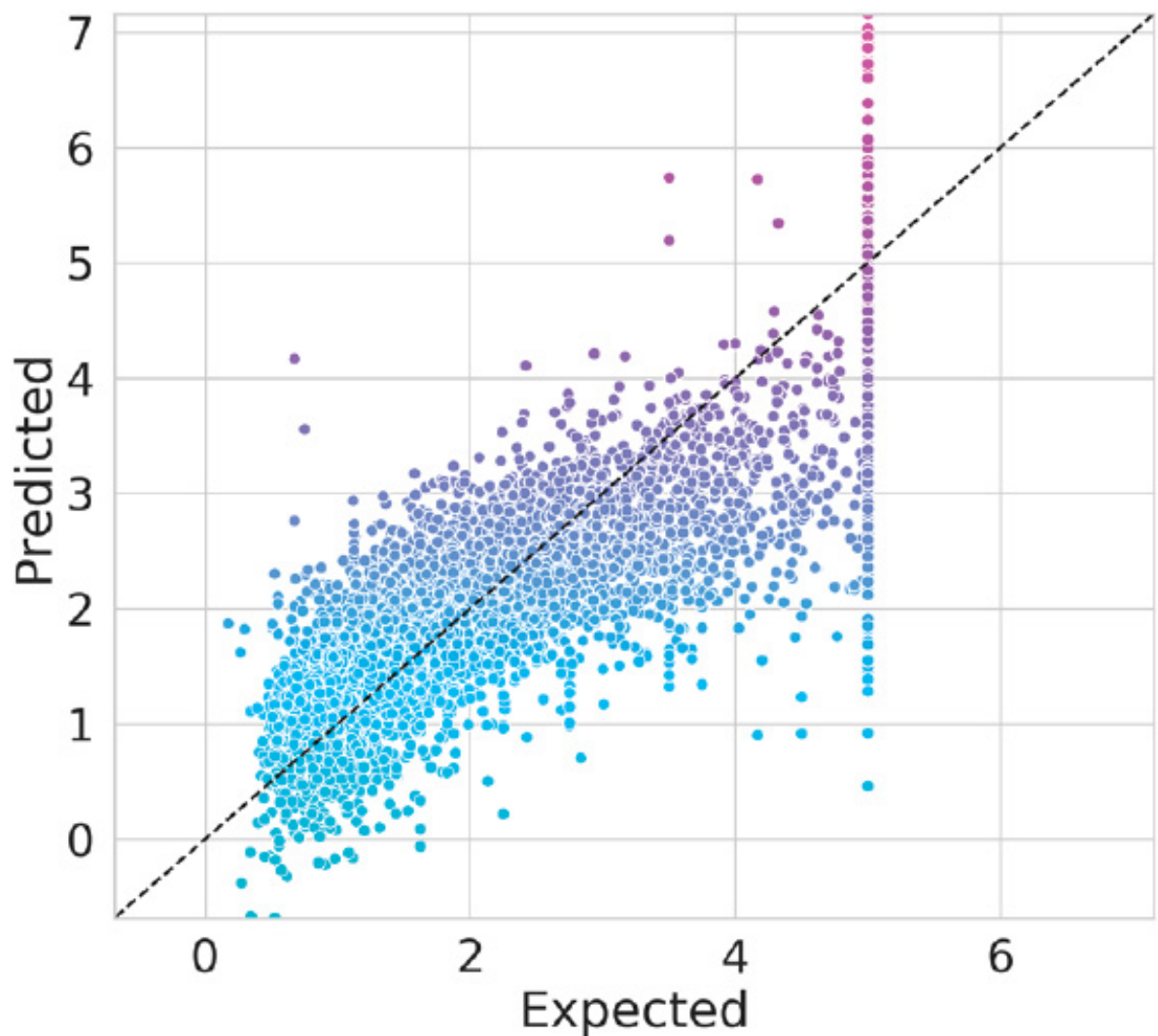
In [42]: axes.set_ylim(start, end)
Out[42]: (-0.6830978604144491, 7.155719818496834)
```

Now, let's plot a line that represents *perfect predictions* (note that this is *not* a regression line). The following snippet displays a line between the points representing the lower-left corner of the graph ($\text{start}, \text{start}$) and the upper-right corner of the graph (end, end). The third argument (`'k--'`) indicates the line's style. The letter `k` represents the color black, and the `--` indicates that plot should draw a dashed line:

[lick here to view code image](#)

```
In [43]: line = plt.plot([start, end], [start, end], 'k--')
```

If every predicted value were to match the expected value, then all the dots would be plotted along the dashed line. In the following diagram, it appears that as the expected median house value increases, more of the predicted values fall below the line. So the model seems to *predict* lower median house values as the *expected* median house value increases.



14.5.8 Regression Model Metrics

Scikit-learn provides many metrics functions for evaluating how well estimators predict results and for comparing estimators to choose the best one(s) for your particular study. These metrics vary by estimator type. For example, the `sklearn.metrics` functions `confusion_matrix` and `classification_report` used in the Digits dataset classification case study are two of many metrics functions specifically for evaluating *classification* estimators.

Among the many metrics for regression estimators is the model's **coefficient of determination**, which is also called the **R^2 score**. To calculate an estimator's R^2 score, call the `sklearn.metrics` module's **`r2_score`** function with the arrays representing the expected and predicted results:

[lick here to view code image](#)

```
In [44]: from sklearn import metrics

In [45]: metrics.r2_score(expected, predicted)
Out[45]: 0.6008983115964333
```

R^2 scores range from 0.0 to 1.0 with 1.0 being the best. An R^2 score of 1.0 indicates that the estimator perfectly predicts the dependent variable's value, given the independent variable(s) value(s). An R^2 score of 0.0 indicates the model cannot make predictions with any accuracy, based on the independent variables' values.

Another common metric for regression models is the **mean squared error**, which

- calculates the difference between each expected and predicted value—this is called the *error*,
- squares each difference and
- calculates the average of the squared values.

To calculate an estimator's mean squared error, call function **mean_squared_error** (from module `sklearn.metrics`) with the arrays representing the expected and predicted results:

[lick here to view code image](#)

```
In [46]: metrics.mean_squared_error(expected, predicted)
Out[46]: 0.5350149774449119
```

When comparing estimators with the mean squared error metric, the one with the value closest to 0 best fits your data. In the next section, we'll run several regression estimators using the California Housing dataset. For the list of scikit-learn's metrics functions by estimator category, see

https://scikit-learn.org/stable/modules/model_evaluation.html

14.5.9 Choosing the Best Model

As we did in the classification case study, let's try several estimators to determine whether any produces better results than the `LinearRegression` estimator. In this example, we'll use the `linear_regression` estimator we already created as well as `ElasticNet`, `Lasso` and `Ridge` regression estimators (all from the `sklearn.linear_model` module). For information about these estimators, see

https://scikit-learn.org/stable/modules/linear_model.html

[lick here to view code image](#)

```
In [47]: from sklearn.linear_model import ElasticNet, Lasso, Ridge

In [48]: estimators = {
...:     'LinearRegression': linear_regression,
...:     'ElasticNet': ElasticNet(),
...:     'Lasso': Lasso(),
...:     'Ridge': Ridge()
...: }
```

Once again, we'll run the estimators using k-fold cross-validation with a `KFold` object and the `cross_val_score` function. Here, we pass to `cross_val_score` the additional keyword argument `scoring='r2'`, which indicates that the function should report the R^2 scores for each fold—again, 1.0 is the best, so it appears that `LinearRegression` and `Ridge` are the best models for this dataset:

[lick here to view code image](#)

```
In [49]: from sklearn.model_selection import KFold, cross_val_score

In [50]: for estimator_name, estimator_object in estimators.items():
...:     kfold = KFold(n_splits=10, random_state=11, shuffle=True)
...:     scores = cross_val_score(estimator=estimator_object,
...:                             X=california.data, y=california.target, cv=kfold,
...:                             scoring='r2')
...:     print(f'{estimator_name:>16}: ' +
...:           f'mean of r2 scores={scores.mean():.3f}')
...:
LinearRegression: mean of r2 scores=0.599
ElasticNet: mean of r2 scores=0.423
Lasso: mean of r2 scores=0.285
Ridge: mean of r2 scores=0.599
```

14.6 CASE STUDY: UNSUPERVISED MACHINE LEARNING, PART 1—DIMENSIONALITY REDUCTION

In our data science presentations, we've focused on getting to know your data.

Unsupervised machine learning and visualization can help you do this by finding patterns and relationships among unlabeled samples.

For datasets like the univariate time series we used earlier in this chapter, visualizing the data is easy. In that case, we had two variables—date and temperature—so we

plotted the data in two dimensions with one variable along each axis. Using Matplotlib, Seaborn and other visualization libraries, you also can plot datasets with three variables using 3D visualizations. But how do you visualize data with more than three dimensions? For example, in the Digits dataset, every sample has 64 features and a target value. In big data, samples can have hundreds, thousands or even millions of features.

To visualize a dataset with many features (that is, many dimensions), we'll first *reduce* the data to two or three dimensions. This requires an unsupervised machine learning technique called **dimensionality reduction**. When you graph the resulting information, you might see patterns in the data that will help you choose the most appropriate machine learning algorithms to use. For example, if the visualization contains *clusters* of points, it might indicate that there are distinct classes of information within the dataset. So a classification algorithm might be appropriate. Of course, you'd first need to determine the class of the samples in each cluster. This might require studying the samples in a cluster to see what they have in common.

Dimensionality reduction also serves other purposes. Training estimators on big data with significant numbers of dimensions can take hours, days, weeks or longer. It's also difficult for humans to think about data with large numbers of dimensions. This is called the **curse of dimensionality**. If the data has closely correlated features, some could be eliminated via dimensionality reduction to improve the training performance. This, however, might reduce the accuracy of the model.

Recall that the Digits dataset is already labeled with 10 classes representing the digits 0–9. Let's ignore those labels and use dimensionality reduction to reduce the dataset's features to two dimensions, so we can visualize the resulting data.

Loading the Digits Dataset

Launch IPython with:

```
ipython --matplotlib
```

then load the dataset:

[lick here to view code image](#)

```
In [1]: from sklearn.datasets import load_digits

In [2]: digits = load_digits()
```

Creating a TSNE Estimator for Dimensionality Reduction

Next, we'll use the **TSNE estimator** (from the `sklearn.manifold` module) to perform dimensionality reduction. This estimator uses an algorithm called t-distributed Stochastic Neighbor Embedding (t-SNE) ⁰ to analyze a dataset's features and reduce them to the specified number of dimensions. We first tried the popular PCA (principal components analysis) estimator but did not like the results we were getting, so we switched to TSNE. We'll show PCA later in this case study.

⁰The algorithm's details are beyond this book's scope. For more information, see <https://scikit-learn.org/stable/modules/manifold.html#t-sne>.

Let's create a TSNE object for reducing a dataset's features to two dimensions, as specified by the keyword argument `n_components`. As with the other estimators we've presented, we used the `random_state` keyword argument to ensure the reproducibility of the “render sequence” when we display the digit clusters:

[lick here to view code image](#)

```
In [3]: from sklearn.manifold import TSNE

In [4]: tsne = TSNE(n_components=2,    random_state=11)
```

Transforming the Digits Dataset's Features into Two Dimensions

Dimensionality reduction in scikit-learn typically involves two steps—training the estimator with the dataset, then using the estimator to transform the data into the specified number of dimensions. These steps can be performed separately with the TSNE methods `fit` and `transform`, or they can be performed in one statement using the `fit_transform` method: ¹

¹Every call to `fit_transform` trains the estimator. If you intend to reuse the estimator to reduce the dimensions of samples multiple times, use `fit` to once train the estimator, then use `transform` to perform the reductions. We'll use this technique with PCA later in this case study.

[lick here to view code image](#)

```
In [5]: reduced_data = tsne.fit_transform(digits.data)
```

TSNE's `fit_transform` method takes some time to train the estimator then perform the reduction. On our system, this took about 20 seconds. When the method completes its task, it returns an array with the same number of rows as `digits.data`, but only two columns. You can confirm this by checking `reduced_data`'s shape:

[lick here to view code image](#)

```
In [6]: reduced_data.shape
Out[6]: (1797, 2)
```

Visualizing the Reduced Data

Now that we've reduced the original dataset to only two dimensions, let's use a scatter plot to display the data. In this case, rather than Seaborn's `scatterplot` function, we'll use Matplotlib's **`scatter` function**, because it returns a collection of the plotted items. We'll use that feature in a second scatter plot momentarily:

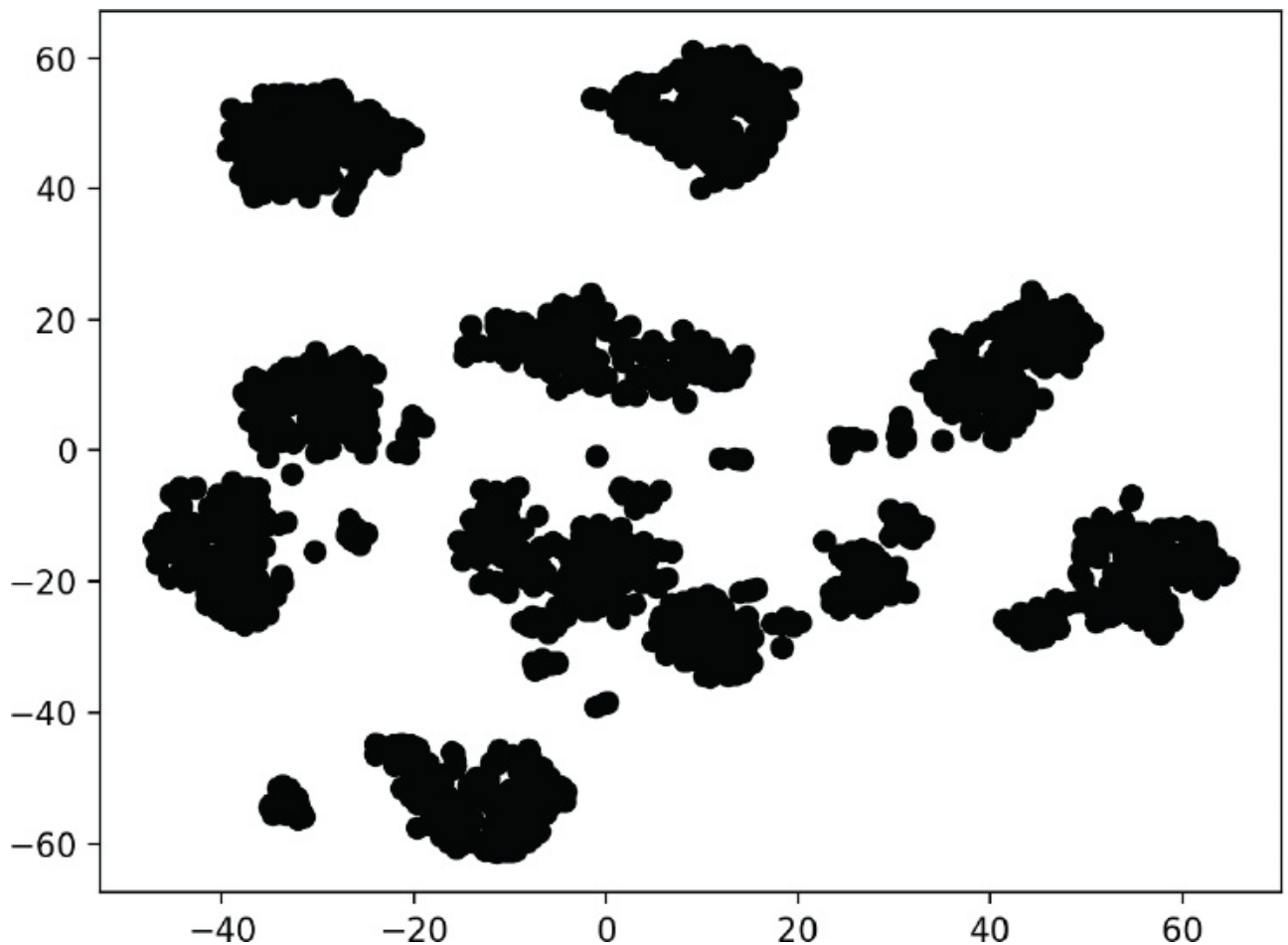
[lick here to view code image](#)

```
In [7]: import matplotlib.pyplot as plt

In [8]: dots = plt.scatter(reduced_data[:, 0], reduced_data[:, 1],
...:                       c='black')
...:
```

Function `scatter`'s first two arguments are `reduced_data`'s columns (0 and 1) containing the data for the x - and y -axes. The keyword argument `c='black'` specifies the color of the dots. We did not label the axes, because they do not correspond to specific features of the original dataset. The new features produced by the TSNE estimator could be quite different from the dataset's original features.

The following diagram shows the resulting scatter plot. There are clearly *clusters* of related data points, though there appear to be 11 main clusters, rather than 10. There also are “loose” data points that do not appear to be part of specific clusters. Based on our earlier- study of the Digits dataset this makes sense because some digits were difficult to classify.



Visualizing the Reduced Data with Different Colors for Each Digit

Though the preceding diagram shows clusters, we do not know whether all the items in each cluster represent the same digit. If they do not, then the clusters are not helpful. Let's use the known `targets` in the Digits dataset to color all the dots so we can see whether these clusters indeed represent specific digits:

[lick here to view code image](#)

```
In [9]: dots = plt.scatter(reduced_data[:, 0], reduced_data[:, 1],
...:                       c=digits.target, cmap=plt.cm.get_cmap('nipy_spectral_r', 10))
...:
...:
```

In this case, `scatter`'s keyword argument `c=digits.target` specifies that the target values determine the dot colors. We also added the keyword argument

[lick here to view code image](#)

```
cmap=plt.cm.get_cmap('nipy_spectral_r', 10)
```

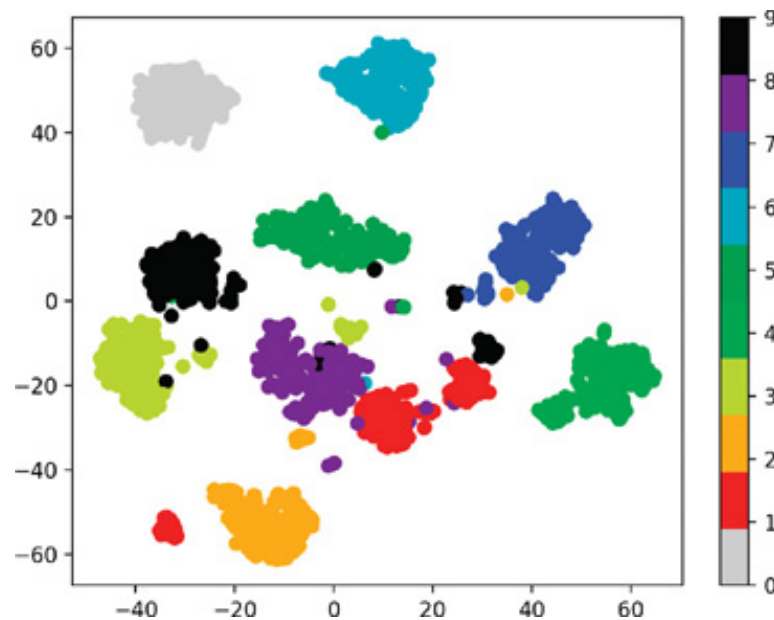
which specifies a color map to use when coloring the dots. In this case, we know we're coloring 10 digits, so we use `get_cmap` method of Matplotlib's `cm` object (from module `matplotlib.pyplot`) to load a color map ('`nipy_spectral_r`') and select 10 distinct colors from the color map.

The following statement adds a color bar key to the right of the diagram so you can see which digit each color represents:

[lick here to view code image](#)

```
In [10]: colorbar = plt.colorbar(dots)
```

Voila! We see 10 clusters corresponding to the digits 0–9. Again, there are a few smaller groups of dots standing alone. Based on this, we might decide that a supervised-learning approach like k-nearest neighbors would work well with this data. As an experiment, you might want to investigate Matplotlib's **Axes3D**, which provides *x*-, *y*- and *z*-axes for plotting in three-dimensional graphs.



14.7 CASE STUDY: UNSUPERVISED MACHINE LEARNING, PART 2—K-MEANS CLUSTERING

In this section, we introduce perhaps the simplest unsupervised machine learning algorithms—**k-means clustering**. This algorithm analyzes *unlabeled* samples and attempts to place them in clusters that appear to be related. The *k* in “k-means” represents the number of clusters you’d like to see imposed on your data.

The algorithm organizes samples into the number of clusters you specify in advance, using distance calculations similar to the k-nearest neighbors clustering algorithm.

Each cluster of samples is grouped around a **centroid**—the cluster’s center point. Initially, the algorithm chooses k centroids at random from the dataset’s samples. Then the remaining samples are placed in the cluster whose centroid is the closest. The centroids are iteratively recalculated and the samples re-assigned to clusters until, for all clusters, the distances from a given centroid to the samples in its cluster are minimized. The algorithm’s results are:

- a one-dimensional array of labels indicating the cluster to which each sample belongs and
- a two-dimensional array of centroids representing the center of each cluster.

Iris Dataset

We’ll work with the popular **Iris dataset** ² bundled with scikit-learn, which is commonly analyzed with both classification and clustering. Although this dataset is labeled, we’ll ignore those labels here to demonstrate clustering. Then, we’ll use the labels to determine how well the k-means algorithm clustered the samples.

²Fisher, R.A., The use of multiple measurements in taxonomic problems, *Annual Eugenics*, 7, Part II, 179-188 (1936); also in *Contributions to Mathematical Statistics* (John Wiley, NY, 1950).

The Iris dataset is referred to as a “toy dataset” because it has only 150 samples and four features. The dataset describes 50 samples for each of three *Iris* flower species—*Iris setosa*, *Iris versicolor* and *Iris virginica*. Photos of these are shown below. Each sample’s features are the sepal length, sepal width, petal length and petal width, all measured in centimeters. The *sepals* are the larger outer parts of each flower that protect the smaller inside *petals* before the flower buds bloom.



Iris setosa:

[https://commons.wikimedia.org/wiki/File:Wild_iris_KEFJ_\(9025144383\).jpg](https://commons.wikimedia.org/wiki/File:Wild_iris_KEFJ_(9025144383).jpg).

Credit: Courtesy of Nation Park services.



Iris versicolor: https://commons.wikimedia.org/wiki/Iris_versicolor#/media/

File:IrisVersicolor-FoxRoost-Newfoundland.jpg.

Credit: Courtesy of Jefficus,

[https://commons.wikimedia.org/w/index.php?
title=User:Jefficus&action=edit&redlink=1](https://commons.wikimedia.org/w/index.php?title=User:Jefficus&action=edit&redlink=1)



Iris virginica: https://commons.wikimedia.org/wiki/File:IMG_7911-Iris_virginica.jpg.

Credit: Christer T Johansson.

14.7.1 Loading the Iris Dataset

Launch IPython with `ipython --matplotlib`, then use the `sklearn.datasets` module's **`load_iris` function** to get a Bunch containing the dataset:

[lick here to view code image](#)

```
In [1]: from sklearn.datasets import load_iris

In [2]: iris = load_iris()
```

The Bunch's `DESCR` attribute indicates that there are 150 samples (Number of Instances), each with four features (Number of Attributes). There are no missing values in this dataset. The dataset classifies the samples by labeling them with the integers 0, 1 and 2, representing *Iris setosa*, *Iris versicolor* and *Iris virginica*, respectively. We'll ignore the labels and let the k-means clustering algorithm try to

determine the samples' classes. We show some key DESCR information in bold.:

[lick here to view code image](#)

```
n [3]: print(iris.DESCR)
.. _iris_dataset:

Iris plants dataset
-----

**Data Set Characteristics:**

: Number of Instances: 150 (50 in each of three classes)
: Number of Attributes: 4 numeric, predictive attributes and the class
: Attribute Information:
    - sepal length in cm
    - sepal width in cm
    - petal length in cm
    - petal width in cm
    - class:
        - Iris-Setosa
        - Iris-Versicolour
        - Iris-Virginica

: Summary Statistics:

=====  =====  =====  =====  =====  =====
              Min    Max    Mean     SD    Class    Correlation
=====  =====  =====  =====  =====  =====
sepal length:  4.3    7.9    5.84    0.83    0.7826
sepal width:   2.0    4.4    3.05    0.43   -0.4194
petal length:  1.0    6.9    3.76    1.76    0.9490    (high!)
petal width:   0.1    2.5    1.20    0.76    0.9565    (high!)
=====  =====  =====  =====  =====  =====

: Missing Attribute Values: None
: Class Distribution: 33.3% for each of 3 classes.
: Creator: R.A. Fisher
: Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
: Date: July, 1988
```

The famous Iris database, first used by Sir R.A. Fisher. The dataset is available from the UCI Machine Learning Repository, which has two wrong data points.

This is perhaps the **best known database to be found in the pattern recognition literature**. Fisher's paper is a classic in the field and is referenced frequently to this day. (See Duda & Hart, for example.) The class refers to a type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.

```
.. topic:: References
```

- Fisher, R.A. "The use of multiple measurements in taxonomic problems"
Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis.
(Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
- Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.
- Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions on Information Theory, May 1972, 431-433.
- See also: 1988 MLC Proceedings, 54-64. Cheeseman et al's AUTOCLASS II conceptual clustering system finds 3 classes in the data.
- Many, many more ...

checking the Numbers of Samples, Features and Targets

You can confirm the number of samples and features per sample via the data array's shape, and you can confirm the number of targets via the target array's shape:

[lick here to view code image](#)

```
In [4]: iris.data.shape
Out[4]: (150, 4)

In [5]: iris.target.shape
Out[5]: (150,)
```

The array `target_names` contains the names for the target array's numeric labels —`dtype='<U10'` indicates that the elements are strings with a maximum of 10 characters:

[lick here to view code image](#)

```
In [6]: iris.target_names
Out[6]: array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

The array `feature_names` contains a list of string names for each column in the data array:

[lick here to view code image](#)


```
In [7]: iris.feature_names
Out[7]:
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

14.7.2 Exploring the Iris Dataset: Descriptive Statistics with Pandas

Let's use a `DataFrame` to explore the Iris dataset. As we did in the California Housing case study, let's set the pandas options for formatting the column-based outputs:

[lick here to view code image](#)

```
In [8]: import pandas as pd

In [9]: pd.set_option('max_columns', 5)

In [10]: pd.set_option('display.width', None)
```

Create a `DataFrame` containing the data array's contents, using the contents of the `feature_names` array as the column names:

[lick here to view code image](#)

```
In [11]: iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

Next, add a column containing each sample's species name. The list comprehension in the following snippet uses each value in the `target` array to look up the corresponding species name in the `target_names` array:

[lick here to view code image](#)

```
In [12]: iris_df['species'] = [iris.target_names[i] for i in iris.targ
```

et's use pandas' to look at a few samples. Once again notice that pandas displays a `\` to the right of the column heads to indicate that there are more columns displayed below:

[lick here to view code image](#)

```
In [13]: iris_df.head()
Out[13]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
0	5.1	3.5	1.4	
1	4.9	3.0	1.4	
2	4.7	3.2	1.3	
3	4.6	3.1	1.5	
4	5.0	3.6	1.4	

	petal width (cm)	species
0	0.2	setosa
1	0.2	setosa
2	0.2	setosa
3	0.2	setosa
4	0.2	setosa

Let's calculate some descriptive statistics for the numerical columns:

[lick here to view code image](#)

```
In [14]: pd.set_option('precision', 2)

In [15]: iris_df.describe()
Out[15]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.00	150.00	150.00	
mean	5.84	3.06	3.76	
std	0.83	0.44	1.77	
min	4.30	2.00	1.00	
25%	5.10	2.80	1.60	
50%	5.80	3.00	4.35	
75%	6.40	3.30	5.10	
max	7.90	4.40	6.90	

	petal width (cm)
count	150.00
mean	1.20
std	0.76
min	0.10
25%	0.30
50%	1.30
75%	1.80
max	2.50

Calling the `describe` method on the `'species'` column confirms that it contains three unique values. Here, we know in advance of working with this data that there are three classes to which the samples belong, though this is not always the case in unsupervised machine learning.

[lick here to view code image](#)

```
In [16]: iris_df['species'].describe()
Out[16]:
count          150
unique           3
top            setosa
freq            50
Name: species, dtype: object
```

14.7.3 Visualizing the Dataset with a Seaborn `pairplot`

Let's visualize the features in this dataset. One way to learn more about your data is to see how the features relate to one another. The dataset has four features. We cannot graph one against the other three in a single graph. However, we can plot pairs of features against one another. Snippet [20] uses Seaborn function `pairplot` to create a grid of graphs plotting each feature against itself and the other specified features:

[lick here to view code image](#)

```
In [17]: import seaborn as sns

In [18]: sns.set(font_scale=1.1)

In [19]: sns.set_style('whitegrid')

In [20]: grid = sns.pairplot(data=iris_df, vars=iris_df.columns[0:4],
...:                        hue='species')
...:
```

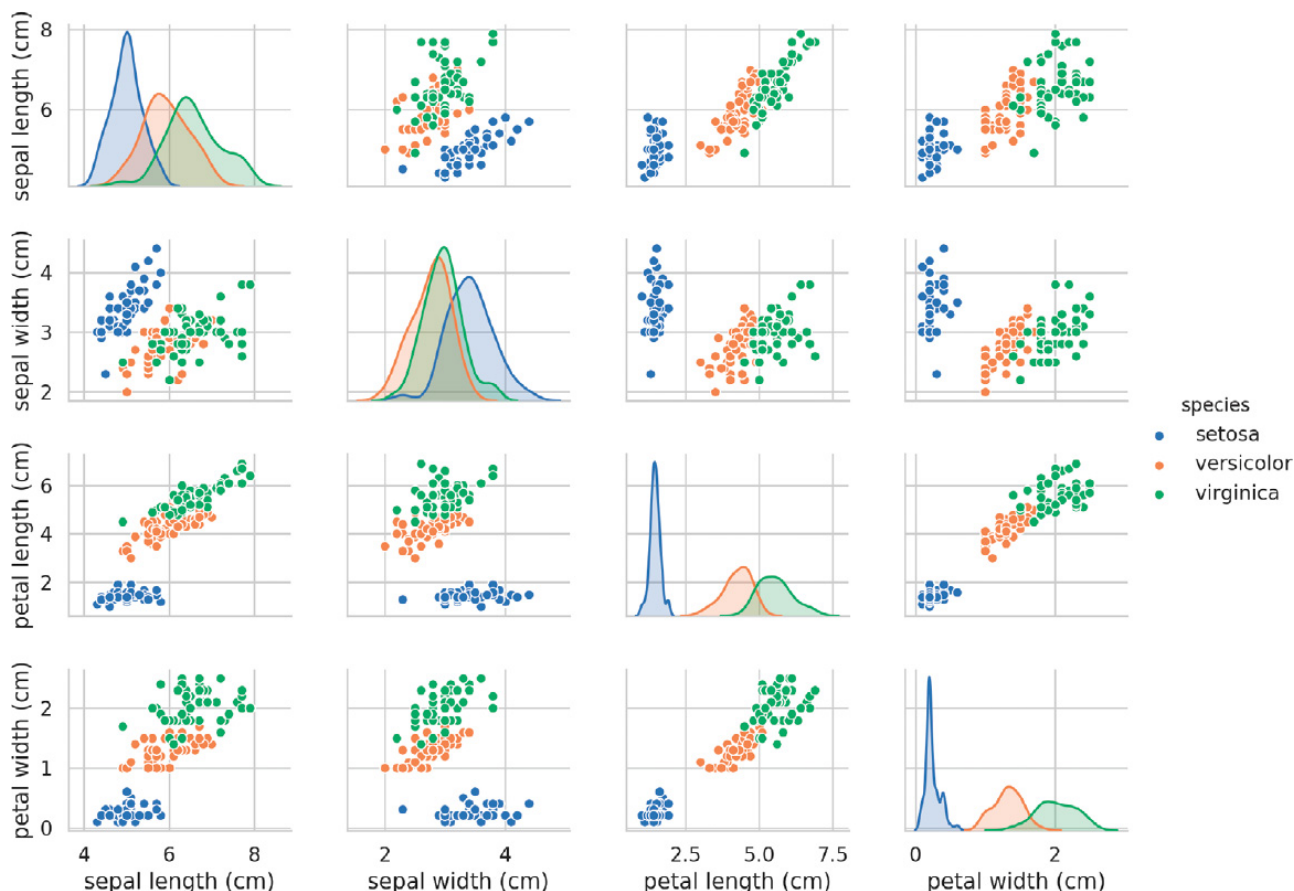
The keyword arguments are:

- `data`—The DataFrame³ containing the data to plot.

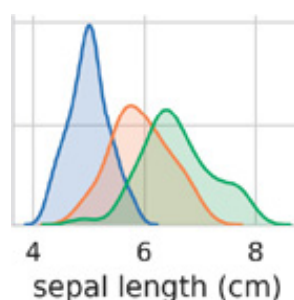
³This also may be a two-dimensional array or list.

- `vars`—A sequence containing the names of the variables to plot. For a DataFrame, these are the names of the columns to plot. Here, we use the first four DataFrame columns, representing the sepal length, sepal width, petal length and petal width, respectively.
- `hue`—The DataFrame column that's used to determine colors of the plotted data. In this case, we'll color the data by *Iris* species.

he preceding call to `pairplot` produces the following 4-by-4 grid of graphs:



The graphs along the top-left-to-bottom-right diagonal, show the **distribution** of just the feature plotted in that column, with the range of values (left-to-right) and the number of samples with those values (top-to-bottom). Consider the sepal-length distributions:



The tallest shaded area indicates that the range of sepal length values (shown along the x -axis) for *Iris setosa* is approximately 4–6 centimeters and that most *Iris setosa* samples are in the middle of that range (approximately 5 centimeters). Similarly, the rightmost shaded area indicates that the range of sepal length values for *Iris virginica* is approximately 4–8.5 centimeters and that the majority of *Iris virginica* samples have sepal length values between 6 and 7 centimeters.

The other graphs in a column show scatter plots of the other features against the feature on the x -axis. In the first column, the other three graphs plot the sepal width, petal length and petal width, respectively, along the y -axis and the sepal length along

the x -axis.

When you run this code, you'll see in the full color output that using separate colors for each *Iris* species shows how the species relate to one another on a feature-by-feature basis. Interestingly, all the scatter plots clearly separate the *Iris setosa* blue dots from the other species' orange and green dots, indicating that *Iris setosa* is indeed in a "class by itself." We also can see that the other two species can sometimes be confused with one another, as indicated by the overlapping orange and green dots. For example, if you look at the scatter plot for sepal width vs. sepal length, you'll see the *Iris versicolor* and *Iris virginica* dots are intermixed. This indicates that it would be difficult to distinguish between these two species if we had only the sepal measurements available to us.

Displaying the `pairplot` in One Color

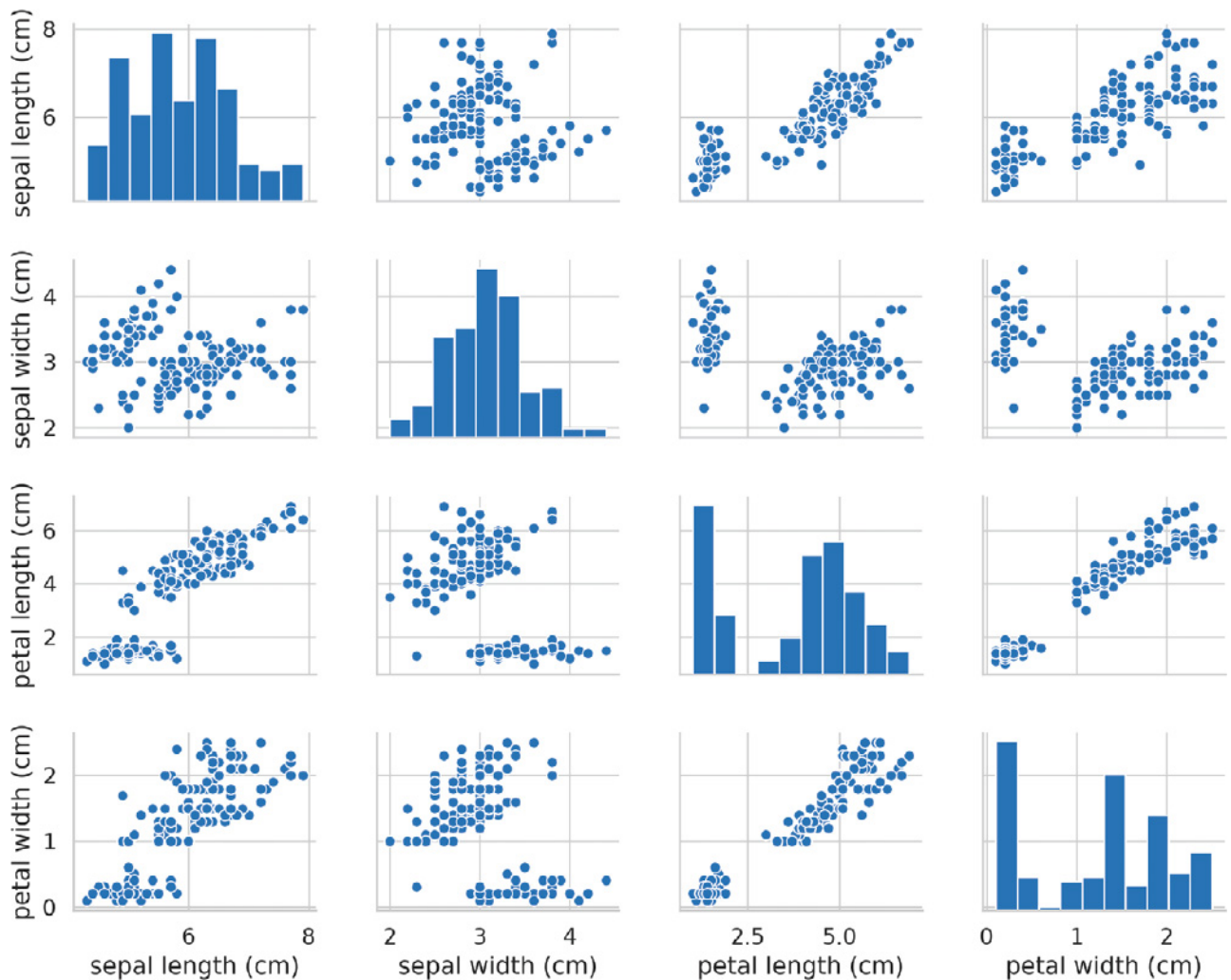
If you remove the `hue` keyword argument, `pairplot` function uses only one color to plot all the data because it does not know how to distinguish the species:

[lick here to view code image](#)

```
In [21]: grid = sns.pairplot(data=iris_df, vars=iris_df.columns[0:4])
```

As you can see in the resulting pair plot on the next page, in this case, the graphs along the diagonal are histograms showing the distributions of all the values for that feature, regardless of the species. As you study each scatter plot, it appears that there may be only *two* distinct clusters, even though for this dataset we know there are *three* species. If you do not know the number of clusters in advance, you might ask a **domain expert** who is thoroughly familiar with the data. Such a person might know that there are three species in the dataset, which would be valuable information as we try to perform machine learning on the data.

The `pairplot` diagrams work well for a *small number of features* or a subset of features so that you have a small number of rows and columns, and for a relatively small number of samples so you can see the data points. As the number of features and samples increases, each scatter plot quickly becomes too small to read. For larger datasets, you may choose to plot a subset of the features and potentially a randomly selected subset of the samples to get a feel for your data.



14.7.4 Using a `KMeans` Estimator

In this section, we'll use k-means clustering via scikit-learn's `KMeans` estimator (from the `sklearn.cluster` module) to place each sample in the Iris dataset into a cluster. As with the other estimators you've used, the `KMeans` estimator hides from you the algorithm's complex mathematical details, making it straightforward to use.

Creating the Estimator

Let's create the `KMeans` object:

[lick here to view code image](#)

```
In [22]: from sklearn.cluster import KMeans

In [23]: kmeans = KMeans(n_clusters=3,    random_state=11)
```

The keyword argument `n_clusters` specifies the k-means clustering algorithm's hyperparameter k , which `KMeans` requires to calculate the clusters and label each sample. When you train a `KMeans` estimator, the algorithm calculates for each cluster a centroid representing the cluster's center data point.

The default value for the `n_clusters` parameter is 8. Often, you'll rely on domain experts knowledgeable about the data to help choose an appropriate k value. However, with hyperparameter tuning, you can estimate the appropriate k , as we'll do later. In this case, we know there are three species, so we'll use `n_clusters=3` to see how well `KMeans` does in labeling the Iris samples. Once again, we used the `random_state` keyword argument for reproducibility.

Fitting the Model

Next, we'll train the estimator by calling the `KMeans` object's `fit` method. This step performs the k-means algorithm discussed earlier:

[lick here to view code image](#)

```
In [24]: kmeans.fit(iris.data)
Out[24]:
KMeans(algorithm='auto', copy_x=True, init='k-means++', max_iter=300,
       n_clusters=3, n_init=10, n_jobs=None, precompute_distances='auto',
       random_state=11, tol=0.0001, verbose=0)
```

As with the other estimator's, the `fit` method returns the estimator object and IPython displays its string representation. You can see the `KMeans` default arguments at:

```
https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.h
```

hen the training completes, the `KMeans` object contains:

- A `labels_` array with values from 0 to `n_clusters - 1` (in this example, 0–2), indicating the clusters to which the samples belong.
- A `cluster_centers_` array in which each row represents a centroid.

Comparing the Computer Cluster Labels to the Iris Dataset's Target Values

Because the Iris dataset is labeled, we can look at its `target` array values to get a sense of how well the k-means algorithm clustered the samples for the three *Iris* species. With unlabeled data, we'd need to depend on a domain expert to help evaluate whether the predicted classes make sense.

In this dataset, the first 50 samples are *Iris setosa*, the next 50 are *Iris versicolor*, and the last 50 are *Iris virginica*. The Iris dataset's `target` array represents these with the values 0–2. If the `KMeans` estimator chose the clusters perfectly, then each group of 50 elements in the estimator's `labels_` array should have a distinct label. As you study the results below, note that the `KMeans` estimator uses the values 0 through $k - 1$ to label clusters, but these are not related to the Iris dataset's `target` array.

Let's use slicing to see how each group of 50 Iris samples was clustered. The following snippet shows that the first 50 samples were all placed in cluster 1:

[lick here to view code image](#)

[illegible]

The next 50 samples should be placed into a second cluster. The following snippet shows that most were placed in cluster 0, but two samples were placed in cluster 2:

[lick here to view code image](#)

[illegible]

Similarly, the last 50 samples should be placed into a third cluster. The following snippet shows that many of these samples were placed in cluster 2, but 14 of the samples were placed in cluster 0, indicating that the algorithm thought they belonged to a different cluster:

[lick here to view code image](#)

```
In [27]: print(kmeans.labels_[100:150])
```

```
[2 0 2 2 2 2 0 2 2 2 2 2 0 0 2 2 2 2 0 2 0 2 0 2 2 0 0 2 2 2 2 0 2  
 2 2 0 2 2 2 0 2 2 2 0 2 2 0]
```

The results of these three snippets confirm what we saw in the `pairplot` diagrams earlier in this section—that *Iris setosa* is “in a class by itself” and that there is some

confusion between *Iris versicolor* and *Iris virginica*.

14.7.5 Dimensionality Reduction with Principal Component Analysis

Next, we'll use the **PCA estimator** (from the `sklearn.decomposition` module) to perform dimensionality reduction. This estimator uses an algorithm called principal component analysis ⁴ to analyze a dataset's features and reduce them to the specified number of dimensions. For the Iris dataset, we first tried the TSNE estimator shown earlier but did not like the results we were getting. So we switched to PCA for the following demonstration.

⁴The algorithms details are beyond this books scope. For more information, see <https://scikit-learn.org/stable/modules/decomposition.html#pca>.

Creating the PCA Object

Like the TSNE estimator, a PCA estimator uses the keyword argument `n_components` to specify the number of dimensions:

[lick here to view code image](#)

```
In [28]: from sklearn.decomposition import PCA

In [29]: pca = PCA(n_components=2, random_state=11)
```

Transforming the Iris Dataset's Features into Two Dimensions

Let's train the estimator and produce the reduced data by calling the PCA estimator's methods **fit** and **transform** methods:

[lick here to view code image](#)

```
In [30]: pca.fit(iris.data)
Out[30]:
PCA(copy=True, iterated_power='auto', n_components=2, random_state=11,
     svd_solver='auto', tol=0.0, whiten=False)

In [31]: iris_pca = pca.transform(iris.data)
```

When the method completes its task, it returns an array with the same number of rows as `iris.data`, but only two columns. Let's confirm this by checking `iris_pca`'s

shape:

[lick here to view code image](#)

```
In [32]: iris_pca.shape
Out[32]: (150, 2)
```

Note that we *separately* called the PCA estimator's `fit` and `transform` methods, rather than `fit_transform`, which we used with the TSNE estimator. In this example, we're going to *reuse* the trained estimator (produced with `fit`) to perform a second `transform` to reduce the cluster centroids from four dimensions to two. This will enable us to plot the centroid locations on each cluster.

Visualizing the Reduced Data

Now that we've reduced the original dataset to only two dimensions, let's use a scatter plot to display the data. In this case, we'll use Seaborn's `scatterplot` function. First, let's transform the reduced data into a `DataFrame` and add a `species` column that we'll use to determine the dot colors:

[lick here to view code image](#)

```
In [33]: iris_pca_df = pd.DataFrame(iris_pca,
...:                                columns=['Component1', 'Component2'])
...:
In [34]: iris_pca_df['species'] = iris_df.species
```

Next, let's scatterplot the data in Seaborn:

[lick here to view code image](#)

```
In [35]: axes = sns.scatterplot(data=iris_pca_df, x='Component1',
...:                             y='Component2', hue='species', legend='brief',
...:                             palette='cool')
...:
```

Each centroid in the `KMeans` object's `cluster_centers_` array has the *same* number of features as the original dataset (four in this case). To plot the centroids, we must reduce their dimensions. You can think of a centroid as the “average” sample in its cluster. So each centroid should be transformed using the same PCA estimator we used

to reduce the other samples in that cluster:

[lick here to view code image](#)

```
In [36]: iris_centers = pca.transform(kmeans.cluster_centers_)
```

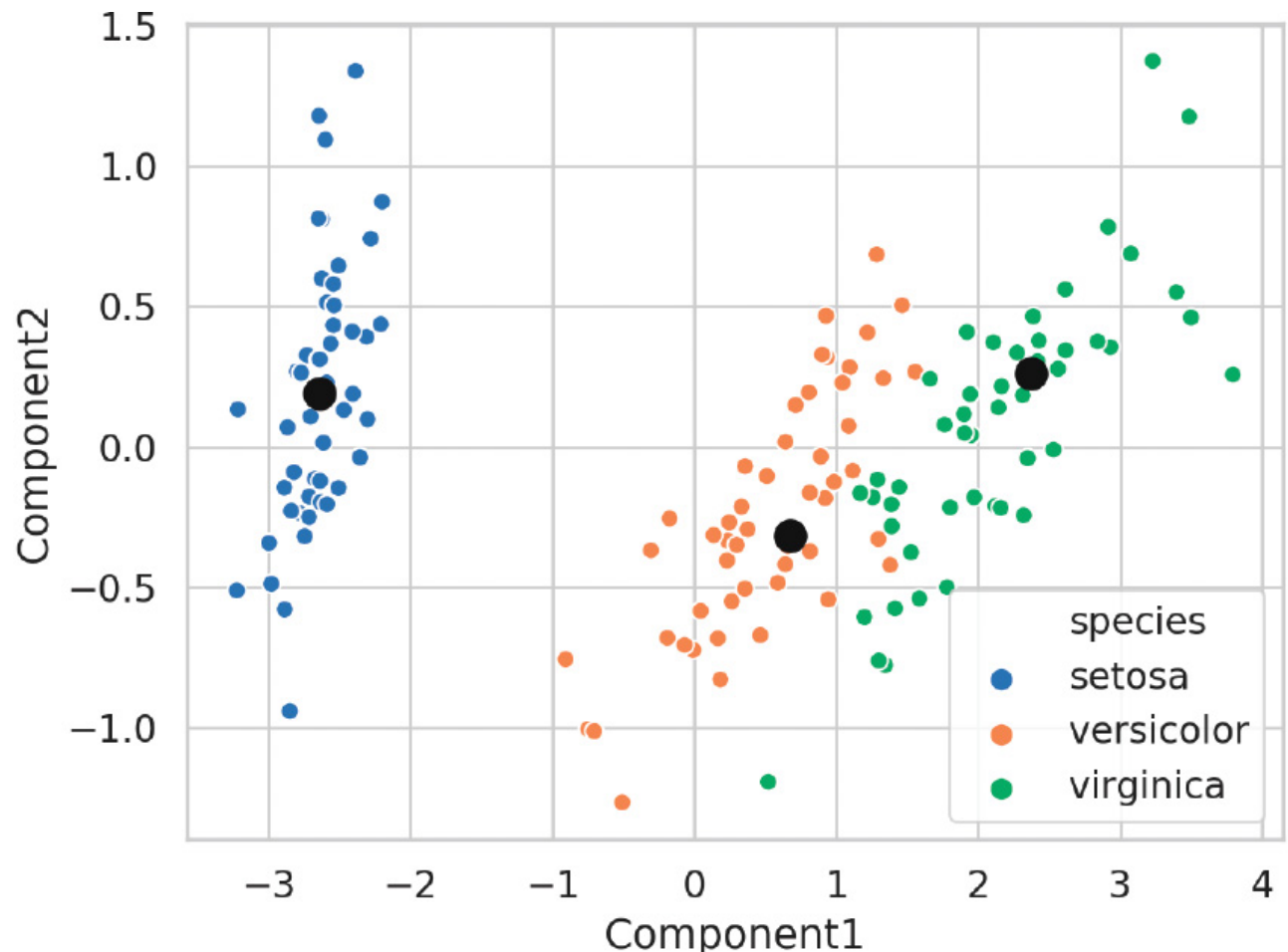
Now, we'll plot the centroids of the three clusters as larger black dots. Rather than transform the `iris_centers` array into a `DataFrame` first, let's use Matplotlib's `scatter` function to plot the three centroids:

[lick here to view code image](#)

```
In [37]: import matplotlib.pyplot as plt

In [38]: dots = plt.scatter(iris_centers[:,0], iris_centers[:,1],
...:                        s=100, c='k')
...:
...:
```

The keyword argument `s=100` specifies the size of the plotted points, and the keyword argument `c='k'` specifies that the points should be displayed in black.



14.7.6 Choosing the Best Clustering Estimator

As we did in the classification and regression case studies, let's run multiple clustering algorithms and see how well they cluster the three species of Iris flowers. Here we'll attempt to cluster the Iris dataset's samples using the `kmeans` object we created earlier ⁵ and objects of scikit-learn's `DBSCAN`, `MeanShift`, `SpectralClustering` and `AgglomerativeClustering` estimators. Like `KMeans`, you specify the number of clusters in advance for the `SpectralClustering` and `AgglomerativeClustering` estimators:

⁵Were running `KMeans` here on the *small* Iris dataset. If you experience performance problems with `KMeans` on larger datasets, consider using the `MiniBatchKMeans` estimator. The scikit-learn documentation indicates that `MiniBatchKMeans` is faster on large datasets and the results are almost as good.

[lick here to view code image](#)

```
In [39]: from sklearn.cluster import DBSCAN, MeanShift,\n...:      SpectralClustering, AgglomerativeClustering\n\nIn [40]: estimators = {\n...:     'KMeans': kmeans,\n...:     'DBSCAN': DBSCAN(),\n...:     'MeanShift': MeanShift(),\n...:     'SpectralClustering': SpectralClustering(n_clusters=3),\n...:     'AgglomerativeClustering':\n...:         AgglomerativeClustering(n_clusters=3)\n...: }
```

Each iteration of the following loop calls one estimator's `fit` method with `iris.data` as an argument, then uses NumPy's `unique` function to get the cluster labels and counts for the three groups of 50 samples and displays the results. Recall that for the `DBSCAN` and `MeanShift` estimators, we did *not* specify the number of clusters in advance. Interestingly, `DBSCAN` correctly predicted three clusters (labeled -1, 0 and 1), though it placed 84 of the 100 *Iris virginica* and *Iris versicolor* samples in the same cluster. The `MeanShift` estimator, on the other hand, predicted only two clusters (labeled as 0 and 1), and placed 99 of the 100 *Iris virginica* and *Iris versicolor* samples in the same cluster:

[lick here to view code image](#)

```
In [41]: import numpy as np
```

```

n [42]: for name, estimator in estimators.items():
...:     estimator.fit(iris.data)
...:     print(f'\n{name}:')
...:     for i in range(0, 101, 50):
...:         labels, counts = np.unique(
...:             estimator.labels_[i:i+50], return_counts=True)
...:         print(f'{i}-{i+50}:')
...:         for label, count in zip(labels, counts):
...:             print(f'        label={label}, count={count}')
...:

```

KMeans:

```

0-50:
    label=1, count=50
50-100:
    label=0, count=48
    label=2, count=2
100-150:
    label=0, count=14
    label=2, count=36

```

DBSCAN:

```

0-50:
    label=-1, count=1
    label=0, count=49
50-100:
    label=-1, count=6
    label=1, count=44
100-150:
    label=-1, count=10
    label=1, count=40

```

MeanShift:

```

0-50:
    label=1, count=50
50-100:
    label=0, count=49
    label=1, count=1
100-150:
    label=0, count=50

```

SpectralClustering:

```

0-50:
    label=2, count=50
50-100:
    label=1, count=50
100-150:
    label=0, count=35
    label=1, count=15

```

AgglomerativeClustering:

```

0-50:
    label=1, count=50

```

```
50-100:  
    label=0, count=49  
    label=2, count=1  
100-150:  
    label=0, count=15  
    label=2, count=35
```

Though these algorithms label every sample, the labels simply indicate the clusters. What do you do with the cluster information once you have it? If your goal is to use the data in supervised machine learning, typically you'd study the samples in each cluster to try to determine how they're related and label them accordingly. As we'll see in the next chapter, unsupervised learning is commonly used in deep-learning applications. Some examples of unlabeled data processed with unsupervised learning include tweets from Twitter, Facebook posts, videos, photos, news articles, customers' product reviews, viewers' movie reviews and more.

14.8 WRAP-UP

In this chapter we began our study of machine learning, using the popular scikit-learn library. We saw that machine learning is divided into two types. Supervised machine learning, which works with labeled data and unsupervised machine learning which works with unlabeled data. Throughout this chapter, we continued emphasizing visualizations using Matplotlib and Seaborn, particularly for getting to know your data.

We discussed how scikit-learn conveniently packages machine-learning algorithms as estimators. Each is encapsulated so you can create your models quickly with a small amount of code, even if you don't know the intricate details of how these algorithms work.

We looked at supervised machine learning with classification, then regression. We used one of the simplest classification algorithms, k-nearest neighbors, to analyze the Digits dataset bundled with scikit-learn. You saw that classification algorithms predicts the classes to which samples belong. Binary classification uses two classes (such as "spam" or "not spam") and multi-classification uses more than two classes (such as the 10 classes in the Digits dataset).

We performed the steps of a typical machine-learning case study, including loading the dataset, exploring the data with pandas and visualizations, splitting the data for training and testing, creating the model, training the model and making predictions. We discussed why you should partition your data into a training set and a testing set. You saw ways to evaluate a classification estimator's accuracy via a confusion matrix

nd a classification report.

We mentioned that it's difficult to know in advance which model(s) will perform best on your data, so you typically try many models and pick the one that performs best. We showed that it's easy to run multiple estimators. We also used hyperparameter tuning with k-fold cross-validation to choose the best value of k for the k-NN algorithm.

We revisited the time series and simple linear regression example from chapter 10's Intro to Data Science section, this time implementing it using a scikit-learn `LinearRegression` estimator. Next, we used a `LinearRegression` estimator to perform multiple linear regression with the California Housing dataset that's bundled with scikit-learn. You saw that the `LinearRegression` estimator, by default, uses all the numerical features in a dataset to make more sophisticated predictions than you can with simple linear regression. Again, we ran multiple scikit-learn estimators to compare how they performed and choose the best one.

Next, we introduced an unsupervised machine learning and mentioned that it's typically accomplished with clustering algorithms. We used introduced dimensionality reduction (with scikit-learn's `TSNE` estimator) and used it to compress the Digits dataset's 64 features down to two for visualization purposes. This enabled us to see the clustering of the digits data.

We presented one of the simplest unsupervised machine learning algorithms, k-means clustering, and demonstrated clustering on the Iris dataset that's also bundled with scikit-learn. We used dimensionality reduction (with scikit-learn's `PCA` estimator) to compress the Iris dataset's four features to two for visualization purposes to show the clustering of the three *Iris* species in the dataset and their centroids. Finally, we ran multiple clustering estimators to compare their ability to label the Iris dataset's samples into three clusters.

In the next chapter, we'll continue our study of machine learning technologies with deep learning. We'll tackle some fascinating and challenging problems.

15. Deep Learning

Objectives

In this chapter you'll:

- Understand what a neural network is and how it enables deep learning.
- Create Keras neural networks.
- Understand Keras layers, activation functions, loss functions and optimizers.
- Use a Keras convolutional neural network (CNN) trained on the MNIST dataset to recognize handwritten digits.
- Use a Keras recurrent neural network (RNN) trained on the IMDb dataset to perform binary classification of positive and negative movie reviews.
- Use TensorBoard to visualize the progress of training deep-learning networks.
- Learn which pretrained neural networks come with Keras.
- Understand the value of using models pretrained on the massive ImageNet dataset for computer vision apps.

Outline

5.1 Introduction

5.1.1 Deep Learning Applications

5.1.2 Deep Learning Demos

5.1.3 Keras Resources

5.2 Keras Built-In Datasets

5.3 Custom Anaconda Environments

5.4 Neural Networks

5.5 Tensors

5.6 Convolutional Neural Networks for Vision; Multi-Classification with the MNIST Dataset

[5.6.1 Loading the MNIST Dataset](#)

[5.6.2 Data Exploration](#)

[5.6.3 Data Preparation](#)

[5.6.4 Creating the Neural Network](#)

[5.6.5 Training and Evaluating the Model](#)

[5.6.6 Saving and Loading a Model](#)

[5.7 Visualizing Neural Network Training with TensorBoard](#)

[5.8 ConvnetJS: Browser-Based Deep-Learning Training and Visualization](#)

[5.9 Recurrent Neural Networks for Sequences; Sentiment Analysis with the IMDb Dataset](#)

[5.9.1 Loading the IMDb Movie Reviews Dataset](#)

[5.9.2 Data Exploration](#)

[5.9.3 Data Preparation](#)

[5.9.4 Creating the Neural Network](#)

[5.9.5 Training and Evaluating the Model](#)

[5.10 Tuning Deep Learning Models](#)

[5.11 Convnet Models Pretrained on ImageNet](#)

[5.12 Wrap-Up](#)

15.1 INTRODUCTION

One of AI's most exciting areas is **deep learning**, a powerful subset of machine learning that has produced impressive results in computer vision and many other areas over the last few years. The availability of big data, significant processor power, faster Internet speeds and advancements in parallel computing hardware and software are making it possible for more organizations and individuals to pursue resource-intensive deep-learning solutions.

Keras and TensorFlow

In the previous chapter, Scikit-learn enabled you to define machine-learning models conveniently with one statement. Deep learning models require more sophisticated setups, typically connecting multiple objects, called **layers**. We'll build our deep learning models with **Keras**, which offers a friendly interface to Google's **TensorFlow**—the most widely used deep-learning library. ¹ François Chollet of the Google Mind team developed Keras to make deep-learning capabilities more accessible. His book *Deep Learning with Python* is a must read. ² Google has thousands of TensorFlow and Keras projects underway internally

nd that number is growing quickly.³,⁴

¹ Keras also serves as a friendlier interface to Microsofts *CNTK* and the Université de Montréal's *Theano*- (which ceased development in 2017). Other popular deep learning frameworks include Caffe (<http://caffe.berkeleyvision.org/>), Apache MXNet (<https://mxnet.apache.org/>) and PyTorch (<https://pytorch.org/>).

² Chollet, François. *Deep Learning with Python*. Shelter Island, NY: Manning Publications, 2018.

³ <http://theweek.com/speedreads/654463/google-more-than-1000-artificial-intelligence-projects-works>.

⁴ <https://www.zdnet.com/article/google-says-exponential-growth-of-i-is-changing-nature-of-compute/>.

Models

Deep learning models are complex and require an extensive mathematical background to understand their inner workings. As we've done throughout the book, we'll avoid heavy mathematics here, preferring English explanations.

Keras is to deep learning as Scikit-learn is to machine learning. Each encapsulates the sophisticated mathematics, so developers need only define, parameterize and manipulate objects. With Keras, you build your models from *pre-existing* components and quickly parameterize those components to your unique requirements. This is what we've been referring to as *object-based programming* throughout the book.

Experiment with Your Models

Machine learning and deep learning are empirical rather than theoretical fields. You'll experiment with many models, tweaking them in various ways until you find the models that perform best for your applications. Keras facilitates such experimentation.

Dataset Sizes

Deep learning works well when you have lots of data, but it also can be effective for smaller datasets when combined with techniques like transfer learning⁵,⁶ and data augmentation⁷,⁸. Transfer learning uses existing knowledge from a previously trained model as the foundation for a new model. Data augmentation adds data to a dataset by deriving new data from existing data. For example, in an image dataset, you might rotate the images left and right so the model can learn about objects in different orientations. In general, though, the more data you have, the better you'll be able to train a deep learning model.

⁵ <https://towardsdatascience.com/transfer-learning-from-pre-trained-models-f2393f124751>.

⁶ <https://medium.com/nanonets/nanonets-how-to-use-deep-learning-when-you-have-limited-data-f68c0b512cab>.

⁷ <https://towardsdatascience.com/data-augmentation-and-images->

aca9bd0dbe8.

⁸ <https://medium.com/nanonets/how-to-use-deep-learning-when-you-ave-limited-data-part-2-data-augmentation-c26971dc8ced>.

Processing Power

Deep learning can require significant processing power. Complex models trained on big-data datasets can take hours, days or even more to train. The models we present in this chapter can be trained in minutes to just less than an hour on computers with conventional CPUs. You'll need only a reasonably current personal computer. We'll discuss the special high-performance hardware called GPUs (Graphics Processing Units) and TPUs (Tensor Processing Units) developed by NVIDIA and Google to meet the extraordinary processing demands of edge-of-the-practice deep-learning applications.

Bundled Datasets

Keras comes packaged with some popular datasets. You'll work with two of these datasets in the chapter's examples. You can find many Keras studies online for each of these datasets, including ones that take different approaches.

In the "Machine Learning" chapter, you worked with Scikit-learn's Digits dataset, which contained 1797 handwritten-digit images that were selected from the much larger MNIST dataset (60,000 training images and 10,000 test images). ⁹ In this chapter you'll work with the full MNIST dataset. You'll build a Keras *convolutional neural network* (CNN or convnet) model that will achieve high performance recognizing digit images in the test set. Convnets are especially appropriate for computer vision tasks, such as recognizing handwritten digits and characters or recognizing objects (including faces) in images and videos. You'll also work with a Keras *recurrent neural network*. In that example, you'll perform sentiment analysis using the IMDb Movie reviews dataset, in which the reviews in the training and testing sets are labeled as positive or negative.

⁹ The MNIST Database. MNIST Handwritten Digit Database, Yann LeCun, Corinna Cortes and Chris Burges. <http://yann.lecun.com/exdb/mnist/>.

Future of Deep Learning

Newer automated deep learning capabilities are making it even easier to build deep-learning solutions. These include Auto-Keras ⁰ from Texas A&M University's DATA Lab, Baidu's EZDL ¹ and Google's AutoML ².

⁰ <https://autokeras.com/>.

¹ <https://ai.baidu.com/ezdl/>.

² <https://cloud.google.com/automl/>.

15.1.1 Deep Learning Applications

Deep learning is being used in a wide range of applications, such as:

- Game playing
- Computer vision: Object recognition, pattern recognition, facial recognition
- Self-driving cars
- Robotics
- Improving customer experiences
- Chatbots
- Diagnosing medical conditions
- Google Search
- Facial recognition
- Automated image captioning and video closed captioning
- Enhancing image resolution
- Speech recognition
- Language translation
- Predicting election results
- Predicting earthquakes and weather
- Google Sunroof to determine whether you can put solar panels on your roof
- Generative applications—Generating original images, processing existing images to look like a specified artist's style, adding color to black-and-white images and video, creating music, creating text (books, poetry) and much more.

15.1.2 Deep Learning Demos

Check out these four deep-learning demos and search online for lots more, including practical applications like we mentioned in the preceding section:

- DeepArt.io—Turn a photo into artwork by applying an art style to the photo.
<https://deepart.io/>.
- DeepWarp Demo—Analyzes a person's photo and makes the person's eyes move in different directions.
https://sites.skoltech.ru/sites/compvision_wiki/static_pages/projects/dee
- Image-to-Image Demo—Translates a line drawing into a picture.
<https://affinelayer.com/pixsrv/>.
- Google Translate Mobile App (download from an app store to your smartphone)—

ranslate text in a photo to another language (e.g., take a photo of a sign or a restaurant menu in Spanish and translate the text to English).

15.1.3 Keras Resources

Here are some resources you might find valuable as you study deep learning:

- To get your questions answered, go to the Keras team’s slack channel at <https://kerasteam.slack.com>.
- For articles and tutorials, visit <https://blog.keras.io>.
- The Keras documentation is at <http://keras.io>.
- If you’re looking for term projects, directed study projects, capstone course projects or thesis topics, visit arXiv (pronounced “archive,” where the X represents the Greek letter “chi”) at <https://arXiv.org>. People post their research papers here in parallel with going through peer review for formal publication, hoping for fast feedback. So, this site gives you access to extremely current research.

15.2 KERAS BUILT-IN DATASETS

Here are some of Keras’s datasets (from the module `tensorflow.keras.datasets`³) for practicing deep learning. We’ll use a couple of these in the chapter’s examples:

³In the standalone Keras library, the module names begin with `keras` rather than `tensorflow.keras`.

- **MNIST⁴ database of handwritten digits**—Used for classifying handwritten digit images, this dataset contains 28-by-28 grayscale digit images labeled as 0 through 9 with 60,000 images for training and 10,000 for testing. We use this dataset in [section 15.6](#), where we study convolutional neural networks.

⁴The MNIST Database. MNIST Handwritten Digit Database, Yann LeCun, Corinna Cortes and Chris Burges. <http://yann.lecun.com/exdb/mnist/>.

- **Fashion-MNIST⁵ database of fashion articles**—Used for classifying clothing images, this dataset contains 28-by-28 grayscale images of clothing labeled in 10 categories⁶ with 60,000 for training and 10,000 for testing. Once you build a model for use with MNIST, you can reuse that model with Fashion-MNIST by changing a few statements.
- **IMDb Movie reviews⁷**—Used for sentiment analysis, this dataset contains reviews labeled as positive (1) or negative (0) sentiment with 25,000 reviews for training and 25,000 for testing. We use this dataset in [section 15.9](#), where we study recurrent neural networks.

⁵Han Xiao and Kashif Rasul and Roland Vollgraf, Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms, arXiv, cs.LG/1708.07747.

⁶ <https://keras.io/datasets/#fashion-mnist-database-of-fashion-articles>.

⁷Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. (2011). Learning Word Vectors for Sentiment Analysis. The 49th Annual Meeting of the Association for Computational Linguistics (ACL 2011).

- **CIFAR10**⁸ **small image classification**—Used for small-image classification, this dataset contains 32-by-32 color images labeled in 10 categories with 50,000 images for training and 10,000 for testing.

⁸ <https://www.cs.toronto.edu/~kriz/cifar.html>.

- **CIFAR100**⁹ **small image classification**—Also, used for small-image classification, this dataset contains 32-by-32 color images labeled in 100 categories with 50,000 images for training and 10,000 for testing.

⁹ <https://www.cs.toronto.edu/~kriz/cifar.html>.

15.3 CUSTOM ANACONDA ENVIRONMENTS

Before running this chapter's examples, you'll need to install the libraries we use. In this chapter's examples, we'll use the TensorFlow deep-learning library's version of Keras.¹⁰ At the time of this writing, TensorFlow does not yet support Python 3.7. So, you'll need Python 3.6.x to execute this chapter's examples. We'll show you how to set up a *custom environment* for working with Keras and TensorFlow.

¹⁰There's also a standalone version that enables you to choose between TensorFlow, Microsoft's CNTK or the Université de Montréal's Theano (which ceased development in 2017).

Environments in Anaconda

The Anaconda Python distribution makes it easy to create custom **environments**. These are separate configurations in which you can install different libraries and different library versions. This can help with *reproducibility* if your code depends on specific Python or library versions.¹¹

¹¹In the next chapter, we'll introduce Docker as another reproducibility mechanism and as a convenient way to install complex environments for use on your local computer.

The default environment in Anaconda is called the *base environment*. This is created for you when you install Anaconda. All the Python libraries that come with Anaconda are installed into the base environment and, unless you specify otherwise, any additional libraries you install also are placed there. Custom environments give you control over the specific libraries you wish to install for your specific tasks.

Creating an Anaconda Environment

The **conda create command** creates an environment. Let's create a TensorFlow

environment and name it `tf_env` (you can name it whatever you like). Run the following command in your Terminal, shell or Anaconda Command Prompt:^{2, 3}

²Windows users should run the Anaconda Command Prompt as Administrator,

³If you have a computer with an NVIDIA GPU that's compatible with TensorFlow, you can replace the `tensorflow` library with `tensorflow-gpu` to get better performance. For more information, see <https://www.tensorflow.org/install/gpu>. Some AMD GPUs also can be used with TensorFlow: <http://timdettmers.com/2018/11/05/which-gpu-or-deep-learning/>.

[click here to view code image](#)

```
conda create -n tf_env tensorflow anaconda ipython jupyterlab scikit-learn matplotlib
```

This will determine the listed libraries' dependencies, then display all the libraries that will be installed in the new environment. There are many dependencies, so this may take a few minutes. When you see the prompt:

```
Proceed ([y]/n)?
```

press *Enter* to create the environment and install the libraries.⁴

⁴When we created our custom environment, conda installed Python 3.6.7, which was the most recent Python version compatible with the `tensorflow` library.

Activating an Alternate Anaconda Environment

To use a custom environment, execute the **conda activate command**:

```
conda activate tf_env
```

This affects only the current Terminal, shell or Anaconda Command Prompt. When a custom environment is activated and you install more libraries, they become part of the activated environment, not the base environment. If you open separate Terminals, shells or Anaconda Command Prompts, they'll use Anaconda's base environment by default.

Deactivating an Alternate Anaconda Environment

When you're done with a custom environment, you can return to the base environment in the current Terminal, shell or Anaconda Command Prompt by executing:

```
conda deactivate
```

Jupyter Notebooks and JupyterLab

This chapter's examples are provided only as Jupyter Notebooks, which will make it easier for you to experiment with the examples. You can tweak the options we present and reexecute

he notebooks. For this chapter, you should launch JupyterLab from the `ch15` examples folder (as discussed in [section 1.5.3](#)).

15.4 NEURAL NETWORKS

Deep learning is a form of machine learning that uses artificial neural networks to learn. An **artificial neural network** (or just neural network) is a software construct that operates similarly to how scientists believe our brains work. Our biological nervous systems are controlled via *neurons*⁵ that communicate with one another along pathways called *synapses*⁶. As we learn, the specific neurons that enable us to perform a given task, like walking, communicate with one another more efficiently. These neurons *activate* anytime we need to walk.⁷

⁵ <https://en.wikipedia.org/wiki/Neuron>.

⁶ <https://en.wikipedia.org/wiki/Synapse>.

⁷ <https://www.sciencenewsforstudents.org/article/learning-rewires-brain>.

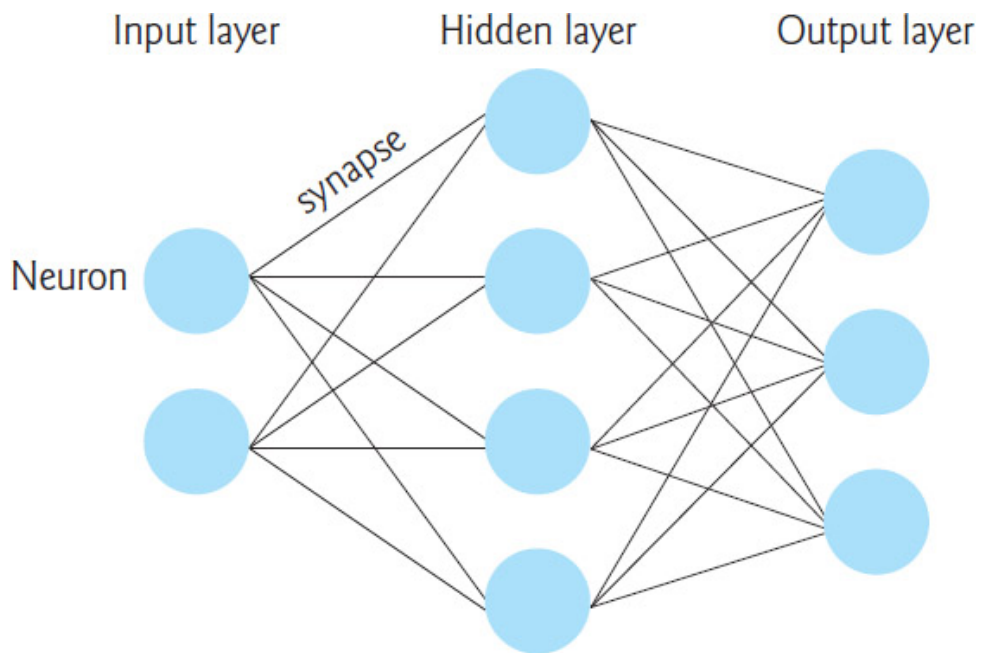
Artificial Neurons

In a neural network, interconnected **artificial neurons** simulate the human brain's neurons to help the network learn. The connections between specific neurons are reinforced during the learning process with the goal of achieving a specific result. In **supervised deep learning**—which we'll use in this chapter—we aim to predict the target labels supplied with data samples. To do this, we'll train a general neural network model that we can then use to make predictions on unseen data.⁸

⁸As in machine learning, you can create *unsupervised* deep learning networks these are beyond this chapters scope.

Artificial Neural Network Diagram

The following diagram shows a three-layer neural network. Each circle represents a neuron, and the lines between them simulate the synapses. The output of a neuron becomes the input of another neuron, hence the term neural network. This particular diagram shows a **fully connected network**—every neuron in a given layer is connected to *all* the neurons in the next layer:



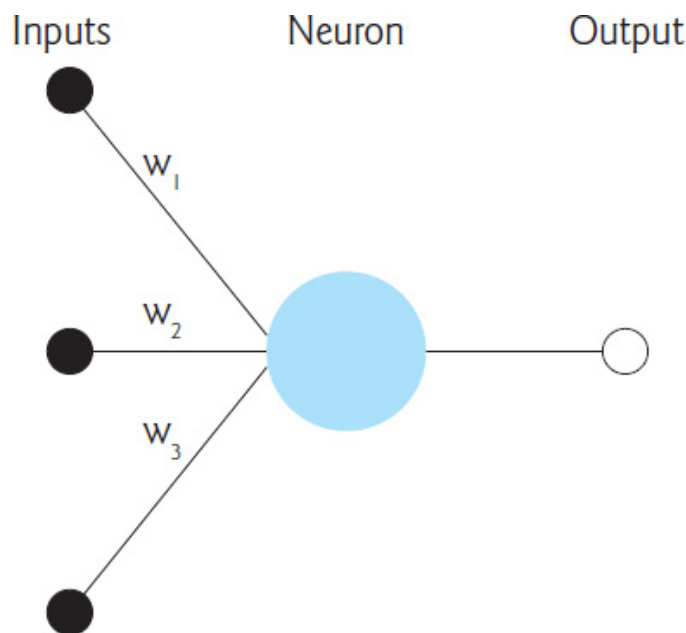
Learning Is an Iterative Process

When you were a baby, you did not learn to walk instantaneously. You learned that process *over time* with repetition. You built up the smaller components of the movements that enabled you to walk—learning to stand, learning to balance to remain standing, learning to lift your foot and move it forward, etc. And you got feedback from your environment. When you walked successfully your parents smiled and clapped. When you fell, you might have bumped your head and felt pain.

Similarly, we train neural networks iteratively over time. Each iteration is known as an **epoch** and processes every sample in the training dataset once. There's no "correct" number of epochs. This is a hyperparameter that may need tuning, based on your training data and your model. The inputs to the network are the features in the training samples. Some layers learn new features from previous layers' outputs and others interpret those features to make predictions.

How Artificial Neurons Decide Whether to Activate Synapses

During the training phase, the network calculates values called **weights** for every connection between the neurons in one layer and those in the next. On a neuron-by-neuron basis, each of its inputs is multiplied by that connection's weight, then the sum of those weighted inputs is passed to the neuron's **activation function**. This function's output determines which neurons to activate based on the inputs—just like the neurons in your brain passing information around in response to inputs coming from your eyes, nose, ears and more. The following diagram shows a neuron receiving three inputs (the black dots) and producing an output (the hollow circle) that would be passed to all or some of neurons in the next layer, depending on the types of the neural network's layers:



The values w_1 , w_2 and w_3 are weights. In a new model that you train from scratch, these values are initialized randomly by the model. As the network trains, it tries to minimize the error rate between the network’s predicted labels and the samples’ actual labels. The error rate is known as the **loss**, and the calculation that determines the loss is called the **loss function**. Throughout training, the network determines the amount that each neuron contributes to the overall loss, then goes back through the layers and adjusts the weights in an effort to minimize that loss. This technique is called **backpropagation**. Optimizing these weights occurs gradually—typically via a process called **gradient descent**.

15.5 TENSORS

Deep learning frameworks generally manipulate data in the form of **tensors**. A “tensor” is basically a multidimensional array. Frameworks like TensorFlow pack all your data into one or more tensors, which they use to perform the mathematical calculations that enable neural networks to learn. These tensors can become quite large as the number of dimensions increases and as the richness of the data increases (for example, images, audios and videos are richer than text). Chollet discusses the types of tensors typically encountered in deep learning:⁹

⁹Chollet, François. *Deep Learning with Python*. Section 2.2. Shelter Island, NY: Manning Publications, 2018.

- **0D (o-dimensional) tensor**—This is one value and is known as a *scalar*.
- **1D tensor**—This is similar to a one-dimensional array and is known as a *vector*. A 1D tensor might represent a sequence, such as hourly temperature readings from a sensor or the words of one movie review.
- **2D tensor**—This is similar to a two-dimensional array and is known as a *matrix*. A 2D tensor could represent a grayscale image in which the tensor’s two dimensions are the image’s width and height in pixels, and the value in each element is the intensity of that pixel.
- **3D tensor**—This is similar to a three-dimensional array and could be used to represent a

olor image. The first two dimensions would represent the width and height of the image in pixels and the *depth* at each location might represent the red, green and blue (RGB) components of a given pixel's color. A 3D tensor also could represent a *collection* of 2D tensors containing grayscale images.

- **4D tensor**—A 4D tensor could be used to represent a *collection* of color images in 3D tensors. It also could be used to represent one video. Each frame in a video is essentially a color image.
- **5D tensor**—This could be used to represent a collection of 4D tensors containing videos.

A tensor's *shape* typically is represented as a tuple of values in which the number of elements specifies the tensor's number of dimensions and each value in the tuple specifies the size of the tensor's corresponding dimension.

Let's assume we're creating a deep-learning network to identify and track objects in 4K (high-resolution) videos that have 30 frames-per-second. Each frame in a 4K video is 3840-by-2160 pixels. Let's also assume the pixels are presented as red, green and blue components of a color. So *each frame* would be a 3D tensor containing a total of 24,883,200 elements ($3840 * 2160 * 3$) and each video would be a 4D tensor containing the sequence of frames. If the videos are one minute long, you'd have 44,789,760,000 elements *per tensor*!

Over 600 hours of video are uploaded to YouTube every minute⁹ so, in just one minute of uploads, Google could have a tensor containing 1,612,431,360,000,000 elements to use in training deep-learning models—that's *big data*. As you can see, tensors can quickly become *enormous*, so manipulating them efficiently is crucial. This is one of the key reasons that most deep learning is performed on GPUs. More recently Google created TPUs (Tensor Processing Units) that are specifically designed to perform tensor manipulations, executing faster than GPUs.

⁹ <https://www.inc.com/tom-popomaronis/youtube-analyzed-trillions-of-data-points-in-2018-revealing-5-eye-opening-behavioral-statistics.html>.

High-Performance Processors

Powerful processors are needed for real-world deep learning because the size of tensors can be enormous and large-tensor operations can place crushing demands on processors. The processors most commonly used for deep learning are:

- **NVIDIA GPUs (Graphics Processing Units)**—Originally developed by companies like NVIDIA for computer gaming, GPUs are much faster than conventional CPUs for processing large amounts of data, thus enabling developers to train, validate and test deep-learning models more efficiently—and thus experiment with more of them. GPUs are optimized for the mathematical matrix operations typically performed on tensors, an essential aspect of how deep learning works “under the hood.” NVIDIA's Volta Tensor Cores are specifically designed for deep learning.^{1, 2} Many NVIDIA GPUs are compatible with TensorFlow, and hence Keras, and can enhance the performance of your deep-learning models.³

¹ <https://www.nvidia.com/en-us/data-center/tensorcore/>.

² <https://devblogs.nvidia.com/tensor-core-ai-performance-milestones/>.

³ <https://www.tensorflow.org/install/gpu>.

- Google TPUs (Tensor Processing Units)—Recognizing that deep learning is crucial to its future, Google developed TPUs (Tensor Processing Units), which they now use in their Cloud TPU service, which “can provide up to 11.5 petaflops of performance in a single pod” ⁴ (that’s 11.5 *quadrillion* floating-point operations per second). Also, TPUs are designed to be especially energy efficient. This is a key concern for companies like Google with already massive computing clusters that are growing exponentially and consuming vast amounts of energy.

⁴ <https://cloud.google.com/tpu/>.

15.6 CONVOLUTIONAL NEURAL NETWORKS FOR VISION; MULTI-CLASSIFICATION WITH THE MNIST DATASET

In the “Machine Learning” chapter, we classified handwritten digits using the 8-by-8-pixel, low-resolution images from the Digits dataset bundled with Scikit-learn. That dataset is based on a subset of the higher-resolution MNIST handwritten digits dataset. Here, we’ll use MNIST to explore deep learning with a **convolutional neural network** ⁵ (also called a **convnet** or **CNN**). Convnets are common in computer-vision applications, such as recognizing handwritten digits and characters, and recognizing objects in images and video. They’re also used in non-vision applications, such as natural-language processing and recommender systems.

⁵ https://en.wikipedia.org/wiki/Convolutional_neural_network.

The Digits dataset has only 1797 samples, whereas MNIST has 70,000 labeled digit image samples—60,000 for training and 10,000 for testing. Each sample is a grayscale 28-by-28 pixel image (784 total features) represented as a NumPy array. Each pixel is a value from 0 to 255 representing the intensity (or shade) of that pixel—the Digits dataset uses less granular shading with values from 0 to 16. MNIST’s labels are integer values in the range 0 through 9, indicating the digit each image represents.

The machine-learning model you used in the previous chapter produced as its output a digit image’s predicted class—an integer in the range 0–9. The convnet model we’ll build will perform **probabilistic classification**. ⁶ For each digit image, the model will output an *array* of 10 probabilities, each indicating the likelihood that the digit belongs to a particular one of the classes 0 through 9. The class with the *highest* probability is the predicted value.

⁶ https://en.wikipedia.org/wiki/Probabilistic_classification.

Reproducibility in Keras and Deep Learning

We’ve discussed the importance of *reproducibility* throughout the book. In deep learning, reproducibility is more difficult because the libraries heavily parallelize operations that

perform floating-point calculations. Each time operations execute, they may execute in a different order. This can produce differences in your results. Getting reproducible results in Keras requires a combination of environment settings and code settings that are described in the Keras FAQ:

[lick here to view code image](#)

```
https://keras.io/getting-started/faq/#how-can-i-obtain-reproducible-results-usi
```

asic Keras Neural Network

A Keras neural network consists of the following components:

- A **network** (also called a **model**)—A sequence of **layers** containing the neurons used to learn from the samples. Each layer's neurons receive inputs, process them (via an *activation function*) and produce outputs. The data is fed into the network via an **input layer** that specifies the dimensions of the sample data. This is followed by **hidden layers** of neurons that implement the learning and an **output layer** that produces the predictions. The more layers you *stack*, the deeper the network is, hence the term deep learning.
- A **loss function**—This produces a measure of how well the network predicts the target values. Lower loss values indicate better predictions.
- An **optimizer**—This attempts to minimize the values produced by the loss function to tune the network to make better predictions.

Launch JupyterLab

This section assumes that you've activated the `tf_env` Anaconda environment you created in section 15.3 and launched JupyterLab from the `ch15` examples folder. You can either open the `MNIST_CNN.ipynb` file in JupyterLab and execute the code in the cells we provided, or you can create a new notebook and enter the code on your own. If you prefer, you can work at the command line in IPython, however, placing your code in a Jupyter Notebook makes it significantly easier for you to *re-execute* this chapter's examples.

As a reminder, you can reset a Jupyter Notebook and remove its outputs by selecting **Restart Kernel and Clear All Outputs** from JupyterLab's **Kernel** menu. This terminates the notebook's execution and removes its outputs. You might do this if your model is not performing well and you want to try different hyperparameters or possibly restructure your neural network.⁷ You can then re-execute the notebook one cell at a time or execute the entire notebook by selecting **Run All** from JupyterLab's **Run** menu.

⁷We found that we sometimes had to execute this menu option twice to clear the outputs.

15.6.1 Loading the MNIST Dataset

Let's import the `tensorflow.keras.datasets.mnist` module so we can load the dataset:

[lick here to view code image](#)

```
[1]: from tensorflow.keras.datasets import mnist
```

Note that because we're using the version of Keras built into TensorFlow, the Keras module names begin with "tensorflow.". In the standalone Keras version, the module names begin with "keras.", so `keras.datasets` would be used above. Keras uses *TensorFlow* to execute the deep-learning models.

The `mnist` module's **`load_data` function** loads the MNIST training and testing sets:

[lick here to view code image](#)

```
[2]: (X_train, y_train), (X_test, y_test) = mnist.load_data()
```

When you call `load_data` it will download the MNIST data to your system. The function returns a tuple of two elements containing the training and testing sets. Each element is itself a tuple containing the samples and labels, respectively.

15.6.2 Data Exploration

Let's get to know the data before working with it. First, we check the dimensions of the training set images (`X_train`), training set labels (`y_train`), testing set images (`X_test`) and testing set labels (`y_test`):

```
[3]: X_train.shape
[3]: (60000, 28, 28)

[4]: y_train.shape
[4]: (60000,)

[5]: X_test.shape
[5]: (10000, 28, 28)

[6]: y_test.shape
[6]: (10000,)
```

You can see from `X_train`'s and `X_test`'s shapes that the images are higher resolution than those in Scikit-learn's Digits dataset (which are 8-by-8).

Visualizing Digits

Let's visualize some of the digit images. First, enable Matplotlib in the notebook, import Matplotlib and Seaborn and set the font scale:

[lick here to view code image](#)

```
[7]: %matplotlib inline
[8]: import matplotlib.pyplot as plt

[9]: import seaborn as sns
```

```
[10]: sns.set(font_scale=2)
```

The IPython magic

```
%matplotlib inline
```

indicates that Matplotlib-based graphics should be displayed *in the notebook* rather than in separate windows. For more IPython magics, you can use in Jupyter Notebooks, see:

```
https://ipython.readthedocs.io/en/stable/interactive/magics.html
```

Next, we'll display a randomly selected set of 24 MNIST training set images. Recall from the “Array-Oriented Programming with NumPy” chapter that you can pass a sequence of indexes as a NumPy array's subscript to select only the array elements at those indexes. We'll use that capability here to select the elements at the same indexes in both the `X_train` and `y_train` arrays. This ensures that we display the correct label for each randomly selected image.

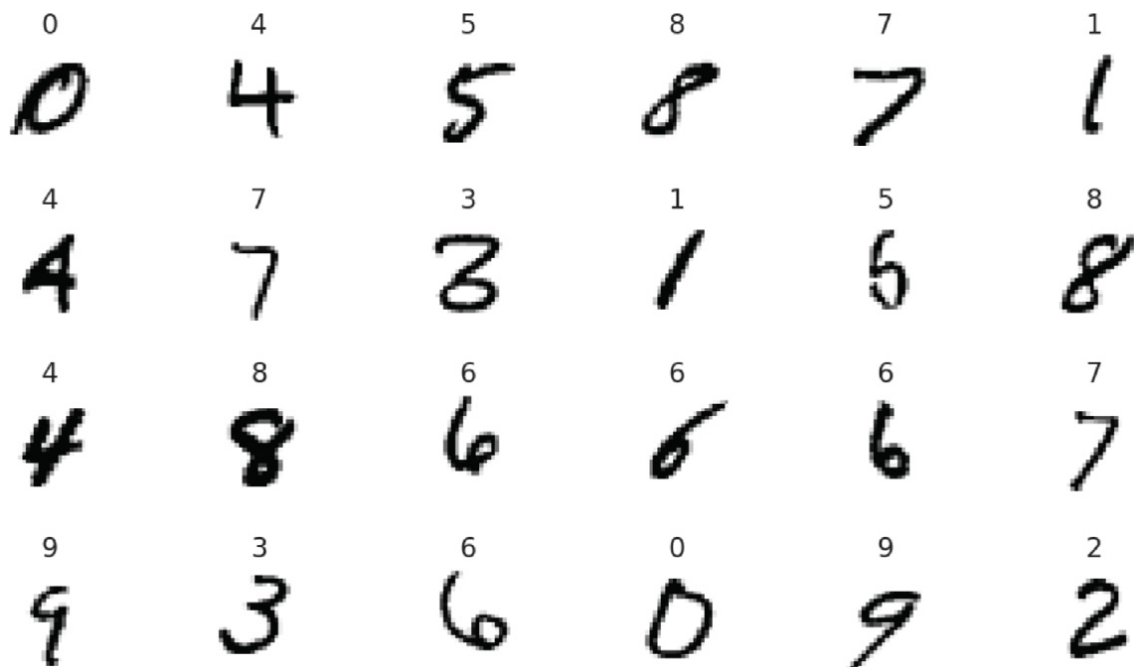
NumPy's **choice function** (from the `numpy.random` module) randomly selects the number of elements specified in its second argument (24) from the array of values in its first argument (in this case, an array containing `X_train`'s range of indices). The function returns an array containing the selected values, which we store in `index`. The expressions `X_train[index]` and `y_train[index]` use `index` to get the corresponding elements from both arrays. The rest of this cell is the visualization code from the previous chapter's Digits case study:

[lick here to view code image](#)

```
[11]: import numpy as np
      index = np.random.choice(np.arange(len(X_train)), 24, replace=False)
      figure, axes = plt.subplots(nrows=4, ncols=6, figsize=(16, 9))

      for item in zip(axes.ravel(), X_train[index], y_train[index]):
          axes, image, target = item
          axes.imshow(image, cmap=plt.cm.gray_r)
          axes.set_xticks([]) # remove x-axis tick marks
          axes.set_yticks([]) # remove y-axis tick marks
          axes.set_title(target)
      plt.tight_layout()
```

You can see in the output below that MNIST's digit images have higher resolution than those in Scikit-learn's Digits dataset.



Looking at the digits, you can see why handwritten digit recognition is a challenge:

- Some people write “open” 4s (like the ones in the first and third rows), and some write “closed” 4s (like the one in the second row). Though each 4 has some similar features, they’re all different from one another.
- The 3 in the second row looks strange—more like a merged 6 and 7. Compare this to the much clearer 3 in the fourth row.
- The 5 in the second row could easily be confused with a 6.
- Also, people write their digits at different angles, as you can see with the four 6s in the third and fourth rows—two are upright, one leans left and one leans right.

If you run the preceding snippet multiple times, you can see additional randomly selected digits.⁸ You’ll probably find that—if not for the labels displayed above each digit—it would be difficult for you to identify some of the digits. We’ll soon see how accurately our first convnet will predict the digits in the MNIST test set.

⁸If you do run the cell multiple times, the snippet number next to the cell will increment each time, as it does in IPython at the command line.

15.6.3 Data Preparation

Recall from the “Machine Learning” chapter that Scikit-learn’s bundled datasets were preprocessed into the shapes its models required. In real-world studies, you’ll generally have to do some or all of the data preparation. The MNIST dataset requires some preparation for use in a Keras convnet.

Reshaping the Image Data

Keras convnets require NumPy array inputs in which each sample has the shape:

```
(width, height, channels)
```

For MNIST, each image's *width* and *height* are 28 pixels, and each pixel has one *channel* (the grayscale shade of the pixel from 0 to 255), so each sample's shape will be:

```
(28, 28, 1)
```

Full-color images with RGB (red/green/blue) values for each pixel, would have three *channels*—one channel each for the red, green and blue components of a color.

As the neural network learns from the images, it creates many more channels. Rather than shade or color, the learned channels will represent more complex features, like edges, curves and lines, that will eventually enable the network to recognize digits based on these additional features and how they're combined.

Let's reshape the 60,000 training and 10,000 testing set images into the correct dimensions for use in our convnet and confirm their new shapes. Recall that NumPy array method `reshape` receives a tuple representing the array's new shape:

[lick here to view code image](#)

```
[12]: X_train = X_train.reshape((60000, 28, 28, 1))

[13]: X_train.shape
[13]: (60000, 28, 28, 1)

[14]: X_test = X_test.reshape((10000, 28, 28, 1))

[15]: X_test.shape
[15]: (10000, 28, 28, 1)
```

Normalizing the Image Data

Numeric features in data samples may have value ranges that vary widely. Deep learning networks perform better on data that is scaled either into the range 0.0 to 1.0, or to a range for which the data's mean is 0.0 and its standard deviation is 1.0.⁹ Getting your data into one of these forms is known as **normalization**.

⁹S. Ioffe and Szegedy, C.. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. <https://arxiv.org/abs/1502.03167>.

In MNIST, each pixel is an integer in the range 0–255. The following statements convert the values to 32-bit (4-byte) floating-point numbers using the NumPy array method `astype`, then divide every element in the resulting array by 255, producing normalized values in the range 0.0–1.0:

[lick here to view code image](#)

```
[16]: X_train = X_train.astype('float32') / 255

[17]: X_test = X_test.astype('float32') / 255
```

One-Hot Encoding: Converting the Labels From Integers to Categorical Data

As we mentioned, the convnet’s prediction for each digit will be an array of 10 probabilities, indicating the likelihood that the digit belongs to a particular one of the classes 0 through 9. When we evaluate the model’s accuracy, Keras compares the model’s predictions to the labels. To do that, Keras requires both to have the same shape. The MNIST label for each digit, however, is one integer value in the range 0–9. So, we must transform the labels into **categorical data**—that is, arrays of categories that match the format of the predictions. To do this, we’ll use a process called **one-hot encoding**,^o which converts data into arrays of 1.0s and 0.0s in which only one element is 1.0 and the rest are 0.0s. For MNIST, the one-hot-encoded values will be 10-element arrays representing the categories 0 through 9. One-hot encoding also can be applied to other types of data.

^o This term comes from certain digital circuits in which a group of bits is allowed to have only one bit turned on (that is, to have the value 1). <https://en.wikipedia.org/wiki/One-hot>.

We know precisely which category each digit belongs to, so the categorical representation of a digit label will consist of a 1.0 at that digit’s index and 0.0s for all the other elements (again, Keras uses floating-point numbers internally). So, a 7’s categorical representation is:

[lick here to view code image](#)

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0]
```

and a 3’s representation is:

[lick here to view code image](#)

```
[0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

The **tensorflow.keras.utils module** provides function **to_categorical** to perform one-hot encoding. The function counts the unique categories then, for each item being encoded, creates an array of that length with a 1.0 in the correct position. Let’s transform `y_train` and `y_test` from one-dimensional arrays containing the values 0–9 into two-dimensional arrays of categorical data. After doing so, the rows of these arrays will look like those shown above. Snippet [21] outputs one sample’s categorical data for the digit 5 (recall that NumPy shows the decimal point, but not trailing 0s on floating-point values):

[lick here to view code image](#)

```
[18]: from tensorflow.keras.utils import to_categorical

[19]: y_train = to_categorical(y_train)

[20]: y_train.shape
[20]: (60000, 10)

[21]: y_train[0]
[21]: array([ 0.,  0.,  0.,  0.,  0.,  1.,  0.,  0.,  0.,  0.], dtype=float32)

[22]: y_test = to_categorical(y_test)
```

```
[23]: y_test.shape  
[23]: (10000, 10)
```

5.6.4 Creating the Neural Network

Now that we've prepared the data, we'll configure a convolutional neural network. We begin with the Keras **Sequential model** from the **tensorflow.keras.models module**:

[lick here to view code image](#)

```
[24]: from tensorflow.keras.models import Sequential  
  
[25]: cnn = Sequential()
```

The resulting network will execute its layers sequentially—the output of one layer becomes the input to the next. This is known as a **feed-forward network**. As you'll see when we discuss *recurrent neural networks*, not all neural network operate this way.

Adding Layers to the Network

A typical convolutional neural network consists of several layers—an *input layer* that receives the training samples, *hidden layers* that learn from the samples and an *output layer* that produces the prediction probabilities. We'll create a basic convnet here. Let's import from the **tensorflow.keras.layers module** the layer classes we'll use in this example:

[lick here to view code image](#)

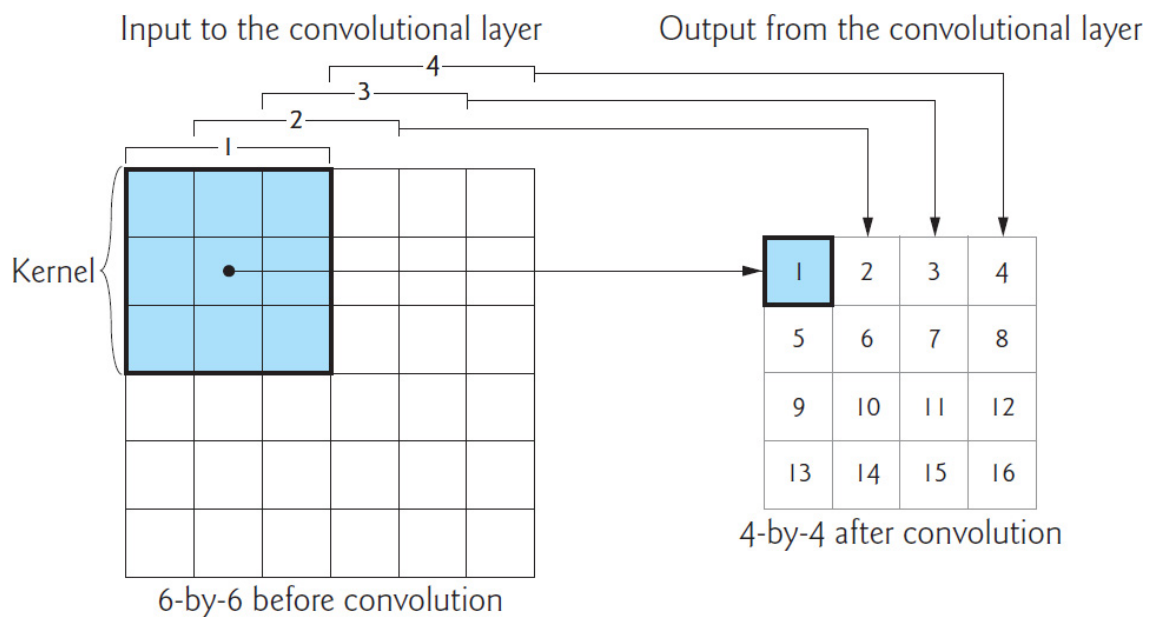
```
[26]: from tensorflow.keras.layers import Conv2D, Dense, Flatten,  
      MaxPooling2D-
```

We discuss each below.

Convolution

We'll begin our network with a **convolution layer**, which uses the relationships between pixels that are close to one another to learn useful features (or patterns) in small areas of each sample. These features become inputs to subsequent layers.

The small areas that convolution learns from are called **kernels** or **patches**. Let's examine convolution on a 6-by-6 image. Consider the following diagram in which the 3-by-3 shaded square represents the kernel—the numbers are simply position numbers showing the order in which the kernels are visited and processed:



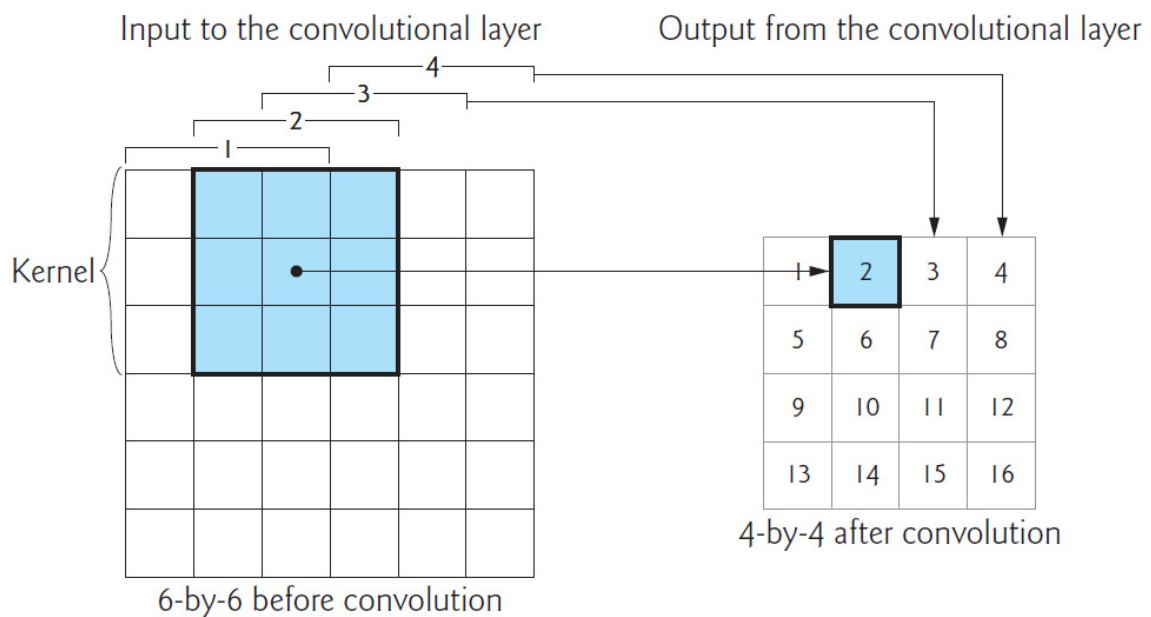
The small areas that convolution learns from are called **kernels** or **patches**. Let's examine convolution on a 6-by-6 image. Consider the following diagram in which the 3-by-3 shaded square represents the kernel—the numbers are simply position numbers showing the order in which the kernels are visited and processed:

You can think of the kernel as a “sliding window” that the convolution layer moves one pixel at a time left-to-right across the image. When the kernel reaches the right edge, the convolution layer moves the kernel one pixel down and repeats this left-to-right process. Kernels typically are 3-by-3,¹ though we found convnets that used 5-by-5 and 7-by-7 for higher-resolution images. Kernel-size is a tunable hyperparameter.

¹ <https://www.quora.com/How-can-I-decide-the-kernel-size-output-maps-and-layers-of-CNN>.

Initially, the kernel is in the upper-left corner of the original image—kernel position 1 (the shaded square) in the input layer above. The convolution layer performs mathematical calculations using those *nine* features to “learn” about them, then outputs *one* new feature to position 1 in the layer's output. By looking at features near one another, the network begins to recognize features like edges, straight lines and curves.

Next, the convolution layer moves the kernel one pixel to the right (known as the *stride*) to position 2 in the input layer. This new position *overlaps* with two of the three columns in the previous position, so that the convolution layer can learn from all the features that touch one another. The layer learns from the nine features in kernel position 2 and outputs one new feature in position 2 of the output, as in:



For a 6-by-6 image and a 3-by-3 kernel, the convolution layer does this two more times to produce features for positions 3 and 4 of the layer's output. Then, the convolution layer moves the kernel one pixel down and begins the left-to-right process again for the next four kernel positions, producing outputs in positions 5–8, then 9–12 and finally 13–16. The complete pass of the image left-to-right and top-to-bottom is called a **filter**. For a 3-by-3 kernel, the filter dimensions (4-by-4 in our sample above) will be *two less than the input dimensions* (6-by-6). For each 28-by-28 MNIST image, the filter will be 26-by-26.

The number of filters in the convolutional layer is commonly 32 or 64 when processing small images like those in MNIST, and each filter produces different results. The number of filters depends on the image dimensions—higher-resolution images have more features, so they require more filters. If you study the code the Keras team used to produce their pretrained convnets, ² you'll find that they used 64, 128 or even 256 filters in their first convolutional layers. Based on their convnets and the fact that the MNIST images are small, we'll use 64 filters in our first convolutional layer. The set of filters produced by a convolution layer is called a **feature map**.

² https://github.com/keras-team/keras-applications/tree/master/keras_applications.

Subsequent convolution layers combine features from previous feature maps to recognize larger features and so on. If we were doing facial recognition, early layers might recognize lines, edges and curves, and subsequent layers might begin combining those into larger features like eyes, eyebrows, noses, ears and mouths. Once the network learns a feature, because of convolution, it can recognize that feature anywhere in the image. This is one of the reasons that convnets are used for object recognition in images.

Adding a Convolution Layer

Let's add a **Conv2D** convolution layer to our model:

[lick here to view code image](#)

```
[27]: cnn.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu',
                    input_shape=(28, 28, 1)))
```

he Conv2D layer is configured with the following arguments:

- `filters=64`—The number of filters in the resulting feature map.
- `kernel_size=(3, 3)`—The size of the kernel used in each filter.
- `activation='relu'`—The '**relu**' (**Rectified Linear Unit**) **activation function** is used to produce this layer's output. '`relu`' is the most widely used activation function in today's deep learning networks ³ and is good for performance because it's easy to calculate. ⁴ It's commonly recommended for convolutional layers. ⁵

³Chollet, François. *Deep Learning with Python*. p. 72. Shelter Island, NY: Manning Publications, 2018.

⁴ <https://towardsdatascience.com/exploring-activation-functions-for-neural-networks-73498da59b02>.

⁵ <https://www.quora.com/How-should-I-choose-a-proper-activation-function-for-the-neural-network>.

Because this is the first layer in the model, we also pass the `input_shape=(28, 28, 1)` argument to specify the shape of each sample. This automatically creates an input layer to load the samples and pass them into the Conv2D layer, which is actually the first *hidden* layer. In Keras, each subsequent layer infers its `input_shape` from the previous layer's output shape, making it easy to *stack* layers.

Dimensionality of the First Convolution Layer's Output

In the preceding convolutional layer, the input samples are 28-by-28-by-1—that is, 784 features each. We specified 64 filters and a 3-by-3 kernel size for the layer, so the output for each image is 26-by-26-by-64 for a total of 43,264 features in the feature map—a significant increase in dimensionality and an enormous number compared to the numbers of features we processed in the “Machine Learning” chapter's models. As each layer adds more features, the resulting feature maps' *dimensionality* becomes significantly larger. This is one of the reasons that deep learning studies often require tremendous processing power.

Overfitting

Recall from the previous chapter, that overfitting can occur when your model is too complex compared to what it is modeling. In the most extreme case, a model memorizes its training data. When you make predictions with an overfit model, they will be accurate if new data matches the training data, but the model could perform poorly with data it has never seen.

Overfitting tends to occur in deep learning as the dimensionality of the layers becomes too large. ^{6, 7, 8} This causes the network to learn *specific* features of the training-set digit images, rather than learning the *general* features of digit images. Some techniques to prevent overfitting include training for fewer epochs, data augmentation, dropout and L1 or L2 regularization. ^{9, 10} We'll discuss dropout later in the chapter.

⁶ <https://cs231n.github.io/convolutional-networks/>.

⁷ <https://medium.com/@cxu24/why-dimensionality-reduction-is-important-dd60b5611543>.

⁸ <https://towardsdatascience.com/preventing-deep-neural-network-from-verfitting-953458db800a>.

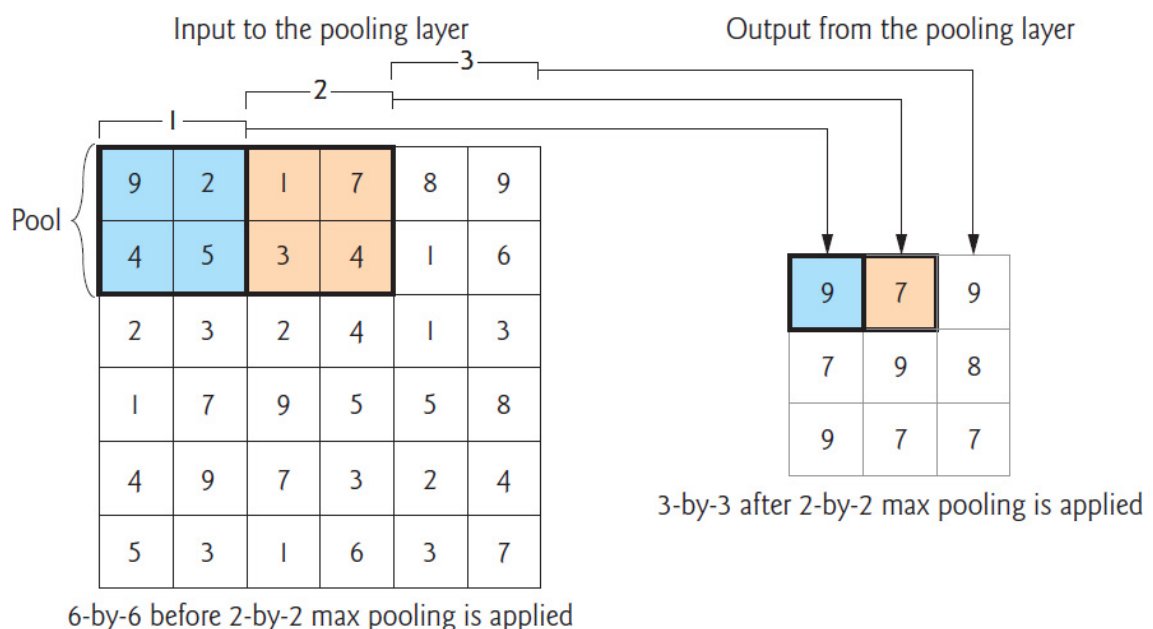
⁹ <https://towardsdatascience.com/deep-learning-3-more-on-cnns-andling-overfitting-2bd5d99abe5d>.

⁰ <https://www.kdnuggets.com/2015/04/preventing-overfitting-neural-networks.html>.

Higher dimensionality also increases (and sometimes explodes) computation time. If you're performing the deep learning on CPUs rather than GPUs or TPUs, the training could become intolerably slow.

Adding a Pooling Layer

To reduce overfitting and computation time, a convolution layer is often followed by one or more layers that *reduce the dimensionality* of the convolution layer's output. A **pooling layer** *compresses* (or *down-samples*) the results by discarding features, which helps make the model more general. The most common pooling technique is called **max pooling**, which examines a 2-by-2 square of features and keeps only the maximum feature. To understand pooling, let's once again assume a 6-by-6 set of features. In the following diagram, the numeric values in the 6-by-6 square represent the features that we wish to compress and the 2-by-2 blue square in position 1 represents the initial pool of features to examine:



The max pooling layer first looks at the pool in position 1 above, then outputs the *maximum* feature from that pool—9 in our diagram. Unlike convolution, there's *no overlap* between pools. The pool moves by its width—for a 2-by-2 pool, the *stride* is 2. For the second pool, represented by the orange 2-by-2 square, the layer outputs 7. For the third pool, the layer outputs 9. Once the pool reaches the right edge, the pooling layer moves the pool down by its height—2 rows—then continues from left-to-right. Because every group of four features is reduced to one, 2-by-2 pooling *compresses* the number of features by 75%.

Let's add a **MaxPooling2D** layer to our model:

[lick here to view code image](#)

```
[28]: cnn.add(MaxPooling2D(pool_size=(2, 2)))
```

This reduces the previous layer's output from 26-by-26-by-64 to 13-by-13-by-64. ¹

¹Another technique for reducing overfitting is to add **Dropout** layers.

Though pooling is a common technique to reduce overfitting, some research suggests that additional convolutional layers which use larger strides for their kernels can reduce dimensionality and overfitting *without* discarding features. ²

²Tobias, Jost, Dosovitskiy, Alexey, Brox, Thomas, Riedmiller, and Martin. Striving for Simplicity: The All Convolutional Net. April 13, 2015.

<https://arxiv.org/abs/1412.6806>.

Adding Another Convolutional Layer and Pooling Layer

Convnets often have many convolution and pooling layers. The Keras team's convnets tend to double the number of filters in subsequent convolutional layers to enable the model to learn more relationships between the features. ³ So, let's add a second convolution layer with 128 filters, followed by a second pooling layer to once again reduce the dimensionality by 75%:

³ https://github.com/keras-team/keras-applications/tree/master/keras_applications.

[lick here to view code image](#)

```
[29]: cnn.add(Conv2D(filters=128, kernel_size=(3, 3), activation='relu'))  
  
[30]: cnn.add(MaxPooling2D(pool_size=(2, 2)))
```

The input to the second convolution layer is the 13-by-13-by-64 output of the first pooling layer. So, the output of snippet [29] will be 11-by-11-by-128. For odd dimensions like 11-by-11, Keras pooling layers round *down* by default (in this case to 10-by-10), so this pooling layer's output will be 5-by-5-by-128.

Flattening the Results

At this point, the previous layer's output is three-dimensional (5-by-5-by-128), but the final output of our model will be a *one-dimensional* array of 10 probabilities that classify the digits. To prepare for the one-dimensional final predictions, we first need to *flatten* the previous layer's three-dimensional output. A Keras **Flatten** layer reshapes its input to one dimension. In this case, the **Flatten** layer's output will be 1-by-3200 (that is, $5 * 5 * 128$):

```
[31]: cnn.add(Flatten())
```

Adding a Dense Layer to Reduce the Number of Features

The layers before the `Flatten` layer learned digit features. Now we need to take all those features and learn the relationships among them so our model can classify which digit each image represents. Learning the relationships among features and performing classification is accomplished with fully connected **Dense** layers, like those shown in the neural network diagram earlier in the chapter. The following `Dense` layer creates 128 neurons (`units`) that learn from the 3200 outputs of the previous layer:

[lick here to view code image](#)

```
[32]: cnn.add(Dense(units=128, activation='relu'))
```

Many convnets contain at least one `Dense` layer like the one above. Convnets geared to more complex image datasets with higher-resolution images like Image-Net—a dataset of over 14 million images ⁴—often have several `Dense` layers, commonly with 4096 neurons. You can see such configurations in several of Keras’s pretrained Image-Net convnets ⁵—we list these in [section 15.11](#).

⁴ <http://www.image-net.org>.

⁵ https://github.com/keras-team/keras-applications/tree/master/keras_applications.

Adding Another Dense Layer to Produce the Final Output

Our final layer is a `Dense` layer that classifies the inputs into neurons representing the classes 0 through 9. The **softmax activation function** converts the values of these remaining 10 neurons into classification probabilities. The neuron that produces the highest probability represents the prediction for a given digit image:

[lick here to view code image](#)

```
[33]: cnn.add(Dense(units=10, activation='softmax'))
```

Printing the Model’s Summary

A model’s **summary method** shows you the model’s layers. Some interesting things to note are the output shapes of the various layers and the number of parameters. The parameters are the *weights* that the network learns during training. ^{6, 7} This is a relatively small network, yet it will need to learn nearly 500,000 parameters! And this is for tiny images that have less than one quarter of the resolution of the icons on most smartphone home screens. Imagine how many features a network would have to learn to process high-resolution 4K video frames or the super-high-resolution images produced by today’s digital cameras. In the `Output Shape`, `None` simply means that the model does not know in advance how many training samples you’re going to provide—this is known only when you start the training.

⁶ <https://hackernoon.com/everything-you-need-to-know-about-neural->

etworks-8988c3ee4491.

⁷ <https://www.kdnuggets.com/2018/06/deep-learning-best-practices-eight-initialization-.html>.

[lick here to view code image](#)

```
[34]: cnn.summary()
```

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 64)	640
max_pooling2d_1 (MaxPooling2)	(None, 13, 13, 64)	0
conv2d_2 (Conv2D)	(None, 11, 11, 128)	73856
max_pooling2d_2 (MaxPooling2)	(None, 5, 5, 128)	0
flatten_1 (Flatten)	(None, 3200)	0
dense_1 (Dense)	(None, 128)	409728
dense_2 (Dense)	(None, 10)	1290

=====
Total params: 485,514
Trainable params: 485,514
Non-trainable params: 0
=====

Also, note that there are no “non-trainable” parameters. By default, Keras trains *all* parameters, but it is possible to prevent training for specific layers, which is typically done when you’re tuning your networks or using another model’s learned parameters in a new model (a process called *transfer learning*). ⁸

⁸ <https://keras.io/getting-started/faq/#how-can-i-freeze-keras-layers>.

Visualizing a Model’s Structure

You can visualize the model summary using the **plot_model** function from the module `tensorflow.keras.utils`:

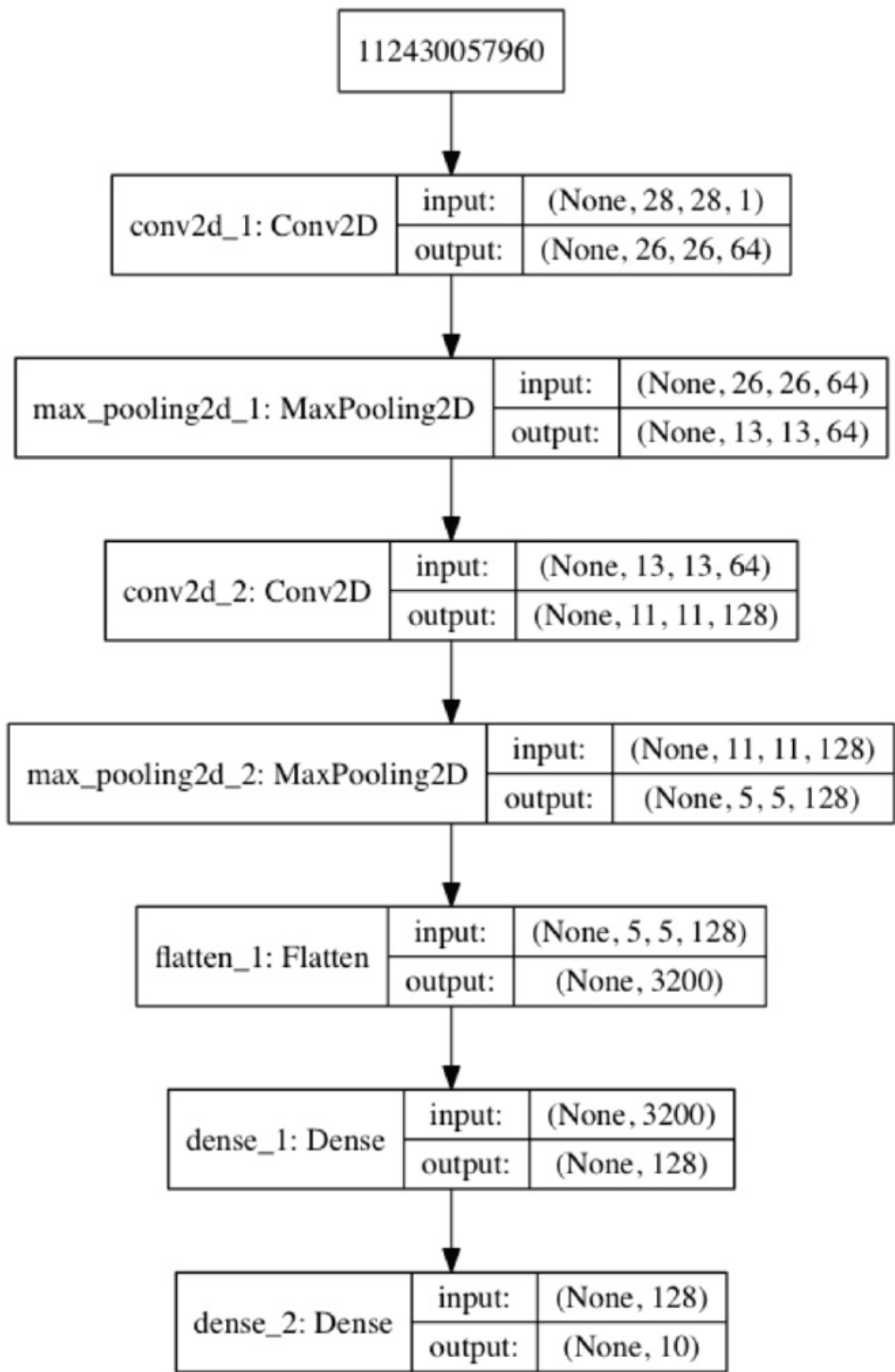
[lick here to view code image](#)

```
[35]: from tensorflow.keras.utils import plot_model
      from IPython.display import Image
      plot_model(cnn, to_file='convnet.png', show_shapes=True,
                  show_layer_names=True)
      Image(filename='convnet.png')
```

After storing the visualization in `convnet.png`, we use module `IPython.display`’s **Image** class to show the image in the notebook. Keras assigns the layer names in the image: ⁹

⁹The node with the large integer value 112430057960 at the top of the diagram appears to be

a bug in the current version of Keras. This node represents the input layer and should say InputLayer.



Compiling the Model

Once you've added all the layers you complete the model by calling its **compile method**:

[lick here to view code image](#)


```
[36]: cnn.compile(optimizer='adam',  
                 loss='categorical_crossentropy',  
                 metrics=['accuracy'])
```

The arguments are:

- `optimizer='adam'`—The *optimizer* this model will use to adjust the weights throughout the neural network as it learns. There are many optimizers⁰— 'adam' performs well across a wide variety of models.^{1, 2}

⁰For more Keras optimizers, see <https://keras.io/optimizers/>.

¹ <https://medium.com/octavian-ai/which-optimizer-and-learning-rate-should-i-use-for-deep-learning-5acb418f9b2>.

² <https://towardsdatascience.com/types-of-optimization-algorithms-used-in-neural-networks-and-ways-to-optimize-gradient-5ae5d39529f>.

- `loss='categorical_crossentropy'`—This is the *loss function* used by the optimizer in multi-classification networks like our convnet, which will predict 10 classes. As the neural network learns, the optimizer attempts to minimize the values returned by the loss function. The lower the loss, the better the neural network is at predicting what each image is. For binary classification (which we'll use later in this chapter), Keras provides 'binary_crossentropy', and for regression, 'mean_squared_error'. For other loss functions, see <https://keras.io/losses/>.
- `metrics=['accuracy']`—This is a list of the *metrics* that the network will produce to help you evaluate the model. Accuracy is a commonly used metric in classification models. In this example, we'll use the `accuracy` metric to check the percentage of correct predictions. For a list of other metrics, see <https://keras.io/metrics/>.

15.6.5 Training and Evaluating the Model

Similar to Scikit-learn's models, we train a Keras model by calling its **fit method**:

- As in Scikit-learn, the first two arguments are the training data and the categorical target labels.
- **epochs** specifies the number of times the model should process the entire set of training data. As we mentioned earlier, neural networks are trained iteratively.
- **batch_size** specifies the number of samples to process at a time during each epoch. Most models specify a power of 2 from 32 to 512. Larger batch sizes can decrease model accuracy.³ We chose 64. You can try different values to see how they affect the model's performance.

³Keskar, Nitish Shirish, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy and Ping Tak Peter Tang. On Large-Batch Training for Deep Learning: Generalization Gap

and Sharp Minima. CoRR abs/1609.04836 (2016).

<https://arxiv.org/abs/1609.04836>.

- In general, some samples should be used to *validate* the model. If you specify validation data, after each epoch, the model will use it to make predictions and display the *validation loss and accuracy*. You can study these values to tune your layers and the `fit` method's hyperparameters, or possibly change the layer composition of your model. Here, we used the **`validation_split`** argument to indicate that the model should reserve the *last* 10% (0.1) of the training samples for validation ⁴—in this case, 6000 samples will be used for validation. If you have separate validation data, you can use the `validation_data` argument (as you'll see in section 15.9) to specify a tuple containing arrays of samples and target labels. In general, it's better to get *randomly selected validation data*. You can use scikit-learn's `train_test_split` function for this purpose (as we'll do later in this chapter), then pass the randomly selected data with the `validation_data` argument.

⁴ <https://keras.io/getting-started/faq/#how-is-the-validation-split-computed>.

In the following output, we highlighted the training accuracy (`acc`) and validation accuracy (`val_acc`) in bold:

[lick here to view code image](#)

```
[37]: cnn.fit(X_train, y_train, epochs=5, batch_size=64,
            validation_split=0.1)
Train on 54000 samples, validate on 6000 samples
Epoch 1/5
54000/54000 [=====] - 68s 1ms/step - loss: 0.1407 -
poch 2/5
54000/54000 [=====] - 64s 1ms/step - loss: 0.0426 -
poch 3/5
54000/54000 [=====] - 69s 1ms/step - loss: 0.0299 -
poch 4/5
54000/54000 [=====] - 70s 1ms/step - loss: 0.0197 -
poch 5/5
54000/54000 [=====] - 63s 1ms/step - loss: 0.0155 -
[37]: <tensorflow.python.keras.callbacks.History at 0x7f105ba0ada0>
```

In section 15.7, we'll introduce *TensorBoard*—a TensorFlow tool for visualizing data from your deep-learning models. In particular, we'll view charts showing how the training and validation accuracy and loss values change through the epochs. In section 15.8, we'll demonstrate Andrej Karpathy's ConvnetJS tool, which trains convnets in your web browser and dynamically visualizes the layers' outputs, including what each convolutional layer "sees" as it learns. Also run his MNIST and CIFAR10 models. These will help you better understand neural networks' complex operations.

As the training proceeds, the `fit` method outputs information showing you the progress of each epoch, how long the epoch took to execute (in this case, each took 63–70 seconds), and the evaluation metrics for that pass. During the last epoch of this model, the accuracy reached

99.48% for the training samples (`acc`) and 99.27% for the validation samples (`val_acc`). Those are impressive numbers, given that we have not yet tried to tune the hyperparameters or tweak the number and types of the layers, which could lead to even better (or worse) results. Like machine learning, deep learning is an empirical science that benefits from lots of experimentation.

Evaluating the Model

Now we can check the accuracy of the model on data the model has not yet seen. To do so, we call the model's `evaluate` method, which displays as its output, how long it took to process the test samples (four seconds and 366 microseconds in this case):

[lick here to view code image](#)

```
[38]: loss, accuracy = cnn.evaluate(X_test, y_test)
10000/10000 [=====] - 4s   366us/step

[39]: loss
[39]: 0.026809450998473768

[40]: accuracy
[40]: 0.9917
```

According to the preceding output, our convnet model is 99.17% accurate when predicting the labels for unseen data—and, at this point, we have not tried to tune the model. With a little online research, you can find models that can predict MNIST with nearly 100% accuracy. Try experimenting with different numbers of layers, types of layers and layer parameters and observe how those changes affect your results.

Making Predictions

The model's `predict` method predicts the classes of the digit images in its argument array (`X_test`):

[lick here to view code image](#)

```
[41]: predictions = cnn.predict(X_test)
```

We can check what the first sample digit should be by looking at `y_test[0]`:

[lick here to view code image](#)

```
[42]: y_test[0]
[42]: array([0., 0., 0., 0., 0., 0., 0., 1., 0., 0.], dtype=float32)
```

According to this output, the first sample is the digit 7, because the categorical representation of the test sample's label specifies a 1.0 at index 7—recall that we created this representation via *one-hot encoding*.

Let's check the probabilities returned by the `predict` method for the first test sample:

[lick here to view code image](#)

```
[43]: for index, probability in enumerate(predictions[0]):
        print(f'{index}: {probability:.10%}')
0: 0.0000000201%
1: 0.0000001355%
2: 0.0000186951%
3: 0.0000015494%
4: 0.0000000003%
5: 0.0000000012%
6: 0.0000000000%
7: 99.9999761581%
8: 0.0000005577%
9: 0.0000011416%
```

According to the output, `predictions[0]` indicates that our model believes this digit is a 7 with *nearly* 100% certainty. Not all predictions have this level of certainty.

Locating the Incorrect Predictions

Next, we'd like to view some of the *incorrectly* predicted images to get a sense of the ones our model has trouble with. For example, if it's always mispredicting 8s, perhaps we need more 8s in our training data.

Before we can view incorrect predictions, we need to locate them. Consider `predictions[0]` above. To determine whether the prediction was correct, we must compare the index of the largest probability in `predictions[0]` to the index of the element containing 1.0 in `y_test[0]`. If these index values are the same, then the prediction was correct; otherwise, it was incorrect. NumPy's `argmax` function determines the index of the highest valued element in its array argument. Let's use that to locate the incorrect predictions. In the following snippet, `p` is the predicted value array, and `e` is the expected value array (the expected values are the labels for the dataset's test images):

[lick here to view code image](#)

```
[44]: images = X_test.reshape((10000, 28, 28))
        incorrect_predictions = []

        for i, (p, e) in enumerate(zip(predictions, y_test)):
            predicted, expected = np.argmax(p), np.argmax(e)

            if predicted != expected:
                incorrect_predictions.append(
                    (i, images[i], predicted, expected))
```

In this snippet, we first reshape the samples from the shape `(28, 28, 1)` that Keras required for learning back to `(28, 28)`, which Matplotlib requires to display the images. Next, we populate the list `incorrect_predictions` using the `for` statement. We `zip` the rows that represent each sample in the arrays `predictions` and `y_test`, then `enumerate` those so we can capture their indexes. If the `argmax` results for `p` and `e` are different, then the prediction was incorrect, and we append a tuple to `incorrect_predictions` containing that sample's index, image, the predicted value and the expected value. We

can confirm the total number of incorrect predictions (out of 10,000 images in the test set) with:

```
[45]: len(incorrect_predictions)
[45]: 83
```

Visualizing Incorrect Predictions

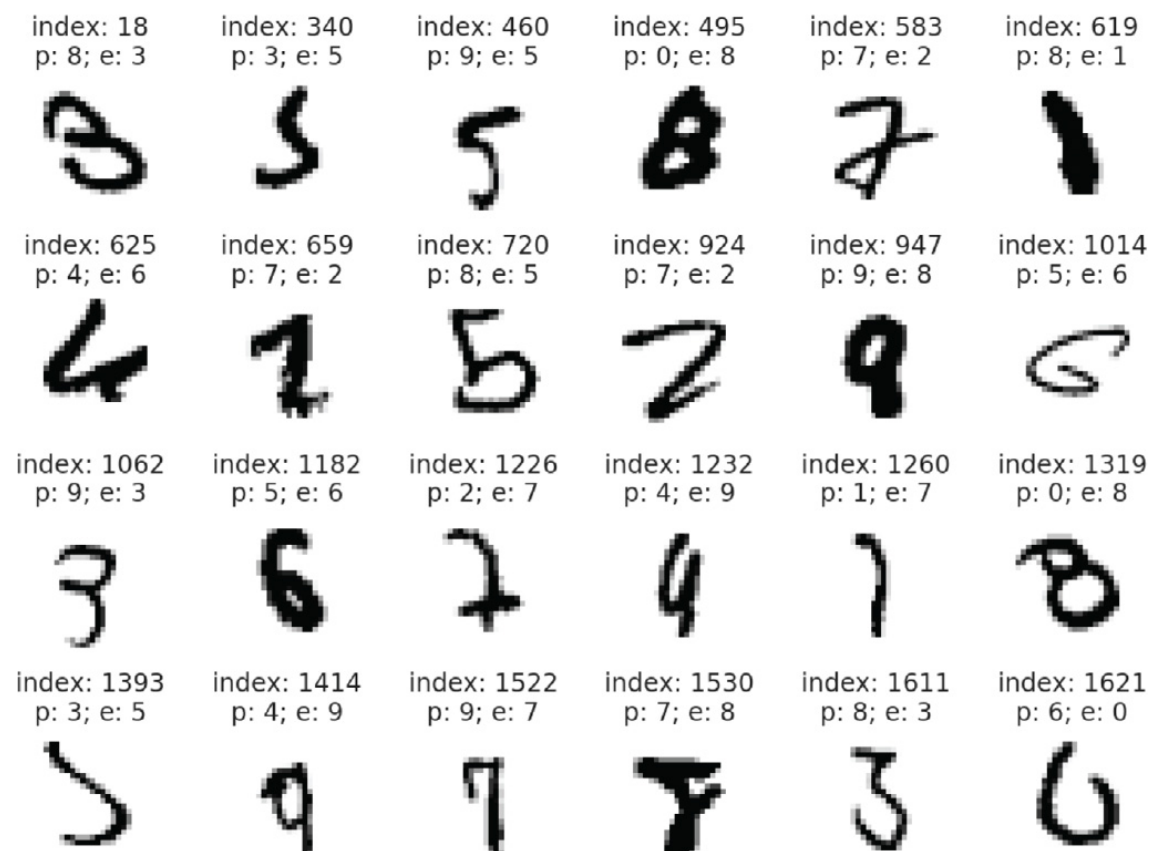
The following snippet displays 24 of the incorrect images labeled with each image's index, predicted value (p) and expected value (e):

[lick here to view code image](#)

```
[46]: figure, axes = plt.subplots(nrows=4, ncols=6, figsize=(16, 12))

for axes, item in zip(axes.ravel(), incorrect_predictions):
    index, image, predicted, expected = item
    axes.imshow(image, cmap=plt.cm.gray_r)
    axes.set_xticks([]) # remove x-axis tick marks
    axes.set_yticks([]) # remove y-axis tick marks
    axes.set_title(
        f'index: {index}\np: {predicted}; e: {expected}')
plt.tight_layout()
```

Before reading the expected values, look at each digit and write down what digit you think it is. This is an important part of getting to know your data:



Displaying the Probabilities for Several Incorrect Predictions

Let's look at the probabilities of some incorrect predictions. The following function displays the probabilities for the specified prediction array:

[lick here to view code image](#)

```
[47]: def display_probabilities(prediction):  
      for index, probability in enumerate(prediction):  
          print(f'{index}: {probability:.10%}')
```

Though the 8 (at index 495) in the first line of the image output looks like an 8, our model had trouble with it. As you can see in the following output, the model predicted this image as a 0, but also thought there was 16% chance it was a 6 and a 23% chance it was an 8:

[lick here to view code image](#)

```
[48]: display_probabilities(predictions[495])  
0: 59.7235262394%  
1: 0.0000015465%  
2: 0.8047289215%  
3: 0.0001740813%  
4: 0.0016636326%  
5: 0.0030567855%  
6: 16.1390662193%  
7: 0.0000001781%  
8: 23.3022540808%  
9: 0.0255270657%
```

The 2 (at index 583) in the first row was predicted to be a 7 with 62.7% certainty, but the model also thought there was a 36.4% chance it was a 2:

[lick here to view code image](#)

```
[49]: display_probabilities(predictions[583])  
0: 0.0000003016%  
1: 0.0000005715%  
2: 36.4056706429%  
3: 0.0176281916%  
4: 0.0000561930%  
5: 0.0000000003%  
6: 0.0000000019%  
7: 62.7455413342%  
8: 0.8310816251%  
9: 0.0000114385%
```

The 6 (at index 625) at the beginning of the second row was predicted to be a 4, though that was far from certain. In this case, the probability of a 4 (51.6%) was only slightly higher than the probability of a 6 (48.38%):

[lick here to view code image](#)

```
[50]: display_probabilities(predictions[625])  
0: 0.0008245181%  
1: 0.0000041209%  
2: 0.0012774357%  
3: 0.0000000009%  
4: 51.6223073006%  
5: 0.0000001779%  
6: 48.3754962683%
```

```
7: 0.0000000085%
8: 0.0000048182%
9: 0.0000785786%
```

15.6.6 Saving and Loading a Model

Neural network models can require significant training time. Once you've designed and tested a model that suits your needs, you can save its state. This allows you to load it later to make more predictions. Sometimes models are loaded and further trained for new problems. For example, layers in our model already know how to recognize features such as lines and curves, which could be useful in handwritten character recognition (as in the EMNIST dataset) as well. So you could potentially load the existing model and use it as the basis for a more robust model. This process is called **transfer learning**^{5, 6}—you transfer an existing model's knowledge into a new model. A Keras model's **save method** stores the model's architecture and state information in a format called **Hierarchical Data Format (HDF5)**. Such files use the `.h5` file extension by default:

⁵ <https://towardsdatascience.com/transfer-learning-from-pre-trained-models-f2393f124751>.

⁶ <https://medium.com/nanonets/nanonets-how-to-use-deep-learning-when-you-have-limited-data-f68c0b512cab>.

```
[51]: cnn.save('mnist_cnn.h5')
```

You can load a saved model with the **load_model function** from the `tensorflow.keras.models` module:

[lick here to view code image](#)

```
from tensorflow.keras.models import load_model
cnn = load_model('mnist_cnn.h5')
```

You can then invoke its methods. For example, if you've acquired more data, you could call `predict` to make additional predictions on new data, or you could call `fit` to start training with the additional data.

Keras provides several additional functions that enable you to save and load various aspects of your models. For more information, see

<https://keras.io/getting-started/faq/#how-can-i-save-a-keras-model>

15.7 VISUALIZING NEURAL NETWORK TRAINING WITH TENSORBOARD

With deep learning networks, there's so much complexity and so much going on internally that's hidden from you that it's difficult to know and fully understand all the details. This creates challenges in testing, debugging and updating models and algorithms. Deep learning

learns the features but there may be enormous numbers of them, and they may not be apparent to you.

Google provides the **TensorBoard**^{7, 8} tool for visualizing neural networks implemented in TensorFlow and Keras. Just as a car's dashboard visualizes data from your car's sensors, such as your speed, engine temperature and the amount of gas remaining, a **TensorBoard dashboard** visualizes data from a deep learning model that can give you insights into how well your model is learning and potentially help you tune its hyperparameters. Here, we'll introduce TensorBoard.

⁷ <https://github.com/tensorflow/tensorboard/blob/master/README.md>.

⁸ https://www.tensorflow.org/guide/summaries_and_tensorboard.

Executing TensorBoard

TensorBoard monitors a folder on your system looking for files containing the data it will visualize in a web browser. Here, you'll create that folder, execute the TensorBoard server, then access it via a web browser. Perform the following steps:

1. Change to the `ch15` folder in your Terminal, shell or Anaconda Command Prompt.
2. Ensure that your custom Anaconda environment `tf_env` is activated:

```
conda activate tf_env
```

3. Execute the following command to create a subfolder named `logs` in which your deep-learning models will write the information that TensorBoard will visualize:

```
mkdir logs
```

4. Execute TensorBoard

```
tensorboard --logdir=logs
```

5. You can now access TensorBoard in your web browser at

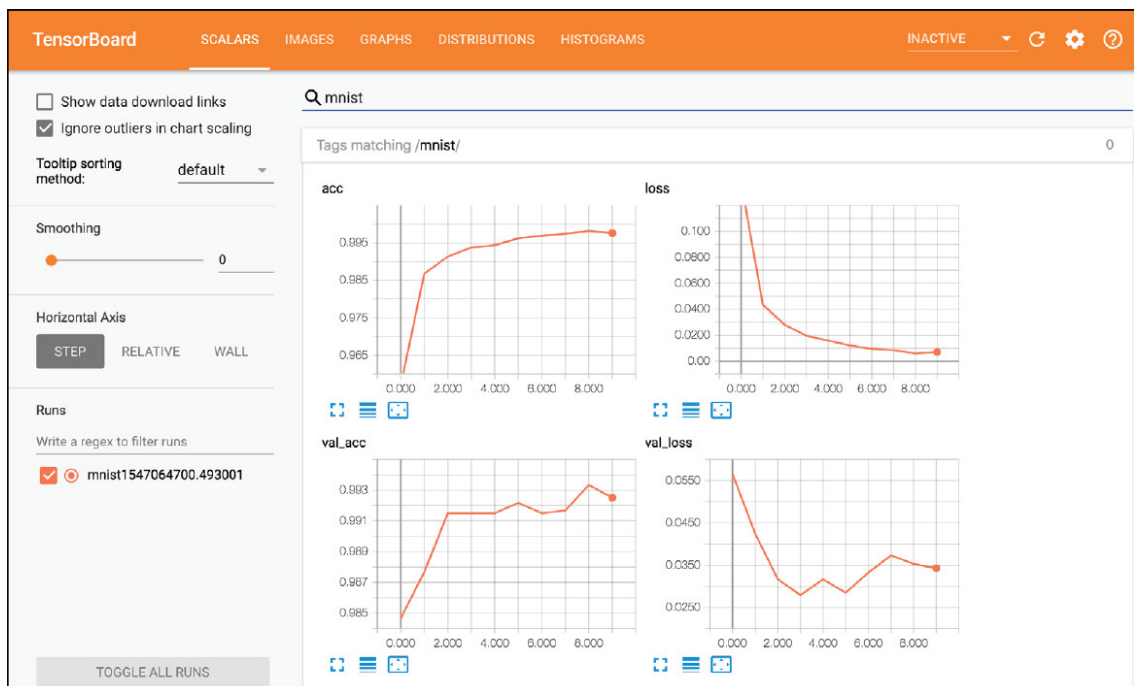
```
http://localhost:6006
```

If you connect to TensorBoard before executing any models, it will initially display a page indicating “No dashboards are active for the current data set.”⁹

⁹TensorBoard does not currently work with Microsofts Edge browser.

The TensorBoard Dashboard

TensorBoard monitors the folder you specified looking for files output by the model during training. When TensorBoard sees updates, it loads the data into the dashboard:



You can view the data as you train or after training completes. The dashboard above shows the TensorBoard **SCALARS** tab, which displays charts for individual values that change over time, such as the training accuracy (`acc`) and training loss (`loss`) shown in the first row, and the validation accuracy (`val_acc`) and validation loss (`val_loss`) shown in the second row. The diagrams visualize a 10-epoch run of our MNIST convnet, which we provided in the notebook `MNIST_CNN_TensorBoard.ipynb`. The epochs are displayed along the *x*-axes starting from 0 for the first epoch. The accuracy and loss values are displayed on the *y*-axes. Looking at the training and validation accuracies, you can see in the first 5 epochs similar results to the five-epoch run in the previous section.

For the 10-epoch run, the training accuracy continued to improve through the 9th epoch, then decreased slightly. This might be the point at which we’re starting to overfit, but we might need to train longer to find out. For the validation accuracy, you can see that it jumped up quickly, then was relatively flat for five epochs before jumping up then decreasing. For the training loss, you can see that it drops quickly, then continuously declines through the ninth epoch, before a slight increase. The validation loss dropped quickly then bounced around. We could run this model for more epochs to see whether results improve, but based on these diagrams, it appears that around the sixth epoch we get a nice combination of training and validation accuracy with minimal validation loss.

Normally these diagrams are stacked vertically in the dashboard. We used the search field (above the diagrams) to show any that had the name “mnist” in their folder name—we’ll configure that in a moment. TensorBoard can load data from multiple models at once and you can choose which to visualize. This makes it easy to compare several different models or multiple runs of the same model.

Copy the MNIST Convnet’s Notebook

To create the new notebook for this example:

1. Right-click the `MNIST_CNN.ipynb` notebook in JupyterLab’s **File Browser** tab and select **Duplicate** to make a copy of the notebook.

2. Right-click the new notebook named `MNIST_CNN-Copy1.ipynb`, then select **Rename**, enter the name `MNIST_CNN_TensorBoard.ipynb` and press *Enter*.

Open the notebook by double-clicking its name.

Configuring Keras to Write the TensorBoard Log Files

To use TensorBoard, before you `fit` the model, you need to configure a **TensorBoard** object (module `tensorflow.keras.callbacks`), which the model will use to write data into a specified folder that TensorBoard monitors. This object is known as a **callback** in Keras. In the notebook, click to the left of snippet that calls the model's `fit` method, then type `a`, which is the shortcut for adding a new code cell *above* the current cell (use `b` for *below*). In the new cell, enter the following code to create the `TensorBoard` object:

[lick here to view code image](#)

```
from tensorflow.keras.callbacks import TensorBoard
import time

tensorboard_callback = TensorBoard(log_dir=f'./logs/mnist{time.time()}',
                                   histogram_freq=1, write_graph=True)
```

The arguments are:

- `log_dir`—The name of the folder in which this model's log files will be written. The notation `'./logs/'` indicates that we're creating a new folder within the logs folder you created previously, and we follow that with `'mnist'` and the current time. This ensures that each new execution of the notebook will have its own log folder. That will enable you to compare multiple executions in TensorBoard.
- `histogram_freq`—The frequency in *epochs* that Keras will output to the model's log files. In this case, we'll write data to the logs for every epoch.
- `write_graph`—When this is true, a graph of the model will be output. You can view the graph in the **GRAPHS** tab in TensorBoard.

Updating Our Call to `fit`

Finally, we need to modify the original `fit` method call in snippet 37. For this example, we set the number of epochs to 10, and we added the `callbacks` argument, which is a list of callback objects ^o:

^oYou can view Keras other callbacks at <https://keras.io/callbacks/>.

[lick here to view code image](#)

```
cnn.fit(X_train, y_train, epochs=10, batch_size=64,
        validation_split=0.1, callbacks=[tensorboard_callback])
```

You can now re-execute the notebook by selecting **Kernel > Restart Kernel and Run All**

Cells in JupyterLab. After the first epoch completes, you'll start to see data in TensorBoard.

15.8 CONVNETJS: BROWSER-BASED DEEP-LEARNING TRAINING AND VISUALIZATION

In this section, we'll overview Andrej Karpathy's JavaScript-based ConvnetJS tool for training and visualizing convolutional neural networks in your web browser: ¹

¹You also can download ConvnetJS from GitHub at
`https://github.com/karpathy/convnetjs`.

[lick here to view code image](#)

```
https://cs.stanford.edu/people/karpathy/convnetjs/
```

You can run the ConvnetJS sample convolutional neural networks or create your own. We've used the tool on several desktop, tablet and phone browsers.

The ConvnetJS MNIST demo trains a convolutional neural network using the MNIST dataset we presented in [ection 15.6](#). The demo presents a scrollable dashboard that updates dynamically as the model trains and contains several sections.

Training Stats

This section contains a **Pause** button that enables you to stop the learning and “freeze” the current dashboard visualizations. Once you pause the demo, the button text changes to **resume**. Clicking the button again continues training. This section also presents training statistics, including the training and validation accuracy and a graph of the training loss.

Instantiate a Network and Trainer

In this section, you'll find the JavaScript code that creates the convolutional neural network. The default network has similar layers to the convnet in [ection 15.6](#). The Conv-netJS documentation ² shows the supported layer types and how to configure them. You can experiment with different layer configurations in the provided textbox and begin training an updated network by clicking the **change network** button.

² `https://cs.stanford.edu/people/karpathy/convnetjs/docs.html`.

Network Visualization

This key section shows one training image at a time and how the network processes that image through each layer. Click the **Pause** button to inspect all the layers' outputs for a given digit to get a sense of what the network “sees” as it learns. The network's last layer produces the probabilistic classifications. It shows 10 squares—9 black and 1 white, indicating the predicted class of the current digit image.

Example Predictions on Test Set

The final section shows a random selection of the test set images and the top three possible classes for each digit. The one with the highest probability is shown on a green bar and the

other two are displayed on red bars. The length of each bar is a visual indication of that class's probability.

15.9 RECURRENT NEURAL NETWORKS FOR SEQUENCES; SENTIMENT ANALYSIS WITH THE IMDB DATASET

In the MNIST CNN network, we focused on stacked layers that were applied *sequentially*. Non-sequential models are possible, as you'll see here with *recurrent neural networks*. In this section, we use Keras's bundled IMDB (the Internet Movie Database) movie reviews dataset ³ to perform **binary classification**, predicting whether a given review's sentiment is positive or negative.

³Maas, Andrew L. and Daly, Raymond E. and Pham, Peter T. and Huang, Dan and Ng, Andrew Y. and Potts, Christopher, "Learning Word Vectors for Sentiment Analysis," *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, June 2011. Portland, Oregon, USA. Association for Computational Linguistics, pp. 142150. <http://www.aclweb.org/anthology/P11-015>.

We'll use a **recurrent neural network (RNN)**, which processes sequences of data, such as time series or text in sentences. The term "recurrent" comes from the fact that the neural network contains *loops* in which the output of a given layer becomes the input to that same layer in the next **time step**. In a time series, a time step is the next point in time. In a text sequence, a "time step" would be the next word in a sequence of words.

The looping in RNNs enables them to learn and remember relationships among the data in the sequence. For example, consider the following sentences we used in the "Natural Language Processing" chapter. The sentence

```
The food is not good.
```

clearly has negative sentiment. Similarly, the sentence

```
The movie was good.
```

has positive sentiment, though not as positive as

```
The movie was excellent!
```

In the first sentence, the word "good" on its own has positive sentiment. However, when *preceded by* "not," which appears earlier in the sequence, the sentiment becomes negative. RNNs take into account the relationships among the earlier and later parts of a sequence.

In the preceding example, the words that determined sentiment were adjacent. However, when determining the meaning of text there can be many words to consider and an arbitrary number of words in between them. In this section, we'll use a **Long Short-Term Memory (LSTM)** layer, which makes the neural network *recurrent* and is optimized to handle

learning from sequences like the ones we described above.

RNNs have been used for many tasks including: ^{4, 5, 6}

⁴ <https://www.analyticsindiamag.com/overview-of-recurrent-neural-networks-and-their-applications/>.

⁵ https://en.wikipedia.org/wiki/Recurrent_neural_network#Applications.

⁶ <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>.

- predictive text input—displaying possible next words as you type,
- sentiment analysis,
- responding to questions with the predicted best answers from a corpus,
- inter-language translation, and
- automated closed captioning in video.

15.9.1 Loading the IMDb Movie Reviews Dataset

The IMDb movie reviews dataset included with Keras contains 25,000 training samples and 25,000 testing samples, each labeled with its positive (1) or negative (0) sentiment. Let's import the `tensorflow.keras.datasets.imdb` module so we can load the dataset:

[lick here to view code image](#)

```
[1]: from tensorflow.keras.datasets import imdb
```

The `imdb` module's `load_data` function returns the IMDb training and testing sets. There are over 88,000 unique words in the dataset. The `load_data` function enables you to specify the number of unique words to import as part of the training and testing data. In this case, we loaded only the top 10,000 most frequently occurring words due to the memory limitations of our system and the fact that we're (intentionally) training on a CPU rather than a GPU (because most of our readers will not have access to systems with GPUs and TPUs). The more data you load, the longer training will take, but more data may help produce better models:

[lick here to view code image](#)

```
[2]: number_of_words = 10000

[3]: (X_train, y_train), (X_test, y_test) = imdb.load_data(
    num_words=number_of_words)
```

The `load_data` function returns a tuple of two elements containing the training and testing sets. Each element is itself a tuple containing the samples and labels, respectively. In a given review, `load_data` replaces any words outside the top 10,000 with a placeholder value, which we'll discuss shortly.

15.9.2 Data Exploration

Let's check the dimensions of the training set samples (`X_train`), training set labels (`y_train`), testing set samples (`X_test`) and testing set labels (`y_test`):

[click here to view code image](#)

```
[4]: X_train.shape
[4]: (25000,)

[5]: y_train.shape
[5]: (25000,)

[6]: X_test.shape
[6]: (25000,)

[7]: y_test.shape
[7]: (25000,)
```

The arrays `y_train` and `y_test` are one-dimensional arrays containing 1s and 0s, indicating whether each review is positive or negative. Based on the preceding outputs, `X_train` and `X_test` also appear to be one-dimensional. However, their elements actually are *lists* of integers, each representing one review's contents, as shown in snippet [9]: ⁷

⁷Here we used the `%pprint` magic to turn off pretty printing so the following snippets output could be displayed horizontally rather than vertically to save space. You can turn pretty printing back on by re-executing the `%pprint` magic.

[click here to view code image](#)

```
[8]: %pprint
[8]: Pretty printing has been turned OFF

[9]: X_train[123]
[9]: [1, 307, 5, 1301, 20, 1026, 2511, 87, 2775, 52, 116, 5, 31, 7, 4, 91, 12
```

Keras deep learning models require *numeric data*, so the Keras team preprocessed the IMDb dataset for you.

Movie Review Encodings

Because the movie reviews are numerically encoded, to view their original text, you need to know the word to which each number corresponds. Keras's IMDb dataset provides a dictionary that maps the words to their indexes. Each word's corresponding value is its frequency ranking among all the words in the entire set of reviews. So the word with the ranking 1 is the most frequently occurring word (calculated by the Keras team from the dataset), the word with ranking 2 is the second most frequently occurring word, and so on.

Though the dictionary values begin with 1 as the most frequently occurring word, in each encoded review (like `X_train[123]` shown previously), the ranking values are *offset by 3*. So any review containing the most frequently occurring word will have the value 4 wherever that word appears in the review. Keras reserves the values 0, 1 and 2 in each encoded review

for the following purposes:

- The value 0 in a review represents *padding*. Keras deep learning algorithms expect all the training samples to have the same dimensions, so some reviews may need to be expanded to a given length and some shortened to that length. Reviews that need to be expanded are padded with 0s.
- The value 1 represents a token that Keras uses internally to indicate the start of a text sequence for learning purposes.
- The value 2 in a review represents an unknown word—typically a word that was not loaded because you called `load_data` with the `num_words` argument. In this case, any review that contained words with frequency rankings greater than `num_words` would have those words' numeric values replaced with 2. This is all handled by Keras when you load the data.

Because each review's numeric values are offset by 3, we'll have to account for this when we decode the review.

Decoding a Movie Review

Let's decode a review. First, get the word-to-index dictionary by calling the function `get_word_index` from the `tensorflow.keras.datasets.imdb` module:

[lick here to view code image](#)

```
[10]: word_to_index = imdb.get_word_index()
```

The word `'great'` might appear in a positive movie review, so let's see whether it's in the dictionary:

[lick here to view code image](#)

```
[11]: word_to_index['great']  
[11]: 84
```

According to the output, `'great'` is the dataset's 84th most frequent word. If you look up a word that's not in the dictionary, you'll get an exception.

To transform the frequency ratings into words, let's first reverse the `word_to_index` dictionary's mapping, so we can look up every word by its frequency rating. The following dictionary comprehension reverses the mapping:

[lick here to view code image](#)

```
[12]: index_to_word = \n      {index: word for (word, index) in word_to_index.items() }
```

Recall that a dictionary's `items` method enables us to iterate through tuples of key-value

pairs. We unpack each tuple into the variables `word` and `index`, then create an entry in the new dictionary with the expression `index: word`.

The following list comprehension gets the top 50 words from the new dictionary—recall that the most frequent word has the value 1:

[lick here to view code image](#)

```
[13]: [index_to_word[i] for i in range(1, 51)]
[13]: ['the', 'and', 'a', 'of', 'to', 'is', 'br', 'in', 'it', 'i', 'this', 't
```

ote that most of these are *stop words*. Depending on the application, you might want to remove or keep the stop words. For example, if you were creating a predictive-text application that suggests the next word in a sentence the user is typing, you'd want to keep the stop words so they can be displayed as predictions.

Now, we can decode a review. We use the `index_to_word` dictionary's two-argument method `get` rather than the `[]` operator to get value for each key. If a value is not in the dictionary, the `get` method returns its second argument, rather than raising an exception. The argument `i - 3` accounts for the offset in the encoded reviews of each review's frequency ratings. When the Keras reserved values 0–2 appear in a review, `get` returns '?'; otherwise, `get` returns the word with the key `i - 3` in the `index_to_word` dictionary:

[lick here to view code image](#)

```
[14]: ' '.join([index_to_word.get(i - 3, '?') for i in X_train[123]])
[14]: '? beautiful and touching movie rich colors great settings good
      acting and one of the most charming movies i have seen in a while i
      never saw such an interesting setting when i was in china my wife
      liked it so much she asked me to ? on and rate it so other would
      enjoy too'
```

We can see from the `y_train` array that this review is classified as positive:

```
[15]: y_train[123]
[15]: 1
```

15.9.3 Data Preparation

The number of words per review varies, but the Keras requires all samples to have the same dimensions. So, we need to perform some data preparation. In this case, we need to restrict every review to the *same* number of words. Some reviews will need to be *padded* with additional data and others will need to be *truncated*. The `pad_sequences` utility function (module `tensorflow.keras.preprocessing.sequence`) reshapes `X_train`'s samples (that is, its rows) to the number of features specified by the `maxlen` argument (200) and returns a two-dimensional array:

[lick here to view code image](#)


```
[16]: words_per_review = 200

[17]: from tensorflow.keras.preprocessing.sequence import pad_sequences

[18]: X_train = pad_sequences(X_train, maxlen=words_per_review)
```

If a sample has more features, `pad_sequences` truncates it to the specified length. If a sample has fewer features, `pad_sequences` adds 0s to the beginning of the sequence to pad it to the specified length. Let's confirm `X_train`'s new shape:

```
[19]: X_train.shape
[19]: (25000, 200)
```

We also must reshape `X_test` for later in this example when we evaluate the model:

[lick here to view code image](#)

```
[20]: X_test = pad_sequences(X_test, maxlen=words_per_review)

[21]: X_test.shape
[21]: (25000, 200)
```

Splitting the Test Data into Validation and Test Data

In our convnet, we used the `fit` method's `validation_split` argument to indicate that 10% of our training data should be set aside to validate the model as it trains. For this example, we'll manually split the 25,000 test samples into 20,000 test samples and 5,000 validation samples. We'll then pass the 5,000 validation samples to the model's `fit` method via the argument `validation_data`. Let's use Scikit-learn's `train_test_split` function from the previous chapter to split the test set:

[lick here to view code image](#)

```
[22]: from sklearn.model_selection import train_test_split
      X_test, X_val, y_test, y_val = train_test_split(
          X_test, y_test, random_state=11, test_size=0.20)
```

Let's also confirm the split by checking `X_test`'s and `X_val`'s shapes:

```
[23]: X_test.shape
[23]: (20000, 200)

[24]: X_val.shape
[24]: (5000, 200)
```

15.9.4 Creating the Neural Network

Next, we'll configure the RNN. Once again, we begin with a `Sequential` model to which we'll add the layers that compose our network:

[lick here to view code image](#)

```
[25]: from tensorflow.keras.models import Sequential

[26]: rnn = Sequential()
```

Next, let's import the layers we'll use in this model:

[lick here to view code image](#)

```
[27]: from tensorflow.keras.layers import Dense, LSTM

[28]: from tensorflow.keras.layers.embeddings import Embedding
```

Adding an Embedding Layer

Previously, we used *one-hot encoding* to convert the MNIST dataset's integer labels into *categorical* data. The result for each label was a vector in which all but one element was 0. We could do that for the index values that represent our words. However, this example processes 10,000 unique words. That means we'd need a 10,000-by-10,000 array to represent all the words. That's 100,000,000 elements, and almost all the array elements would be 0. This is not an efficient way to encode the data. If we were to process all 88,000+ unique words in the dataset, we'd need an array of nearly *eight billion* elements!

To *reduce dimensionality*, RNNs that process text sequences typically begin with an **embedding layer** that encodes each word in a more compact *dense-vector representation*. The vectors produced by the embedding layer also capture the word's context—that is, how a given word relates to the words around it. So the embedding layer enables the RNN to learn word relationships among the training data.

There are also **predefined word embeddings**, such as **Word2Vec** and **GloVe**. You can load these into neural networks to save training time. They're also sometimes used to add basic word relationships to a model when smaller amounts of training data are available. This can improve the model's accuracy by allowing it to build upon previously learned word relationships, rather than trying to learn those relationships with insufficient amounts of data.

Let's create an **Embedding** layer (module `tensorflow.keras.layers`):

[lick here to view code image](#)

```
[29]: rnn.add(Embedding(input_dim=number_of_words, output_dim=128,
                        input_length=words_per_review))
```

The arguments are:

- `input_dim`—The number of unique words.
- `output_dim`—The size of each word embedding. If you load pre-existing embeddings ⁸

like *Word2Vec* and *GloVe*, you must set this to match the size of the word embeddings you load.

⁸ <https://blog.keras.io/using-pre-trained-word-embeddings-in-a-keras-model.html>.

- `input_length=words_per_review`—The number of words in each input sample.

Adding an LSTM Layer

Next, we'll add an LSTM layer:

[lick here to view code image](#)

```
[30]: rnn.add(LSTM(units=128, dropout=0.2, recurrent_dropout=0.2))
```

The arguments are:

- `units`—The number of neurons in the layer. The more neurons the more the network can remember. As a guideline, you can start with a value between the length of the sequences you're processing (200 in this example) and the number of classes you're trying to predict (2 in this example).⁹

⁹ <https://towardsdatascience.com/choosing-the-right-hyperparameters-for-a-simple-lstm-using-keras-f8e9ed76f046>.

- `dropout`—The percentage of neurons to randomly disable when processing the layer's input and output. Like the pooling layers in our convnet, **dropout** is a proven technique^{0, 1} that reduces overfitting. Keras provides a **Dropout** layer that you can add to your models.

⁰Yarin, Ghahramani, and Zoubin. A Theoretically Grounded Application of Dropout in Recurrent Neural Networks. October 05, 2016.

<https://arxiv.org/abs/1512.05287>.

¹Srivastava, Nitish, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* 15 (June 14, 2014): 1929-1958.

<http://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>.

- `recurrent_dropout`—The percentage of neurons to randomly disable when the layer's output is fed back into the layer again to allow the network to learn from what it has seen previously.

The mechanics of how the LSTM layer performs its task are beyond the scope of this book. Chollet says: “you don't need to understand anything about the specific architecture of an LSTM cell; as a human, it shouldn't be your job to understand it. Just keep in mind what the LSTM cell is meant to do: allow past information to be reinjected at a later time.”²

Adding a Dense Output Layer

Finally, we need to take the LSTM layer's output and reduce it to one result indicating whether a review is positive or negative, thus the value 1 for the `units` argument. Here we use the '**sigmoid**' **activation function**, which is preferred for binary classification.³ It reduces arbitrary values into the range 0.0–1.0, producing a probability:

³Chollet, François. *Deep Learning with Python*. p.114. Shelter Island, NY: Manning Publications, 2018.

[lick here to view code image](#)

```
[31]: rnn.add(Dense(units=1, activation='sigmoid'))
```

Compiling the Model and Displaying the Summary

Next, we compile the model. In this case, there are only two possible outputs, so we use the **binary_crossentropy loss function**:

[lick here to view code image](#)

```
[32]: rnn.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
```

The following is the summary of our model. Notice that even though we have fewer layers than our convnet, the RNN has nearly three times as many trainable parameters (the network's weights) as the convnet and more parameters means more training time. The large number of parameters primarily comes from the number of words in the vocabulary (we loaded 10,000) times the number of neurons in the `Embedding` layer's output (128):

[lick here to view code image](#)

```
[33]: rnn.summary()
```

Layer (type)	Output Shape	Param #
=====		
embedding_1 (Embedding)	(None, 200, 128)	1280000

lstm_1 (LSTM)	(None, 128)	131584

dense_1 (Dense)	(None, 1)	129
=====		
Total params: 1,411,713		
Trainable params: 1,411,713		
Non-trainable params: 0		

15.9.5 Training and Evaluating the Model

Let's train our model. ⁴ Notice for each epoch that the model takes significantly longer to train than our convnet did. This is due to the larger numbers of parameters (weights) our RNN model needs to learn. We bolded the accuracy (`acc`) and validation accuracy (`val_acc`) values for readability—these represent the percentage of training samples and the percentage of `validation_data` samples that the model predicts correctly.

⁴At the time of this writing, TensorFlow displayed a warning when we executed this statement. This is a known TensorFlow issue and, according to the forums, you can safely ignore the warning.

[lick here to view code image](#)

```
[34]: rnn.fit(X_train, y_train, epochs=10, batch_size=32,
            validation_data=(X_test, y_test))
Train on 25000 samples, validate on 5000 samples
Epoch 1/5
25000/25000 [=====] - 299s   12ms/step - loss: 0.6574
poch 2/5
25000/25000 [=====] - 298s   12ms/step - loss: 0.4577
poch 3/5
25000/25000 [=====] - 296s   12ms/step - loss: 0.3277
poch 4/5
25000/25000 [=====] - 307s   12ms/step - loss: 0.2675
poch 5/5
25000/25000 [=====] - 310s   12ms/step - loss: 0.2217
[34]: <tensorflow.python.keras.callbacks.History object at 0xb3ba882e8>
```

Finally, we can evaluate the results using the test data. Function `evaluate` returns the loss and accuracy values. In this case, the model was 85.99% accurate:

[lick here to view code image](#)

```
[35]: results = rnn.evaluate(X_test, y_test)
20000/20000 [=====] - 42s 2ms/step

[36]: results
[36]: [0.3415240607559681, 0.8599]
```

Note that the accuracy of this model seems low compared to our MNIST convnet's results, but this is a much more difficult problem. If you search online for other IMDB sentiment-analysis binary-classification studies, you'll find lots of results in the high 80s. So we did reasonably well with our small recurrent neural network of only three layers. You might want to study some online models and try to produce a better model.

15.10 TUNING DEEP LEARNING MODELS

In section 15.9.5, notice in the `fit` method's output that both the testing accuracy (85.99%) and validation accuracy (87.04%) were significantly less than the 90.83% training accuracy. Such disparities are usually the result of overfitting, so there is plenty of room for improvement in our model. ⁵ ⁶ If you look at the output of each epoch, you'll notice both the training and validation accuracy continue to increase. Recall that training for too many

pochs can lead to overfitting, but it's possible we have not yet trained enough. Perhaps one hyperparameter tuning option for this model would be to increase the number of epochs.

⁵ <https://towardsdatascience.com/deep-learning-overfitting-46bf5b35e24>.

⁶ <https://hackernoon.com/memorizing-is-not-learning-6-tricks-to-revent-overfitting-in-machine-learning-820b091dc42>.

Some variables that affect your models' performance include:

- having more or less data to train with
- having more or less to test with
- having more or less to validate with
- having more or fewer layers
- the types of layers you use
- the order of the layers

In our IMDB RNN example, some things we could tune include:

- trying different amounts of the training data—we used only the top 10,000 words
- different numbers of words per review—we used only 200,
- different numbers of neurons in our layers,
- more layers or
- possibly loading pre-trained word vectors rather than having our Embedding layer learn them from scratch.

The compute time required to train models multiple times is significant so, in deep learning, you generally do not tune hyperparameters with techniques like k-fold cross-validation or grid search. ⁷ There are various tuning techniques, ⁸, ⁹, ⁰, ¹ but one particularly promising area is automated machine learning (AutoML). For example, the Auto-Keras ² library is specifically geared to automatically choosing the best configurations for your Keras models. Google's Cloud AutoML and Baidu's EZDL are among various other automated machine learning efforts.

⁷ <https://www.quora.com/Is-cross-validation-heavily-used-in-deep-learning-or-is-it-too-expensive-to-be-used>.

⁸ <https://towardsdatascience.com/what-are-hyperparameters-and-how-to-une-the-hyperparameters-in-a-deep-neural-network-d0604917584a>.

⁹ <https://medium.com/machine-learning-bites/deeplearning-series-deep-neural-networks-tuning-and-optimization-39250ff7786d>.

⁰ <https://flyyufelix.github.io/2016/10/03/fine-tuning-in-keras-art1.html> and <https://flyyufelix.github.io/2016/10/08/fine-tuning-in-keras-part2.html>.

¹ <https://towardsdatascience.com/a-comprehensive-guide-on-how-to-fine-tune-deep-neural-networks-using-keras-on-google-colab-free-aaaa0aced8f>.

² <https://autokeras.com/>.

5.11 CONVNET MODELS PRETRAINED ON IMAGENET

With deep learning, rather than starting fresh on every project with costly training, validating and testing, you can use **pretrained deep neural network models** to:

- make new predictions,
- continue training them further with new data or
- transfer the weights learned by a model for a similar problem into a new model—this is called *transfer learning*.

Keras Pretrained Convnet Models

Keras comes bundled with the following pretrained convnet models, ³ each pretrained on Image-Net ⁴—a growing dataset of 14+ million images:

³ <https://keras.io/applications/>.

⁴ <http://www.image-net.org>.

- Xception
- VGG16
- VGG19
- ResNet50
- Inception v3
- Inception-ResNet v2
- MobileNet v1
- DenseNet
- NASNet

- MobileNet v2

Reusing Pretrained Models

ImageNet is too big for efficient training on most computers, so most people interested in using it start with one of the smaller pretrained models.

You can reuse just the architecture of each model and train it with new data, or you can reuse the pretrained weights. For a few simple examples, see:

```
https://keras.io/applications/
```

ImageNet Challenge

In the end-of-chapter projects, you'll research and use some of these bundled models. You'll also investigate the *ImageNet Large Scale Visual Recognition Challenge* for evaluating object-detection and image-recognition models.⁵ This competition ran from 2010 through 2017. ImageNet now has a continuously running challenge on the Kaggle competition site called the *ImageNet Object Localization Challenge*.⁶ The goal is to identify "all objects within an image, so those images can then be classified and annotated." ImageNet releases the current participants leaderboard once per quarter.

⁵ <http://www.image-net.org/challenges/LSVRC/>.

⁶ <https://www.kaggle.com/c/imagenet-object-localization-challenge>.

A lot of what you've seen in the machine learning and deep learning chapters is what the Kaggle competition website is all about. There's no obvious optimal solution for many machine learning and deep learning tasks. People's creativity is really the only limit. On Kaggle, companies and organizations fund competitions where they encourage people worldwide to develop better-performing solutions than they've been able to do for something that's important to their business or organization. Sometimes companies offer prize money, which has been as high as \$1,000,000 on the famous Netflix competition. Netflix wanted to get a 10% or better improvement in their model for determining whether people will like a movie, based on how they rated previous ones.⁷ They used the results to help make better recommendations to members. Even if you do not win a Kaggle competition, it's a great way to get experience working on problems of current interest.

⁷ <https://netflixprize.com/rules.html>.

15.12 WRAP-UP

In chapter 16, you peered into the future of AI. Deep Learning has captured the imagination of the computer-science and data science-communities. This may be the most important AI chapter in the book.

We mentioned the key deep-learning platforms, indicating that Google's TensorFlow is the most widely used. We discussed why Keras, which presents a friendly interface to TensorFlow, has become so popular.

e set up a custom Anaconda environment for TensorFlow, Keras and JupyterLab, then used the environment to implement the Keras examples.

We explained what tensors are and why they're crucial to deep learning. We discussed the basics of neurons and multi-layered neural networks for building Keras deep-learning models. We considered some popular types of layers and how to order them.

We introduced convolutional neural networks (convnets) and indicated that they're especially appropriate for computer-vision applications. We then built, trained, validated and tested a convnet using the MNIST database of handwritten digits for which we achieved 99.17% prediction accuracy. This is remarkable, given that we achieved it by working with a only a basic model and without doing any hyperparameter tuning. You can try more sophisticated models and tune the hyperparameters to try to achieve better performance. We listed a variety of intriguing computer vision tasks.

We introduced TensorBoard for visualizing TensorFlow and Keras neural network training and validation. We also discussed ConvnetJS, a browser-based convnet training and visualization tool, which enables you to peek inside the training process.

Next, we presented recurrent neural networks (RNNs) for processing sequences of data, such as time series or text in sentences. We used an RNN with the IMDB movie reviews dataset to perform binary classification, predicting whether each review's sentiment was positive or negative. We also discussed tuning deep learning models and how high-performance hardware, like NVIDIA's GPUs and Google's TPUs, is making it possible for more people to tackle more substantial deep-learning studies.

Given how costly and time-consuming it is to train deep-learning models, we explained the strategy of using pretrained models. We listed various Keras convnet image-processing models that were trained on the massive ImageNet dataset, and discussed how transfer learning enables you to use these models to create new ones quickly and effectively. Deep learning is a large, complex topic. We focused on the basics in the chapter.

In the next chapter, we present the big data infrastructure that supports the kinds of AI technologies we've discussed in chapters 12 through 15. We'll consider the Hadoop and Spark platforms for big data batch processing and real-time streaming applications. We'll look at relational databases and the SQL language for querying them—these have dominated the database field for many decades. We'll discuss how big data presents challenges that relational databases don't handle well, and consider how NoSQL databases are designed to handle those challenges. We'll conclude the book with a discussion of the Internet of Things (IoT), which will surely be the world's largest big-data source and will present many opportunities for entrepreneurs to develop leading-edge businesses that will truly make a difference in people's lives.



16. Big Data: Hadoop, Spark, NoSQL and IoT

Objectives

In this chapter you'll:

- Understand what big data is and how quickly it's getting bigger.
- Manipulate a SQLite relational database using Structured Query Language (SQL).
- Understand the four major types of NoSQL databases.
- Store tweets in a MongoDB NoSQL JSON document database and visualize them on a Folium map.
- Understand Apache Hadoop and how it's used in big-data batch-processing applications.
- Build a Hadoop MapReduce application on Microsoft's Azure HDInsight cloud service.
- Understand Apache Spark and how it's used in high-performance, real-time big-data applications.
- Use Spark streaming to process data in mini-batches.
- Understand the Internet of Things (IoT) and the publish/subscribe model.
- Publish messages from a simulated Internet-connected device and visualize its messages in a dashboard.
- Subscribe to PubNub's live Twitter and IoT streams and visualize the data.

outline

6.1 Introduction

6.2 Relational Databases and Structured Query Language (SQL)

6.3.1 A books Database

6.3.2 SELECT Queries

6.3.3 WHERE Clause

6.3.4 ORDER BY Clause

6.3.5 Merging Data from Multiple Tables: INNER JOIN

6.3.6 INSERT INTO Statement

16.3.7 UPDATE Statement

16.3.8 DELETE FROM Statement

6.3 NoSQL and NewSQL Big-Data Databases: A Brief Tour

6.3.1 NoSQL Key–Value Databases

6.3.2 NoSQL Document Databases

6.3.3 NoSQL Columnar Databases

6.3.4 NoSQL Graph Databases

6.3.5 NewSQL Databases

6.4 Case Study: A MongoDB JSON Document Database

6.4.1 Creating the MongoDB Atlas Cluster

6.4.2 Streaming Tweets into MongoDB

6.5 Hadoop

6.5.1 Hadoop Overview

6.6.2 Summarizing Word Lengths in *Romeo and Juliet* via MapReduce

6.5.3 Creating an Apache Hadoop Cluster in Microsoft Azure HDInsight

6.5.4 Hadoop Streaming

6.5.5 Implementing the Mapper

6.5.6 Implementing the Reducer

6.5.7 Preparing to Run the MapReduce Example

6.5.8 Running the MapReduce Job

6.6 Spark

6.6.1 Spark Overview

6.6.2 Docker and the Jupyter Docker Stacks

6.6.3 Word Count with Spark

6.6.4 Spark Word Count on Microsoft Azure

6.7 Spark Streaming: Counting Twitter Hashtags Using the `pyspark-notebook` Docker
tack

6.7.1 Streaming Tweets to a Socket

6.7.2 Summarizing Tweet Hashtags; Introducing Spark SQL

6.8 Internet of Things and Dashboards

6.8.1 Publish and Subscribe

6.8.2 Visualizing a PubNub Sample Live Stream with a Freeboard Dashboard

6.8.3 Simulating an Internet-Connected Thermostat in Python

6.8.4 Creating the Dashboard with Freeboard.io

6.8.5 Creating a Python PubNub Subscriber

6.9 Wrap-Up

16.1 INTRODUCTION

In [Section 1.7](#), we introduced big data. In this capstone chapter, we discuss popular hardware and software infrastructure for working with big data, and we develop complete applications on several desktop and cloud-based big-data platforms.

Databases

Databases are critical big-data infrastructure for storing and manipulating the massive amounts of data we’re creating. They’re also critical for securely and confidentially maintaining that data, especially in the context of ever-stricter privacy laws such as **HIPAA (Health Insurance Portability and Accountability Act)** in the United States and **GDPR (General Data Protection Regulation)** for the European Union.

First, we’ll present **relational databases**, which store **structured data** in tables with a fixed-size number of columns per row. You’ll manipulate relational databases via **Structured Query Language (SQL)**.

Most data produced today is **unstructured data**, like the content of Facebook posts and Twitter tweets, or **semi-structured data** like JSON and XML documents. Twitter processes each tweet’s contents into a semi-structured JSON document with lots of *metadata*, as you saw in the “Data Mining Twitter” chapter. Relational databases are not geared to the unstructured and semi-structured data in big-data applications. So, as big data evolved, new kinds of databases were created to handle such data efficiently. We’ll discuss the four major types of these **NoSQL databases**—key–value, document, columnar and graph databases. Also, we’ll overview **NewSQL databases**, which blend the benefits of relational and NoSQL databases. Many NoSQL and NewSQL vendors make it easy to get started with their products through free tiers and free trials, and typically in cloud-based environments that require minimal installation and setup. This makes it practical for you to gain big-data experience before “diving in.”

Apache Hadoop

Much of today’s data is so large that it cannot fit on one system. As big data grew, we needed

istributed data storage and parallel processing capabilities to process the data more efficiently. This led to complex technologies like Apache Hadoop for distributed data processing with massive parallelism among clusters of computers where the intricate details are handled for you automatically and correctly. We'll discuss Hadoop, its architecture and how it's used in big-data applications. We'll guide you through configuring a multi-node Hadoop cluster using the Microsoft Azure HDInsight cloud service, then use it to execute a Hadoop MapReduce job that you'll implement in Python. Though HDInsight is not free, Microsoft gives you a generous new-account credit that should enable you to run the chapter's code examples without incurring additional charges.

Apache Spark

As big-data processing needs grow, the information-technology community is continually looking for ways to increase performance. Hadoop executes tasks by breaking them into pieces that do lots of disk I/O across many computers. Spark was developed as a way to perform certain big-data tasks *in memory* for better performance.

We'll discuss Apache Spark, its architecture and how it's used in high-performance, real-time big-data applications. You'll implement a Spark application using functional-style filter/map/reduce programming capabilities. First, you'll build this example using a Jupyter Docker stack that runs locally on your desktop computer, then you'll implement it using a cloud-based Microsoft Azure HDInsight multi-node Spark cluster.

We'll introduce Spark streaming for processing streaming data in mini-batches. Spark streaming gathers data for a short time interval you specify, then gives you that batch of data to process. You'll implement a Spark streaming application that processes tweets. In that example, you'll use Spark SQL to query data stored in a Spark `DataFrame` which, unlike pandas `DataFrames`, may contain data distributed over many computers in a cluster.

Internet of Things

We'll conclude with an introduction to the Internet of Things (IoT)—billions of devices that are continuously producing data worldwide. We'll introduce the *publish/subscribe model* that IoT and other types of applications use to connect data users with data providers. First, without writing any code, you'll build a web-based dashboard using Freeboard.io and a sample live stream from the PubNub messaging service. Next, you'll simulate an Internet-connected thermostat which publishes messages to the free Dweet.io messaging service using the Python module Dweepy, then create a dashboard visualization of the data with Freeboard.io. Finally, you'll build a Python client that *subscribes* to a sample live stream from the PubNub service and dynamically visualizes the stream with Seaborn and a Matplotlib `FuncAnimation`.

Experience Cloud and Desktop Big-Data Software

Cloud vendors focus on **service-oriented architecture (SOA)** technology in which they provide “as-a-Service” capabilities that applications connect to and use in the cloud. Common services provided by cloud vendors include: ¹

¹ For more as-a-Service acronyms, see
https://en.wikipedia.org/wiki/Cloud_computing and
https://en.wikipedia.org/wiki/As_a_service.

“As-a-Service” acronyms (note that several are the same)

Big data as a Service (BDaaS)	Platform as a Service (PaaS)
Hadoop as a Service (HaaS)	Software as a Service (SaaS)
Hardware as a Service (HaaS)	Storage as a Service (SaaS)
Infrastructure as a Service (IaaS)	Spark as a Service (SaaS)

You’ll get hands-on experience in this chapter with several cloud-based tools. In this chapter’s examples, you’ll use the following platforms:

- A *free* MongoDB Atlas cloud-based cluster.
- A multi-node Hadoop cluster running on Microsoft’s Azure HDInsight cloud-based service—for this you’ll use the *credit* that comes with a new Azure account.
- A *free* single-node Spark “cluster” running on your desktop computer, using a Jupyter Docker-stack container.
- A multi-node Spark cluster, also running on Microsoft’s Azure HDInsight—for this you’ll continue using your Azure new-account *credit*.

There are many other options, including cloud-based services from Amazon Web Services, Google Cloud and IBM Watson, and the free *desktop* versions of the Hortonworks and Cloudera platforms (there also are cloud-based paid versions of these). You also could try a single-node Spark cluster running on the *free* cloud-based Databricks Community Edition. Spark’s creators founded Databricks.

Always check the latest terms and conditions of each service you use. Some require you to enable credit-card billing to use their clusters. *Caution:* Once you allocate Microsoft Azure HDInsight clusters (or other vendors’ clusters), they incur costs. When you complete the case studies using services such as Microsoft Azure, be sure to delete your cluster(s) and their other resources (like storage). This will help extend the life of your Azure new-account credit.

Installation and setups vary across platforms and over time. Always follow each vendor’s latest steps. If you have questions, the best sources for help are the vendor’s support capabilities and forums. Also, check sites such as `stackoverflow.com`—other people may have asked questions about similar problems and received answers from the developer community.

Algorithms and Data

Algorithms and data are the core of Python programming. The first few chapters of this book were mostly about algorithms. We introduced control statements and discussed algorithm development. Data was small—primarily individual integers, floats and strings. hapters 5–

emphasized *structuring* data into lists, tuples, dictionaries, sets, arrays and files.

Data's Meaning

But, what about the *meaning* of the data? Can we use the data to gain insights to better diagnose cancers? Save lives? Improve patients' quality of life? Reduce pollution? Conserve water? Increase crop yields? Reduce damage from devastating storms and fires? Develop better treatment regimens? Create jobs? Improve company profitability?

The data-science case studies of [chapters 11– 5](#) all focused on AI. In this chapter, we focus on the big-data infrastructure that supports AI solutions. As the data used with these technologies continues growing exponentially, we want to learn from that data and do so at blazing speed. We'll accomplish these goals with a combination of sophisticated algorithms, hardware, software and networking designs. We've presented various machine-learning technologies, seeing that there are indeed great insights to be mined from data. With more data, and especially with big data, machine learning can be even more effective.

Big-Data Sources

The following articles and sites provide links to hundreds of free big data sources:

ig-data sources

"Awesome-Public-Datasets," GitHub.com,
<https://github.com/caesar0301/awesome-public-datasets>.

"AWS Public Datasets," <https://aws.amazon.com/public-datasets/>.

"Big Data And AI: 30 Amazing (And Free) Public Data Sources For 2018," by
<https://www.forbes.com/sites/bernardmarr/2018/02/26/big-data-and-amazing-and-free-public-data-sources-for-2018/>.

"Datasets for Data Mining and Data Science,"
<http://www.kdnuggets.com/datasets/index.html>.

"Exploring Open Data Sets," <https://datascience.berkeley.edu/open-datasets/>.

"Free Big Data Sources," Datamics, <http://datamics.com/free-big-data-sources/>.

Hadoop Illuminated, chapter 16. Publicly Available Big Data Sets,
http://hadoopilluminated.com/hadoop_illuminated/Public_Bigdata_Sources/.

List of Public Data Sources Fit for Machine Learning,”

<https://blog.bigml.com/list-of-public-data-sources-fit-for-machine-learning/>.

“Open Data,” Wikipedia, https://en.wikipedia.org/wiki/Open_data.

“Open Data 500 Companies,” <http://www.opendata500.com/us/list/>.

“Other Interesting Resources/Big Data and Analytics Educational Resources Research,” B. Marr, http://computing.derby.ac.uk/bigdatares/?page_id=

“6 Amazing Sources of Practice Data Sets,”

<https://www.jigsawacademy.com/6-amazing-sources-of-practice-data->

“20 Big Data Repositories You Should Check Out,” M. Krivanek,

<http://www.datasciencecentral.com/profiles/blogs/20-free-big-data-sources-everyone-should-check-out>.

“70+ Websites to Get Large Data Repositories for Free,”

<http://bigdata-madesimple.com/70-websites-to-get-large-data-repositories-for-free/>.

“Ten Sources of Free Big Data on Internet,” A. Brown,

<https://www.linkedin.com/pulse/ten-sources-free-big-data-internet-rown>.

“Top 20 Open Data Sources,”

<https://www.linkedin.com/pulse/top-20-open-data-sources-zygimantaciikevicius>.

“We’re Setting Data, Code and APIs Free,” NASA, <https://open.nasa.gov/openata/>.

“Where Can I Find Large Datasets Open to the Public?” Quora,

<https://www.quora.com/Where-can-I-find-large-datasets-open-to-the>

6.2 RELATIONAL DATABASES AND STRUCTURED QUERY LANGUAGE (SQL)

Databases are crucial, especially for big data. In [chapter 9](#), we demonstrated sequential text-file processing, working with data from CSV files and working with JSON. Both are useful when *most or all* of a file's data is to be processed. On the other hand, in transaction processing we need to locate and, possibly, update an *individual* data item quickly.

A **database** is an integrated collection of data. A **database management system (DBMS)** provides mechanisms for storing and organizing data in a manner consistent with the database's format. Database management systems allow for convenient access and storage of data without concern for the internal representation of databases.

Relational database management systems (RDBMSs) store data in **tables** and define relationships among the tables. Structured Query Language (SQL) is used almost universally with relational database systems to manipulate data and perform **queries**, which request information that satisfies given criteria. ²

² The writing in this chapter assumes that SQL is pronounced as see-quel. Some prefer ess que el.

Popular *open-source* RDBMSs include SQLite, PostgreSQL, MariaDB and MySQL. These can be downloaded and used *freely* by anyone. All have support for Python. We'll use SQLite, which is bundled with Python. Some popular proprietary RDBMSs include Microsoft SQL Server, Oracle, Sybase and IBM Db2.

Tables, Rows and Columns

A relational database is a logical table-based representation of data that allows the data to be accessed without consideration of its physical structure. The following diagram shows a sample `Employee` table that might be used in a personnel system:

	Number	Name	Department	Salary	Location
Row {	23603	Jones	413	1100	New Jersey
	24568	Kerwin	413	2000	New Jersey
	34589	Larson	642	1800	Los Angeles
	35761	Myers	611	1400	Orlando
	47132	Neumann	413	9000	New Jersey
	78321	Stephens	611	8500	Orlando
	Primary key		Column		

The table's primary purpose is to store employees' attributes. Tables are composed of **rows**, each describing a single entity. Here, each row represents one employee. Rows are composed of **columns** containing individual attribute values. The table above has six rows. The `Number` column represents the **primary key**—a column (or group of columns) with a value that's *unique* for each row. This guarantees that each row can be identified by its primary key. Examples of primary keys are Social Security numbers, employee ID numbers and part numbers in an inventory system—values in each of these are guaranteed to be unique. In this case, the rows are listed in ascending order by primary key, but they could be listed in

descending order or no particular order at all.

Each column represents a different data attribute. Rows are unique (by primary key) within a table, but particular column values may be duplicated between rows. For example, three different rows in the `Employee` table's `Department` column contain number 413.

Selecting Data Subsets

Different database users are often interested in different data and different relationships among the data. Most users require only subsets of the rows and columns. Queries specify which subsets of the data to select from a table. You use Structured Query Language (SQL) to define queries. For example, you might select data from the `Employee` table to create a result that shows where each department is located, presenting the data sorted in increasing order by department number. This result is shown below. We'll discuss SQL shortly.

[click here to view code image](#)

Department	Location
413	New Jersey
611	Orlando
642	Los Angeles

SQLite

The code examples in the rest of [Section 16.2](#) use the open-source SQLite database management system that's included with Python, but most popular database systems have Python support. Each typically provides a module that adheres to Python's **Database Application Programming Interface (DB-API)**, which specifies common object and method names for manipulating any database.

16.2.1 A `books` Database

In this section, we'll present a `books` database containing information about several of our books. We'll set up the database in SQLite via the Python Standard Library's **`sqlite3` module**, using a script provided in the `ch16` example's folder's `sql` subfolder. Then, we'll introduce the database's tables. We'll use this database in an IPython session to introduce various database concepts, including operations that **create, read, update** and **delete** data—the so-called **CRUD** operations. As we introduce the tables, we'll use SQL and pandas `DataFrames` to show you each table's contents. Then, in the next several sections, we'll discuss additional SQL features.

Creating the `books` Database

In your Anaconda Command Prompt, Terminal or shell, change to the `ch16` examples folder's `sql` subfolder. The following **`sqlite3` command** creates a SQLite database named `books.db` and executes the `books.sql` SQL script, which defines how to create the database's tables and populates them with data:

```
sqlite3 books.db < books.sql
```

The notation `<` indicates that `books.sql` is input into the `sqlite3` command. When the

command completes, the database is ready for use. Begin a new IPython session.

Connecting to the Database in Python

To work with the database in Python, first call `sqlite3`'s **connect function** to connect to the database and obtain a **Connection** object:

[click here to view code image](#)

```
In [1]: import sqlite3

In [2]: connection = sqlite3.connect('books.db')
```

authors Table

The database has three tables—`authors`, `author_isbn` and `titles`. The `authors` table stores all the authors and has three columns:

- `id`—The author's unique ID number. This integer column is defined as **autoincremented**—for each row inserted in the table, SQLite increases the `id` value by 1 to ensure that each row has a unique value. This column is the table's primary key.
- `first`—The author's first name (a string).
- `last`—The author's last name (a string).

Viewing the authors Table's Contents

Let's use a SQL query and pandas to view the `authors` table's contents:

[click here to view code image](#)

```
In [3]: import pandas as pd

In [4]: pd.options.display.max_columns = 10

In [5]: pd.read_sql('SELECT * FROM authors', connection,
...:                 index_col=['id'])
...:
Out[5]:
   first  last
id
1     Paul Deitel
2   Harvey Deitel
3    Abbey Deitel
4      Dan Quirk
5 Alexander  Wald
```

Pandas function **read_sql** executes a SQL query and returns a `DataFrame` containing the query's results. The function's arguments are:

- a string representing the SQL query to execute,
- the SQLite database's `Connection` object, and in this case

- an `index_col` keyword argument indicating which column should be used as the DataFrame's row indices (the author's `id` values in this case).

As you'll see momentarily, when `index_col` is not passed, index values starting from 0 appear to the left of the DataFrame's rows.

A SQL **SELECT** query gets rows and columns from one or more tables in a database. In the query:

```
SELECT * FROM authors
```

the **asterisk (*)** is a *wildcard* indicating that the query should get *all* the columns from the `authors` table. We'll discuss `SELECT` queries in more detail shortly.

titles Table

The `titles` table stores all the books and has four columns:

- `isbn`—The book's ISBN (a string) is this table's primary key. ISBN is an abbreviation for "International Standard Book Number," which is a numbering scheme that publishers use to give every book a unique identification number.
- `title`—The book's title (a string).
- `edition`—The book's edition number (an integer).
- `copyright`—The book's copyright year (a string).

Let's use SQL and pandas to view the `titles` table's contents:

[click here to view code image](#)

```
In [6]: pd.read_sql('SELECT * FROM titles', connection)
Out[6]:
```

	isbn	title	edition	copyright
0	0135404673	Intro to Python for CS and DS	1	2020
1	0132151006	Internet & WWW How to Program	5	2012
2	0134743350	Java How to Program	11	2018
3	0133976890	C How to Program	8	2016
4	0133406954	Visual Basic 2012 How to Program	6	2014
5	0134601548	Visual C# How to Program	6	2017
6	0136151574	Visual C++ How to Program	2	2008
7	0134448235	C++ How to Program	10	2017
8	0134444302	Android How to Program	3	2017
9	0134289366	Android 6 for Programmers	3	2016

author_ISBN Table

The `author_ISBN` table uses the following columns to associate authors from the `authors` table with their books in the `titles` table:

- `id`—An author's `id` (an integer).

- `isbn`—The book's ISBN (a string).

The `id` column is a **foreign key**, which is a column in this table that matches a *primary-key* column in another table—in particular, the `authors` table's `id` column. The `isbn` column also is a foreign key—it matches the `titles` table's `isbn` primary-key column. A database might have many tables. A goal when designing a database is to *minimize data duplication* among the tables. To do this, each table represents a specific entity, and foreign keys help link the data in *multiple* tables. The primary keys and foreign keys are designated when you create the database tables (in our case, in the `books.sql` script).

Together the `id` and `isbn` columns in this table form a *composite primary key*. Every row in this table *uniquely* matches *one* author to *one* book's ISBN. This table contains many entries, so let's use SQL and pandas to view just the first five rows:

[lick here to view code image](#)

```
In [7]: df = pd.read_sql('SELECT * FROM author_ISBN', connection)

In [8]: df.head()
Out[8]:
```

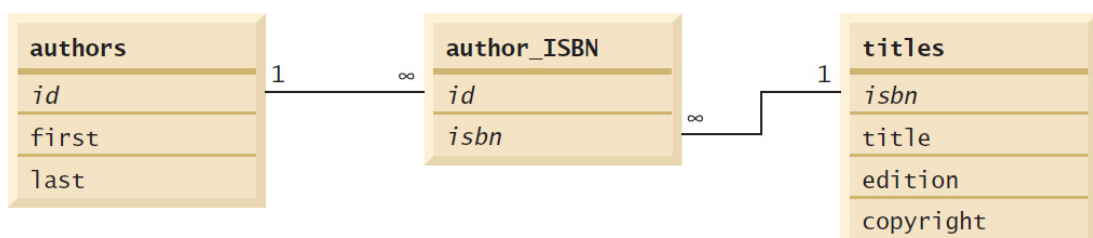
	id	isbn
0	1	0134289366
1	2	0134289366
2	5	0134289366
3	1	0135404673
4	2	0135404673

Every foreign-key value must appear as the primary-key value in a row of another table so the DBMS can ensure that the foreign-key value is valid. This is known as the **Rule of Referential Integrity**. For example, the DBMS ensures that the `id` value for a particular `author_ISBN` row is valid by checking that there is a row in the `authors` table with that `id` as the primary key.

Foreign keys also allow *related* data in *multiple* tables to be *selected* from those tables and combined—this is known as **joining** the data. There is a **one-to-many relationship** between a primary key and a corresponding foreign key—one author can write many books, and similarly one book can be written by many authors. So a foreign key can appear *many* times in its table but only *once* (as the primary key) in another table. For example, in the `books` database, the ISBN 0134289366 appears in several `author_ISBN` rows because this book has several authors, but it appears only once as a primary key in `titles`.

Entity-Relationship (ER) Diagram

The following **entity-relationship- (ER) diagram** for the `books` database shows the database's *tables* and the *relationships* among them:



he first compartment in each box contains the table’s name, and the remaining compartments contain the table’s columns. The names in italic are primary keys. *A table’s primary key uniquely identifies each row in the table.* Every row must have a primary-key value, and that value must be unique in the table. This is known as the **Rule of Entity Integrity**. Again, for the `author_ISBN` table, the primary key is the combination of both columns—this is known as a composite primary key.

The lines connecting the tables represent the *relationships* among the tables. Consider the line between `authors` and `author_ISBN`. On the `authors` end there’s a 1, and on the `author_ISBN` end there’s an infinity symbol (∞). This indicates a *one-to-many relationship*. For *each* author in the `authors` table, there can be an *arbitrary number* of ISBNs for books written by that author in the `author_ISBN` table—that is, an author can write *any* number of books, so an author’s `id` can appear in multiple rows of the `author_ISBN` table. The relationship line links the `id` column in the `authors` table (where `id` is the primary key) to the `id` column in the `author_ISBN` table (where `id` is a foreign key). The line between the tables links the primary key to the matching foreign key.

The line between the `titles` and `author_ISBN` tables illustrates a *one-to-many relationship*—one book can be written by many authors. The line links the primary key `isbn` in table `titles` to the corresponding foreign key in table `author_ISBN`. The relationships in the entity-relationship diagram illustrate that the sole purpose of the `author_ISBN` table is to provide a **many-to-many relationship** between the `authors` and `titles` tables—an author can write *many* books, and a book can have *many* authors.

SQL Keywords

The following subsections continue our SQL presentation in the context of our `books` database, demonstrating SQL queries and statements using the SQL keywords in the following table. Other SQL keywords are beyond this text’s scope:

SQL keyword	Description
SELECT	Retrieves data from one or more tables.
FROM	Tables involved in the query. Required in every SELECT.
WHERE	Criteria for selection that determine the rows to be retrieved, deleted or updated. Optional in a SQL statement.
GROUP BY	Criteria for grouping rows. Optional in a SELECT query.
ORDER	

BY	Criteria for ordering rows. Optional in a SELECT query.
INNER JOIN	Merge rows from multiple tables.
INSERT	Insert rows into a specified table.
UPDATE	Update rows in a specified table.
DELETE	Delete rows from a specified table.

16.2.2 SELECT Queries

The previous section used SELECT statements and the * wildcard character to get all the columns from a table. Typically, you need only a subset of the columns, especially in big data where you could have dozens, hundreds, thousands or more columns. To retrieve only specific columns, specify a comma-separated list of column names. For example, let's retrieve only the columns `first` and `last` from the `authors` table:

[lick here to view code image](#)

```
In [9]: pd.read_sql('SELECT first, last FROM authors', connection)
Out[9]:
```

	first	last
0	Paul	Deitel
1	Harvey	Deitel
2	Abbey	Deitel
3	Dan	Quirk
4	Alexander	Wald

16.2.3 WHERE Clause

You'll often select rows in a database that satisfy certain **selection criteria**, especially in big data where a database might contain millions or billions of rows. Only rows that satisfy the selection criteria (formally called **predicates**) are selected. SQL's **WHERE clause** specifies a query's selection criteria. Let's select the `title`, `edition` and `copyright` for all books with copyright years greater than 2016. String values in SQL queries are delimited by single (') quotes, as in '2016':

[lick here to view code image](#)

```
In [10]: pd.read_sql("""SELECT title, edition, copyright
...:                  FROM titles
...:                  WHERE copyright > '2016'""", connection)
Out[10]:
```

	title	edition	copyright
0	Intro to Python for CS and DS	1	2020
1	Java How to Program	11	2018
2	Visual C# How to Program	6	2017
3	C++ How to Program	10	2017
4	Android How to Program	3	2017

Pattern Matching: Zero or More Characters

The `WHERE` clause may contain the operators `<`, `>`, `<=`, `>=`, `=`, `<>` (not equal) and `LIKE`. Operator **LIKE** is used for **pattern matching**—searching for strings that match a given pattern. A pattern that contains the **percent (%)** wildcard character searches for strings that have *zero or more* characters at the percent character's position in the pattern. For example, let's locate all authors whose last name starts with the letter D:

[lick here to view code image](#)

```
In [11]: pd.read_sql("""SELECT id, first, last
...:                FROM authors
...:                WHERE last LIKE 'D%',
...:                connection, index_col=['id'])
...:
Out[11]:
   first  last
id
1   Paul  Deitel
2  Harvey  Deitel
3   Abbey  Deitel
```

Pattern Matching: Any Character

An **underscore (_)** in the pattern string indicates a single wildcard character at that position. Let's select the rows of all the authors whose last names start with any character, followed by the letter b, followed by any number of additional characters (specified by %):

[lick here to view code image](#)

```
In [12]: pd.read_sql("""SELECT id, first, last
...:                FROM authors
...:                WHERE first LIKE '_b%',
...:                connection, index_col=['id'])
...:
Out[12]:
   first  last
id
3   Abbey  Deitel
```

16.2.4 ORDER BY Clause

The **ORDER BY clause** sorts a query's results into ascending order (lowest to highest) or descending order (highest to lowest), specified with `ASC` and `DESC`, respectively. The default sorting order is ascending, so `ASC` is optional. Let's sort the titles in ascending order:

[lick here to view code image](#)

```
In [13]: pd.read_sql('SELECT title FROM titles ORDER BY title ASC',
```



```

...:         connection)
Out[13]:
           title
0      Android 6 for Programmers
1      Android How to Program
2          C How to Program
3      C++ How to Program
4  Internet & WWW How to Program
5  Intro to Python for CS and DS
6          Java How to Program
7  Visual Basic 2012 How to Program
8      Visual C# How to Program
9      Visual C++ How to Program

```

Sorting By Multiple Columns

To sort by multiple columns, specify a comma-separated list of column names after the `ORDER BY` keywords. Let's sort the authors' names by last name, then by first name for any authors who have the same last name:

[lick here to view code image](#)

```

In [14]: pd.read_sql("""SELECT  id, first, last
...:                  FROM authors
...:                  ORDER BY  last, first""",
...:                  connection, index_col=['id'])
...:
Out[14]:
           first  last
id
3      Abbey  Deitel
2      Harvey  Deitel
1         Paul  Deitel
4         Dan   Quirk
5  Alexander   Wald

```

The sorting order can vary by column. Let's sort the authors in descending order by last name and ascending order by first name for any authors who have the same last name:

[lick here to view code image](#)

```

In [15]: pd.read_sql("""SELECT  id, first, last
...:                  FROM authors
...:                  ORDER BY last  DESC, first ASC""",
...:                  connection, index_col=['id'])
...:
Out[15]:
           first  last
id
5  Alexander   Wald
4         Dan   Quirk
3      Abbey  Deitel
2      Harvey  Deitel
1         Paul  Deitel

```

Combining the WHERE and ORDER BY Clauses

The `WHERE` and `ORDER BY` clauses can be combined in one query. Let's get the `isbn`, `title`, `edition` and `copyright` of each book in the `titles` table that has a title ending with

'How to Program' and sort them in ascending order by title.

[lick here to view code image](#)

```
In [16]: pd.read_sql("""SELECT    isbn, title, edition, copyright
...:                        FROM titles
...:                        WHERE title    LIKE '%How to Program'
...:                        ORDER BY    title""", connection)
Out[16]:
```

	isbn	title	edition	copyright
0	0134444302	Android How to Program	3	2017
1	0133976890	C How to Program	8	2016
2	0134448235	C++ How to Program	10	2017
3	0132151006	Internet & WWW How to Program	5	2012
4	0134743350	Java How to Program	11	2018
5	0133406954	Visual Basic 2012 How to Program	6	2014
6	0134601548	Visual C# How to Program	6	2017
7	0136151574	Visual C++ How to Program	2	2008

16.2.5 Merging Data from Multiple Tables: INNER JOIN

Recall that the `books` database's `author_ISBN` table links authors to their corresponding titles. If we did not separate this information into individual tables, we'd need to include author information with each entry in the `titles` table. This would result in storing *duplicate* author information for authors who wrote multiple books.

You can merge data from multiple tables, referred to as joining the tables, with **INNER JOIN**. Let's produce a list of authors accompanied by the ISBNs for books written by each author—because there are many results for this query, we show just the head of the result:

[lick here to view code image](#)

```
In [17]: pd.read_sql("""SELECT    first, last, isbn
...:                        FROM authors
...:                        INNER JOIN    author_ISBN
...:                        ON    authors.id = author_ISBN.id
...:                        ORDER BY    last, first""", connection).head()
Out[17]:
```

	first	last	isbn
0	Abbey	Deitel	0132151006
1	Abbey	Deitel	0133406954
2	Harvey	Deitel	0134289366
3	Harvey	Deitel	0135404673
4	Harvey	Deitel	0132151006

The **INNER JOIN**'s **ON clause** uses a primary-key column in one table and a foreign-key column in the other to determine which rows to merge from each table. This query merges the `authors` table's `first` and `last` columns with the `author_ISBN` table's `isbn` column and sorts the results in ascending order by `last` then `first`.

Note the syntax `authors.id (table_name.column_name)` in the **ON** clause. This **qualified name** syntax is required if the columns have the same name in both tables. This syntax can be used in any SQL statement to distinguish columns in different tables that have the same name. In some systems, table names qualified with the database name can be used to perform cross-database queries. As always, the query can contain an **ORDER BY** clause.

16.2.6 INSERT INTO Statement

To this point, you’ve queried existing data. Sometimes you’ll execute SQL statements that *modify* the database. To do so, you’ll use a `sqlite3 Cursor` object, which you obtain by calling the `Connection`’s `cursor` method:

[lick here to view code image](#)

```
In [18]: cursor = connection.cursor()
```

The pandas method `read_sql` actually uses a `Cursor` behind the scenes to execute queries and access the rows of the results.

The **INSERT INTO** statement inserts a row into a table. Let’s insert a new author named Sue Red into the `authors` table by calling `Cursor` method `execute`, which executes its SQL argument and returns the `Cursor`:

[lick here to view code image](#)

```
In [19]: cursor = cursor.execute("""INSERT INTO authors (first, last)
...:                               VALUES ('Sue', 'Red')""")
...:
```

The SQL keywords `INSERT INTO` are followed by the table in which to insert the new row and a comma-separated list of column names in parentheses. The list of column names is followed by the SQL keyword **VALUES** and a comma-separated list of values in parentheses. The values provided must match the column names specified both in order and type.

We do not specify a value for the `id` column because it’s an autoincremented column in the `authors` table—this was specified in the script `books.sql` that created the table. For every new row, SQLite assigns a unique `id` value that is the next value in the autoincremented sequence (i.e., 1, 2, 3 and so on). In this case, Sue Red is assigned `id` number 6. To confirm this, let’s query the `authors` table’s contents:

[lick here to view code image](#)

```
In [20]: pd.read_sql('SELECT id, first, last FROM authors',
...:                  connection, index_col=['id'])
...:
Out[20]:
   id  first last
1    1   Paul Deitel
2    2  Harvey Deitel
3    3   Abbey Deitel
4    4    Dan Quirk
5    5 Alexander Wald
6    6    Sue Red
```

Note Regarding Strings That Contain Single Quotes

SQL delimits strings with single quotes (`'`). A string containing a single quote, such as O’Malley, must have *two* single quotes in the position where the single quote appears (e.g.,

'O' 'Malley')). The first acts as an escape character for the second. Not escaping single-quote characters in a string that's part of a SQL statement is a SQL syntax error.

16.2.7 UPDATE Statement

An **UPDATE** statement modifies existing values. Let's assume that Sue Red's last name is incorrect in the database and update it to 'Black':

[lick here to view code image](#)

```
In [21]: cursor = cursor.execute("""UPDATE    authors SET last='Black'
...:                                     WHERE    last='Red' AND first='Sue'""")
```

The **UPDATE** keyword is followed by the table to update, the keyword **SET** and a comma-separated list of *column_name = value* pairs indicating the columns to change and their new values. The change will be applied to *every* row if you do not specify a **WHERE** clause. The **WHERE** clause in this query indicates that we should update only rows in which the last name is 'Red' and the first name is 'Sue'.

Of course, there could be multiple people with the same first and last name. To make a change to only one row, it's best to use the row's unique primary key in the **WHERE** clause. In this case, we could have specified:

```
WHERE id = 6
```

For statements that modify the database, the **Cursor** object's **rowcount** attribute contains an integer value representing the number of rows that were modified. If this value is 0, no changes were made. The following confirms that the **UPDATE** modified one row:

[lick here to view code image](#)

```
In [22]: cursor.rowcount
Out[22]: 1
```

We also can confirm the update by listing the **authors** table's contents:

[lick here to view code image](#)

```
In [23]: pd.read_sql('SELECT id,    first, last FROM authors',
...:                  connection, index_col=['id'])
...:
Out[23]:
   id  first  last
1    Paul  Deitel
2  Harvey  Deitel
3   Abbey  Deitel
4     Dan   Quirk
5 Alexander   Wald
6     Sue   Black
```

16.2.8 DELETE FROM Statement

A SQL **DELETE FROM** statement removes rows from a table. Let's remove Sue Black from the `authors` table using her author ID:

[click here to view code image](#)

```
In [24]: cursor = cursor.execute('DELETE FROM authors WHERE id=6')

In [25]: cursor.rowcount
Out[25]: 1
```

The optional `WHERE` clause determines which rows to delete. If `WHERE` is omitted, all the table's rows are deleted. Here's the `authors` table after the `DELETE` operation:

[click here to view code image](#)

```
In [26]: pd.read_sql('SELECT id, first, last FROM authors',
...:                  connection, index_col=['id'])
...:
Out[26]:
```

	id	first	last
1	1	Paul	Deitel
2	2	Harvey	Deitel
3	3	Abbey	Deitel
4	4	Dan	Quirk
5	5	Alexander	Wald

Closing the Database

When you no longer need access to the database, you should call the `Connection`'s `close` method to disconnect from the database:

```
connection.close()
```

SQL in Big Data

SQL's importance is growing in big data. Later in this chapter, we'll use Spark SQL to query data in a Spark `DataFrame` for which the data may be distributed over many computers in a Spark cluster. As you'll see, Spark SQL looks much like the SQL presented in this section.

16.3 NOSQL AND NEWSQL BIG-DATA DATABASES: A BRIEF TOUR

For decades, relational database management systems have been the standard in data processing. However, they require *structured data* that fits into neat rectangular tables. As the size of the data and the number of tables and relationships increases, relational databases become more difficult to manipulate efficiently. In today's big-data world, NoSQL and NewSQL databases have emerged to deal with the kinds of data storage and processing demands that traditional relational databases cannot meet. Big data requires massive databases, often spread across data centers worldwide in huge clusters of commodity computers. According to `statista.com`, there are currently over 8 million data centers worldwide. ³

³ <https://www.statista.com/statistics/500458/worldwide-datacenter-and-it-sites/>.

oSQL originally meant what its name implies. With the growing importance of SQL in big data—such as SQL on Hadoop and Spark SQL—NoSQL now is said to stand for “Not Only SQL.” NoSQL databases are meant for unstructured data, like photos, videos and the natural language found in e-mails, text messages and social-media posts, and semi-structured data like JSON and XML documents. Semi-structured data often wraps unstructured data with additional information called **metadata**. For example, YouTube videos are unstructured data, but YouTube also maintains metadata for each video, including who posted it, when it was posted, a title, a description, tags that help people discover the videos, privacy settings and more—all returned as JSON from the YouTube APIs. This metadata adds structure to the unstructured video data, making it semi-structured.

The next several subsections overview the four NoSQL database categories—key–value, document, columnar (also called column-based) and graph. In addition, we’ll overview NewSQL databases, which blend features of relational and NoSQL databases. In [Section 16.4](#), we’ll present a case study in which we store and manipulate a large number of JSON tweet objects in a NoSQL document database, then summarize the data in an interactive visualization displayed on a Folium map of the United States.

16.3.1 NoSQL Key–Value Databases

Like Python dictionaries, **key–value databases** ⁴ store key–value pairs, but they’re optimized for distributed systems and big-data processing. For reliability, they tend to replicate data in multiple cluster nodes. Some key–value databases, such as Redis, are implemented in memory for performance, and others store data on disk, such as HBase, which runs on top of Hadoop’s HDFS distributed file system. Other popular key–value databases include Amazon DynamoDB, Google Cloud Datastore and Couchbase. DynamoDB and Couchbase are **multi-model databases** that also support documents. HBase is also a column-oriented database.

⁴ https://en.wikipedia.org/wiki/Key-value_database.

16.3.2 NoSQL Document Databases

A **document database** ⁵ stores semi-structured data, such as JSON or XML documents. In document databases, you typically add indexes for specific attributes, so you can more efficiently locate and manipulate documents. For example, let’s assume you’re storing JSON documents produced by IoT devices and each document contains a type attribute. You might add an index for this attribute so you can filter documents based on their types. Without indexes, you can still perform that task, it will just be slower because you have to search each document in its entirety to find the attribute.

⁵ https://en.wikipedia.org/wiki/Document-oriented_database.

The most popular document database (and most popular overall NoSQL database ⁶) is **MongoDB**, whose name derives from a sequence of letters embedded in the word “humongous.” In an example, we’ll store a large number of tweets in MongoDB for processing. Recall that Twitter’s APIs return tweets in JSON format, so they can be stored directly in MongoDB. After obtaining the tweets we’ll summarize them in a pandas `Data-`

Frame and on a Folium map. Other popular document databases include Amazon DynamoDB (also a key–value database), Microsoft Azure Cosmos DB and Apache CouchDB.

⁶ <https://db-engines.com/en/ranking>.

16.3.3 NoSQL Columnar Databases

In a relational database, a common query operation is to get a specific column’s value for every row. Because data is organized into rows, a query that selects a specific column can perform poorly. The database system must get every matching row, locate the required column and discard the rest of the row’s information. A **columnar database** ^{7, 8}, also called a **column-oriented database**, is similar to a relational database, but it stores structured data in columns rather than rows. Because all of a column’s elements are stored together, selecting all the data for a given column is more efficient.

⁷ https://en.wikipedia.org/wiki/Columnar_database.

⁸ <https://www.predictiveanalyticstoday.com/top-wide-columnar-store-databases/>.

Consider our `authors` table in the `books` database:

[click here to view code image](#)

	first	last
id		
1	Paul	Deitel
2	Harvey	Deitel
3	Abbey	Deitel
4	Dan	Quirk
5	Alexander	Wald

In a relational database, all the data for a row is stored together. If we consider each row as a Python tuple, the rows would be represented as `(1, 'Paul', 'Deitel')`, `(2, 'Harvey', 'Deitel')`, etc. In a columnar database, all the values for a given column would be stored together, as in `(1, 2, 3, 4, 5)`, `('Paul', 'Harvey', 'Abbey', 'Dan', 'Alexander')` and `('Deitel', 'Deitel', 'Deitel', 'Quirk', 'Wald')`. The elements in each column are maintained in row order, so the value at a given index in each column belongs to the same row. Popular columnar databases include MariaDB ColumnStore and HBase.

16.3.4 NoSQL Graph Databases

A graph models relationships between objects. ⁹ The objects are called **nodes** (or **vertices**) and the relationships are called **edges**. Edges are *directional*. For example, an edge representing an airline flight points from the origin city to the destination city, but not the reverse. A **graph database** ⁰ stores nodes, edges and their attributes.

⁹ https://en.wikipedia.org/wiki/Graph_theory.

⁰ https://en.wikipedia.org/wiki/Graph_database.

If you use social networks, like Instagram, Snapchat, Twitter and Facebook, consider your

social graph, which consists of the people you know (nodes) and the relationships between them (edges). Every person has their own social graph, and these are interconnected. The famous “six degrees of separation” problem says that any two people in the world are connected to one another by following a maximum of six edges in the worldwide social graph.¹ Facebook’s algorithms use the social graphs of their billions of monthly active users² to determine which stories should appear in each user’s news feed. By looking at your interests, your friends, their interests and more, Facebook predicts the stories they believe are most relevant to you.³

¹ https://en.wikipedia.org/wiki/Six_degrees_of_separation.

² <https://zephoria.com/top-15-valuable-facebook-statistics/>.

³ <https://newsroom.fb.com/news/2018/05/inside-feed-news-feed-ranking/>.

Many companies use similar techniques to create recommendation engines. When you browse a product on Amazon, they use a graph of users and products to show you comparable products people browsed before making a purchase. When you browse movies on Netflix, they use a graph of users and movies they liked to suggest movies that might be of interest to you.

One of the most popular graph databases is Neo4j. Many real-world use-cases for graph databases are provided at:

<https://neo4j.com/graphgists/>

With most of the use-cases, sample graph diagrams produced by Neo4j are shown. These visualize the relationships between the graph nodes. Check out Neo4j’s free PDF book, *Graph Databases*.⁴

⁴ <https://neo4j.com/graph-databases-book-sx2>.

16.3.5 NewSQL Databases

Key advantages of relational databases include their security and transaction support. In particular, relational databases typically use **ACID (Atomicity, Consistency, Isolation, Durability)**⁵ transactions:

⁵ [https://en.wikipedia.org/wiki/ACID_\(computer_science\)](https://en.wikipedia.org/wiki/ACID_(computer_science)).

- *Atomicity* ensures that the database is modified only if *all* of a transaction’s steps are successful. If you go to an ATM to withdraw \$100, that money is not removed from your account unless you have enough money to cover the withdrawal *and* there is enough money in the ATM to satisfy your request.
- *Consistency* ensures that the database state is always valid. In the withdrawal example above, your new account balance after the transaction will reflect precisely what you withdrew from your account (and possibly ATM fees).
- *Isolation* ensures that concurrent transactions occur as if they were performed sequentially. For example, if two people share a joint bank account and both attempt to

withdraw money at the same time from two separate ATMs, one transaction must wait until the other completes.

- *Durability* ensures that changes to the database survive even hardware failures.

If you research benefits and disadvantages of NoSQL databases, you'll see that NoSQL databases generally do not provide ACID support. The types of applications that use NoSQL databases typically do not require the guarantees that ACID-compliant databases provide. Many NoSQL databases typically adhere to the **BASE (Basic Availability, Soft-state, Eventual consistency)** model, which focuses more on the database's availability. Whereas, ACID databases guarantee consistency when you write to the database, BASE databases provide consistency at some later point in time.

NewSQL databases blend the benefits of both relational and NoSQL databases for big-data processing tasks. Some popular NewSQL databases include VoltDB, MemSQL, Apache Ignite and Google Spanner.

16.4 CASE STUDY: A MONGODB JSON DOCUMENT DATABASE

MongoDB is a document database capable of storing and retrieving JSON documents. Twitter's APIs return tweets to you as JSON objects, which you can write directly into a MongoDB database. In this section, you'll:

- use Tweepy to stream tweets about the 100 U.S. senators and store them into a MongoDB database,
- use pandas to summarize the top 10 senators by tweet activity and
- display an interactive Folium map of the United States with one popup marker per state that shows the state name and both senators' names, their political parties and tweet counts.

You'll use a free cloud-based MongoDB Atlas cluster, which requires no installation and currently allows you to store up to 512MB of data. To store more, you can download the MongoDB Community Server from:

[click here to view code image](#)

```
https://www.mongodb.com/download-center/community
```

and run it locally or you can sign up for MongoDB's *paid* Atlas service.

Installing the Python Libraries Required for Interacting with MongoDB

You'll use the **pymongo library** to interact with MongoDB databases from your Python code. You'll also need the `dnspython` library to connect to a MongoDB Atlas Cluster. To install these libraries, use the following commands:

[click here to view code image](#)

```
conda install -c conda-forge pymongo
conda install -c conda-forge dnspython
```

`keys.py`

The `ch16` examples folder's `TwitterMongoDB` subfolder contains this example's code and `keys.py` file. Edit this file to include your Twitter credentials and your OpenMapQuest key from the “Data Mining Twitter” chapter. After we discuss creating a MongoDB Atlas cluster, you'll also need to add your MongoDB connection string to this file.

16.4.1 Creating the MongoDB Atlas Cluster

To sign up for a free account go to

```
https://mongodb.com
```

then enter your email address and click **Get started free**. On the next page, enter your name and create a password, then read their terms of service. If you agree, click **Get started free** on this page and you'll be taken to the screen for setting up your cluster. Click **Build my first cluster** to get started.

They walk you through the getting started steps with popup bubbles that describe and point you to each task you need to complete. They provide default settings for their free Atlas cluster (**Mo** as they refer to it), so just give your cluster a name in the **Cluster Name** section, then click **Create Cluster**. At this point, they'll take you to the **Clusters** page and begin creating your new cluster, which takes several minutes.

Next, a **Connect to Atlas** popup tutorial will appear, showing a checklist of additional steps required to get you up and running:

- **Create your first database user**—This enables you to log into your cluster.
- **Whitelist your IP address**—This is a security measure which ensures that only IP addresses you verify are allowed to interact with your cluster. To connect to this cluster from multiple locations (school, home, work, etc.), you'll need to whitelist each IP address from which you intend to connect.
- **Connect to your cluster**—In this step, you'll locate your cluster's connection string, which will enable your Python code to connect to the server.

Creating Your First Database User

In the popup tutorial window, click **Create your first database user** to continue the tutorial, then follow the on-screen prompts to view the cluster's **Security** tab and click + **ADD NEW USER**. In the **Add New User** dialog, create a username and password. Write these down—you'll need them momentarily. Click **Add User** to return to the **Connect to Atlas** popup tutorial.

Whitelist Your IP Address

In the popup tutorial window, click **Whitelist your IP address** to continue the tutorial, then follow the on-screen prompts to view the cluster's **IP Whitelist** and click + **ADD IP**

ADDRESS. In the **Add Whitelist Entry** dialog, you can either add your computer's current IP address or allow access from anywhere, which they do not recommend for production databases, but is OK for learning purposes. Click **ALLOW ACCESS FROM ANYWHERE** then click **Confirm** to return to the **Connect to Atlas** popup tutorial.

Connect to Your Cluster

In the popup tutorial window, click **Connect to your cluster** to continue the tutorial, then follow the on-screen prompts to view the cluster's **Connect to *YourClusterName*** dialog. Connecting to a MongoDB Atlas database from Python requires a connection string. To get your connection string, click **Connect Your Application**, then click **Short SRV connection string**. Your connection string will appear below **Copy the SRV address**. Click **COPY** to copy the string. Paste this string into the `keys.py` file as `mongo_connection_string`'s value. Replace "<PASSWORD>" in the connection string with your password, and replace the database name "test" with "senators", which will be the database name in this example. At the bottom of the **Connect to *YourClusterName***, click **Close**. You're now ready to interact with your Atlas cluster.

16.4.2 Streaming Tweets into MongoDB

First we'll present an interactive IPython session that connects to the MongoDB database, downloads current tweets via Twitter streaming and summarizes the top-10 senators by tweet count. Next, we'll present class `TweetListener`, which handles the incoming tweets and stores their JSON in MongoDB. Finally, we'll continue the IPython session by creating an interactive Folium map that displays information from the tweets we stored.

Use Tweepy to Authenticate with Twitter

First, let's use Tweepy to authenticate with Twitter:

[lick here to view code image](#)

```
In [1]: import tweepy, keys

In [2]: auth = tweepy.OAuthHandler(
...:     keys.consumer_key, keys.consumer_secret)
...: auth.set_access_token(keys.access_token,
...:     keys.access_token_secret)
...:
```

Next, configure the Tweepy API object to wait if our app reaches any Twitter rate limits.

[lick here to view code image](#)

```
In [3]: api = tweepy.API(auth, wait_on_rate_limit=True,
...:     wait_on_rate_limit_notify=True)
...:
```

Loading the Senators' Data

We'll use the information in the file `senators.csv` (located in the `ch16 examples` folder's `TwitterMongoDB` subfolder) to track tweets to, from and about every U.S. senator. The file contains the senator's two-letter state code, name, party, Twitter handle and Twitter ID.

Twitter enables you to follow specific users via their numeric Twitter IDs, but these must be submitted as string representations of those numeric values. So, let's load `senators.csv` into pandas, convert the `TwitterID` values to strings (using `Series` method `astype`) and display several rows of data. In this case, we set 6 as the maximum number of columns to display. Later we'll add another column to the `DataFrame` and this setting will ensure that all the columns are displayed, rather than a few with in between:

[lick here to view code image](#)

```
In [4]: import pandas as pd

In [5]: senators_df = pd.read_csv('senators.csv')

In [6]: senators_df['TwitterID'] = senators_df['TwitterID'].astype(str)

In [7]: pd.options.display.max_columns = 6

In [8]: senators_df.head()
Out[8]:
```

	State	Name	Party	TwitterHandle	TwitterID
0	AL	Richard Shelby	R	SenShelby	21111098
1	AL	Doug Jones	D	SenDougJones	941080085121175552
2	AK	Lisa Murkowski	R	lisamurkowski	18061669
3	AK	Dan Sullivan	R	SenDanSullivan	2891210047
4	AZ	Jon Kyl	R	SenJonKyl	24905240

Configuring the MongoClient

To store the tweet's JSON as documents in a MongoDB database, you must first connect to your MongoDB Atlas cluster via a pymongo `MongoClient`, which receives your cluster's connection string as its argument:

[lick here to view code image](#)

```
In [9]: from pymongo import MongoClient

In [10]: atlas_client = MongoClient(keys.mongo_connection_string)
```

Now, we can get a pymongo `Database` object representing the `senators` database. The following statement creates the database if it does not exist:

[lick here to view code image](#)

```
In [11]: db = atlas_client.senators
```

Setting up Tweet Stream

Let's specify the number of tweets to download and create the `TweetListener`. We pass the `db` object representing the MongoDB database to the `TweetListener` so it can write the tweets into the database. Depending on the rate at which people are tweeting about the senators, it may take minutes to hours to get 10,000 tweets. For testing purposes, you might want to use a smaller number:

[lick here to view code image](#)

```

In [12]: from tweetlistener import TweetListener

In [13]: tweet_limit = 10000

In [14]: twitter_stream = tweepy.Stream(api.auth,
...:      TweetListener(api, db, tweet_limit))
...:

```

Starting the Tweet Stream

Twitter live streaming allows you to track up to 400 keywords and follow up to 5,000 Twitter IDs at a time. In this case, let's track the senators' Twitter handles and follow the senator's Twitter IDs. This should give us tweets from, to and about each senator. To show you progress, we display the screen name and time stamp for each tweet received, and the total number of tweets so far. To save space, we show here only one of those tweet outputs and replace the user's screen name with XXXXXXXX:

[lick here to view code image](#)

```

In [15]: twitter_stream.filter(track=senators_df.TwitterHandle.tolist(),
...:      follow=senators_df.TwitterID.tolist())
...:
Screen name: XXXXXXXX
Created at: Sun Dec 16 17:19:19 +0000 2018
Tweets received: 1
...

```

Class TweetListener

For this example, we slightly modified class TweetListener from the “Data Mining Twitter” chapter. Much of the Twitter and Tweepy code shown below is identical to the code you saw previously, so we'll focus on only the new concepts here:

[lick here to view code image](#)

```

1 # tweetlistener.py
2 """TweetListener downloads tweets and stores them in MongoDB."""
3 import json
4 import tweepy
5
6 class TweetListener(tweepy.StreamListener):
7     """Handles incoming Tweet stream."""
8
9     def __init__(self, api, database, limit=10000):
10         """Create instance variables for tracking number of tweets."""
11         self.db = database
12         self.tweet_count = 0
13         self.TWEET_LIMIT = limit # 10,000 by default
14         super().__init__(api) # call superclass's init
15
16     def on_connect(self):
17         """Called when your connection attempt is successful, enabling
18         you to perform appropriate application tasks at that point."""
19         print('Successfully connected to Twitter\n')
20
21     def on_data(self, data):
22         """Called when Twitter pushes a new tweet to you."""
23         self.tweet_count += 1 # track number of tweets processed
24         json_data = json.loads(data) # convert string to JSON

```

```

25         self.db.tweets.insert_one(json_data)      # store in tweets collection
26         print(f'      Screen name: {json_data["user"]["name"]}')
27         print(f'      Created at: {json_data["created_at"]}')
28         print(f'Tweets received: {self.tweet_count}')
29
30         # if TWEET_LIMIT is reached, return False to terminate streaming
31         return self.tweet_count != self.TWEET_LIMIT
32
33     def on_error(self, status):
34         print(status)
35         return True

```

Previously, `TweetListener` overrode method `on_status` to receive Tweepy `Status` objects representing tweets. Here, we override the `on_data` method instead (lines 21–31). Rather than `Status` objects, `on_data` receives each tweet object's *raw JSON*. Line 24 converts the JSON string received by `on_data` into a Python JSON object. Each MongoDB database contains one or more **Collections** of documents. In line 25, the expression

```
self.db.tweets
```

accesses the Database object `db`'s `tweets` Collection, creating it if it does not already exist. Line 25 uses the `tweets` Collection's **`insert_one` method** to store the JSON object in the `tweets` collection.

Counting Tweets for Each Senator

Next, we'll perform a full-text search on the collection of tweets and count the number of tweets containing each senator's Twitter handle. To text search in MongoDB, you must create a *text index* for the collection.⁶ This specifies which document field(s) to search. Each text index is defined as a tuple containing the field name to search and the index type (`'text'`). MongoDB's wildcard specifier (`$**`) indicates that every text field in a document (a JSON tweet object in our case) should be indexed for a full-text search:

⁶For additional details on MongoDB index types, text indexes and operators, see:

<https://docs.mongodb.com/manual/indexes>,

<https://docs.mongodb.com/manual/core/index-text> and

<https://docs.mongodb.com/manual/reference/operator>.

[click here to view code image](#)

```

In [16]: db.tweets.create_index([('**', 'text')])
Out[16]: '**_text'

```

Once the index is defined, we can use the Collection's **`count_documents` method** to count the total number of documents in the collection that contain the specified text. Let's search the database's `tweets` collection for every twitter handle in the `senators_df` DataFrame's `TwitterHandle` column:

[click here to view code image](#)

```
In [17]: tweet_counts = []
```

```
In [18]: for senator in senators_df.TwitterHandle:
...:     tweet_counts.append(db.tweets.count_documents(
...:         {"$text": {"$search": senator}}))
...:
```

The JSON object passed to `count_documents` in this case indicates that we’re using the index named `text` to search for the value of `senator`.

Show Tweet Counts for Each Senator

Let’s create a copy of the DataFrame `senators_df` that contains the `tweet_counts` as a new column, then display the top-10 senators by tweet count:

[lick here to view code image](#)

```
In [19]: tweet_counts_df = senators_df.assign(Tweets=tweet_counts)

In [20]: tweet_counts_df.sort_values(by='Tweets',
...:     ascending=False).head(10)
...:
```

Out[20]:

	State	Name	Party	TwitterHandle	TwitterID	Tweets
78	SC	Lindsey Graham	R	LindseyGrahamSC	432895323	1405
41	MA	Elizabeth Warren	D	SenWarren	970207298	1249
8	CA	Dianne Feinstein	D	SenFeinstein	476256944	1079
20	HI	Brian Schatz	D	brianschatz	47747074	934
62	NY	Chuck Schumer	D	SenSchumer	17494010	811
24	IL	Tammy Duckworth	D	SenDuckworth	1058520120	656
13	CT	Richard Blumenthal	D	SenBlumenthal	278124059	646
21	HI	Mazie Hirono	D	maziehirono	92186819	628
86	UT	Orrin Hatch	R	SenOrrinHatch	262756641	506
77	RI	Sheldon Whitehouse	D	SenWhitehouse	242555999	350

Get the State Locations for Plotting Markers

Next, we’ll use the techniques you learned in the “Data Mining Twitter” chapter to get each state’s latitude and longitude coordinates. We’ll soon use these to place on a Folium map popup markers that contain the names and numbers of tweets mentioning each state’s senators.

The file `state_codes.py` contains a `state_codes` dictionary that maps two-letter state codes to their full state names. We’ll use the full state names with `geopy`’s `OpenMapQuest` `geocode` function to look up the location of each state.⁷ First, let’s import the libraries we need and the `state_codes` dictionary:

⁷We use full state names because, during our testing, the two-letter state codes did not always return correct locations.

[lick here to view code image](#)

```
In [21]: from geopy import OpenMapQuest

In [22]: import time

In [23]: from state_codes import state_codes
```

Next, let's get the `geocoder` object to translate location names into `Location` objects:

[lick here to view code image](#)

```
In [24]: geo = OpenMapQuest(api_key=keys.mapquest_key)
```

There are two senators from each state, so we can look up each state's location once and use the `Location` object for both senators from that state. Let's get the unique state names, then sort them into ascending order:

[lick here to view code image](#)

```
In [25]: states = tweet_counts_df.State.unique()
```

```
In [26]: states.sort()
```

The next two snippets use code from the “Data Mining Twitter” chapter to look up each state's location. In snippet [28], we call the `geocode` function with the state name followed by ', USA' to ensure that we get United States locations,⁸ since there are places outside the United States with the same names as U.S. states. To show progress, we display each new `Location` object's string:

⁸When we initially performed the geocoding for Washington state, OpenMapQuest returned Washington, D.C.s location. So we modified `state_codes.py` to use Washington State instead.

[lick here to view code image](#)

```
In [27]: locations = []

In [28]: for state in states:
...:     processed = False
...:     delay = .1
...:     while not processed:
...:         try:
...:             locations.append(
...:                 geo.geocode(state_codes[state] + ', USA'))
...:             print(locations[-1])
...:             processed = True
...:         except: # timed out, so wait before trying again
...:             print('OpenMapQuest service timed out. Waiting.')
...:             time.sleep(delay)
...:             delay += .1
...:
Alaska, United States of America
Alabama, United States of America
Arkansas, United States of America
...
```

Grouping the Tweet Counts by State

We'll use the total number of tweets for the two senators in a state to color that state on the map. Darker colors will represent the states with higher tweet counts. To prepare the data for mapping, let's use the pandas `DataFrame` method **`groupby`** to group the senators by state and calculate the total tweets by state:

[lick here to view code image](#)

```
In [29]: tweets_counts_by_state = tweet_counts_df.groupby(
...:     'State', as_index=False).sum()
...:

In [30]: tweets_counts_by_state.head()
Out[30]:
```

	State	Tweets
0	AK	27
1	AL	2
2	AR	47
3	AZ	47
4	CA	1135

The `as_index=False` keyword argument in snippet [29] indicates that the state codes should be values in a column of the resulting **GroupBy** object, rather than the indices for the rows. The **GroupBy** object's `sum` method totals the numeric data (the tweets by state). Snippet [30] displays several rows of the **GroupBy** object so you can see some of the results.

Creating the Map

Next, let's create the map. You may want to adjust the zoom. On our system, the following snippet creates a map in which we initially can see only the continental United States. Remember that Folium maps are interactive, so once the map is displayed, you can scroll to zoom in and out or drag to see different areas, such as Alaska or Hawaii:

[lick here to view code image](#)

```
In [31]: import folium

In [32]: usmap = folium.Map(location=[39.8283, -98.5795],
...:     zoom_start=4, detect_retina=True,
...:     tiles='Stamen Toner')
...:
```

Creating a Choropleth to Color the Map

A **choropleth** shades areas in a map using the values you specify to determine color. Let's create a choropleth that colors the states by the number of tweets containing their senators' Twitter handles. First, save Folium's `us-states.json` file at

```
https://raw.githubusercontent.com/python-visualization/folium/master/examples/d
```

o the folder containing this example. This file contains a JSON dialect called **GeoJSON (Geographic JSON)** that describes the boundaries of shapes—in this case, the boundaries of every U.S. state. The choropleth uses this information to shade each state. For more about GeoJSON, see <http://geojson.org/>.⁹ The following snippets create the choropleth, then add it to the map:

⁹Folium provides several other GeoJSON files in its examples folder at <https://github.com/python-visualization/folium/tree/master/examples/data>. You also can create your own

at <http://geojson.io>.

[lick here to view code image](#)

```
In [33]: choropleth = folium.Choropleth(
...:     geo_data='us-states.json',
...:     name='choropleth',
...:     data=tweets_counts_by_state,
...:     columns=['State', 'Tweets'],
...:     key_on='feature.id',
...:     fill_color='YlOrRd',
...:     fill_opacity=0.7,
...:     line_opacity=0.2,
...:     legend_name='Tweets by State'
...: ).add_to(usmap)
...:

In [34]: layer = folium.LayerControl().add_to(usmap)
```

In this case, we used the following arguments:

- `geo_data='us-states.json'`—This is the file containing the GeoJSON that specifies the shapes to color.
- `name='choropleth'`—Folium displays the `Choropleth` as a layer over the map. This is the name for that layer that will appear in the map's layer controls, which enable you to hide and show the layers. These controls appear when you click the layers icon () on the map.
- `data=tweets_counts_by_state`—This is a pandas `DataFrame` (or `Series`) containing the values that determine the `Choropleth` colors.
- `columns=['State', 'Tweets']`—When the data is a `DataFrame`, this is a list of two columns representing the keys and the corresponding values used to color the `Choropleth`.
- `key_on='feature.id'`—This is a variable in the GeoJSON file to which the `Choropleth` binds the values in the `columns` argument.
- `fill_color='YlOrRd'`—This is a color map specifying the colors to use to fill in the states. Folium provides 12 colormaps: 'BuGn', 'BuPu', 'GnBu', 'OrRd', 'PuBu', 'PuBuGn', 'PuRd', 'RdPu', 'YlGn', 'YlGnBu', 'YlOrBr' and 'YlOrRd'. You should experiment with these to find the most effective and eye-pleasing ones for your application(s).
- `fill_opacity=0.7`—A value from 0.0 (transparent) to 1.0 (opaque) specifying the transparency of the fill colors displayed in the states.
- `line_opacity=0.2`—A value from 0.0 (transparent) to 1.0 (opaque) specifying the transparency of lines used to delineate the states.
- `legend_name='Tweets by State'`—At the top of the map, the `Choropleth` displays a color bar (the legend) indicating the value range represented by the colors. This `legend_name` text appears below the color bar to indicate what the colors represent.

The complete list of Choropleth keyword arguments is documented at:

```
http://python-visualization.github.io/folium/modules.html#folium.features.Choro
```

reating the Map Markers for Each State

Next, we'll create `Markers` for each state. To ensure that the senators are displayed in descending order by the number of tweets in each state's `Marker`, let's sort `tweet_counts_df` in descending order by the 'Tweets' column:

[lick here to view code image](#)

```
In [35]: sorted_df = tweet_counts_df.sort_values(
...:     by='Tweets', ascending=False)
...:
```

The loop in the following snippet creates the `Markers`. First,

```
sorted_df.groupby('State')
```

groups `sorted_df` by 'State'. A `DataFrame`'s `groupby` method maintains the *original row order* in each group. Within a given group, the senator with the most tweets will be first, because we sorted the senators in descending order by tweet count in snippet [35]:

[lick here to view code image](#)

```
In [36]: for index, (name, group) in enumerate(sorted_df.groupby('State')):
...:     strings = [state_codes[name]] # used to assemble popup text
...:
...:     for s in group.itertuples():
...:         strings.append(
...:             strings.append(
...:
...:         text = '<br>'.join(strings)
...:         marker = folium.Marker(
...:             (locations[index].latitude, locations[index].longitude),
...:             popup=text)
...:         marker.add_to(usmap)
...:
...:
```

We pass the grouped `DataFrame` to `enumerate`, so we can get an index for each group, which we'll use to look up each state's `Location` in the `locations` list. Each group has a name (the state code we grouped by) and a collection of items in that group (the two senators for that state). The loop operates as follows:

- We look up the full state name in the `state_codes` dictionary, then store it in the `strings` list—we'll use this list to assemble the `Marker`'s popup text.
- The nested loop walks through the items in the `group` collection, returning each as a named tuple that contains a given senator's data. We create a formatted string for the current senator containing the person's name, party and number of tweets, then append

that to the `strings` list.

- The `Marker` text can use HTML for formatting. We join the `strings` list's elements, separating each from the next with an HTML `<br>` element which creates a new line in HTML.
- We create the `Marker`. The first argument is the `Marker`'s location as a tuple containing the latitude and longitude. The `popup` keyword argument specifies the text to display if the user clicks the `Marker`.
- We add the `Marker` to the map.

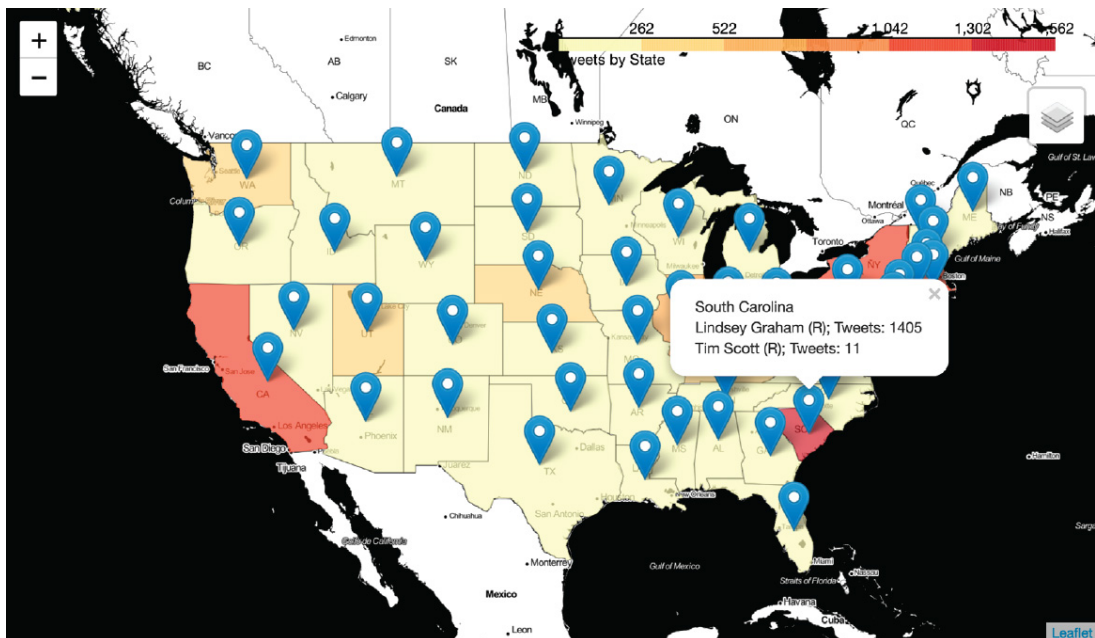
Displaying the Map

Finally, let's save the map into an HTML file

[lick here to view code image](#)

```
In [17]: usmap.save('SenatorsTweets.html')
```

Open the HTML file in your web browser to view and interact with the map. Recall that you can drag the map to see Alaska and Hawaii. Here we show the popup text for the South Carolina marker:



You could enhance this case study to use the sentiment-analysis techniques you learned in previous chapters to rate as positive, neutral or negative the sentiment expressed by people who send tweets (“tweeters”) mentioning each senator’s handle.

16.5 HADOOP

The next several sections show how Apache Hadoop and Apache Spark deal with big-data storage and processing challenges via huge clusters of computers, massively parallel processing, Hadoop MapReduce programming and Spark in-memory processing techniques. Here, we discuss Apache Hadoop, a key big-data infrastructure technology that also serves as the foundation for many recent advancements in big-data processing and an entire ecosystem

f software tools that are continually evolving to support today's big-data needs.

16.5.1 Hadoop Overview

When Google was launched in 1998, the amount of online data was already enormous with approximately 2.4 million websites ⁰—truly big data. Today there are now nearly two billion websites ¹ (almost a thousandfold increase) and Google is handling over two trillion searches per year! ² Having used Google search since its inception, our sense is that today's responses are significantly faster.

⁰ <http://www.internetlivestats.com/total-number-of-websites/>.

¹ <http://www.internetlivestats.com/total-number-of-websites/>.

² <http://www.internetlivestats.com/google-search-statistics/>.

When Google was developing their search engine, they knew that they needed to return search results quickly. The only practical way to do this was to store and index the entire Internet using a clever combination of secondary storage and main memory. Computers of that time couldn't hold that amount of data and could not analyze that amount of data fast enough to guarantee prompt search-query responses. So Google developed a **clustering** system, tying together vast numbers of computers—called **nodes**. Because having more computers and more connections between them meant greater chance of hardware failures, they also built in high levels of *redundancy* to ensure that the system would continue functioning even if nodes within clusters failed. The data was distributed across all these inexpensive “commodity computers.” To satisfy a search request, all the computers in the cluster searched in parallel the portion of the web they had locally. Then the results of those searches were gathered up and reported back to the user.

To accomplish this, Google needed to develop the clustering hardware and software, including distributed storage. Google publishes its designs, but did not open source its software. Programmers at Yahoo!, working from Google's designs in the “Google File System” paper, ³ then built their own system. They open-sourced their work and the Apache organization implemented the system as Hadoop. The name came from an elephant stuffed animal that belonged to a child of one of Hadoop's creators.

³ <http://static.googleusercontent.com/media/research.google.com/en//archive/gfosp2003.pdf>.

Two additional Google papers also contributed to the evolution of Hadoop—“MapReduce: Simplified Data Processing on Large Clusters” ⁴ and “Bigtable: A Distributed Storage System for Structured Data,” ⁵ which was the basis for Apache HBase (a NoSQL key-value and column--based database). ⁶

⁴ <http://static.googleusercontent.com/media/research.google.com/en//archive/masdi04.pdf>.

⁵ <http://static.googleusercontent.com/media/research.google.com/en//archive/bisdi06.pdf>.

⁶Many other influential big-data-related papers (including the ones we mentioned) can be

found at: <https://bigdata-madesimple.com/research-papers-that-changed-the-world-of-big-data/>.

DFS, MapReduce and YARN

Hadoop's key components are:

- **HDFS (Hadoop Distributed File System)** for storing massive amounts of data throughout a cluster, and
- **MapReduce** for implementing the tasks that process the data.

Earlier in the book we introduced basic functional-style programming and filter/map/reduce. Hadoop MapReduce is similar in concept, just on a massively parallel scale. A MapReduce task performs two steps—**mapping** and **reduction**. The mapping step, which also may include *filtering*, processes the original data across the entire cluster and maps it into tuples of key–value pairs. The reduction step then combines those tuples to produce the results of the MapReduce task. The key is how the MapReduce step is performed. Hadoop divides the data into *batches* that it distributes across the nodes in the cluster—anywhere from a few nodes to a Yahoo! cluster with 40,000 nodes and over 100,000 cores.⁷ Hadoop also distributes the MapReduce task's code to the nodes in the cluster and executes the code in parallel on every node. Each node processes only the batch of data stored on that node. The reduction step combines the results from all the nodes to produce the final result. To coordinate this, Hadoop uses **YARN** (“yet another resource negotiator”) to manage all the resources in the cluster and schedule tasks for execution.

⁷ <https://wiki.apache.org/hadoop/PoweredBy>.

Hadoop Ecosystem

Though Hadoop began with HDFS and MapReduce, followed closely by YARN, it has grown into a large ecosystem that includes Spark (discussed in sections 16.6– 16.7) and many other Apache projects:^{8, 9, 10}

⁸ <https://hortonworks.com/ecosystems/>.

⁹ <https://readwrite.com/2018/06/26/complete-guide-of-hadoop-ecosystem-components/>.

¹⁰ <https://www.janbasktraining.com/blog/introduction-architecture-components-hadoop-ecosystem/>.

- **Ambari** (<https://ambari.apache.org>)—Tools for managing Hadoop clusters.
- **Drill** (<https://drill.apache.org>)—SQL querying of non-relational data in Hadoop and NoSQL databases.
- **Flume** (<https://flume.apache.org>)—A service for collecting and storing (in HDFS and other storage) streaming event data, like high-volume server logs, IoT messages and more.
- **HBase** (<https://hbase.apache.org>)—A NoSQL database for big data with “billions

rows by ¹ millions of columns—atop clusters of commodity hardware.”

¹We used the word by to replace X in the original text.

- **Hive** (<https://hive.apache.org>)—Uses SQL to interact with data in data warehouses. A **data warehouse** aggregates data of various types from various sources. Common operations include extracting data, transforming it and loading (known as **ETL**) into another database, typically so you can analyze it and create reports from it.
- **Impala** (<https://impala.apache.org>)—A database for real-time SQL-based queries across distributed data stored in Hadoop HDFS or HBase.
- **Kafka** (<https://kafka.apache.org>)—Real-time messaging, stream processing and storage, typically to transform and process high-volume streaming data, such as website activity and streaming IoT data.
- **Pig** (<https://pig.apache.org>)—A scripting platform that converts data analysis tasks from a scripting language called **Pig Latin** into MapReduce tasks.
- **Sqoop** (<https://sqoop.apache.org>)—Tool for moving structured, semi-structured and unstructured data between databases.
- **Storm** (<https://storm.apache.org>)—A real-time stream-processing system for tasks such as data analytics, machine learning, ETL and more.
- **ZooKeeper** (<https://zookeeper.apache.org>)—A service for managing cluster configurations and coordination between clusters.
- And more.

Hadoop Providers

Numerous cloud vendors provide Hadoop as a service, including Amazon EMR, Google Cloud DataProc, IBM Watson Analytics Engine, Microsoft Azure HDInsight and others. In addition, companies like Cloudera and Hortonworks (which at the time of this writing are merging) offer integrated Hadoop-ecosystem components and tools via the major cloud vendors. They also offer free *downloadable* environments that you can run on the desktop ² for learning, development and testing before you commit to cloud-based hosting, which can incur significant costs. We introduce MapReduce programming in the example in the following sections by using a Microsoft cloud-based Azure HDInsight cluster, which provides Hadoop as a service.

²Check their significant system requirements first to ensure that you have the disk space and memory required to run them.

Hadoop 3

Apache continues to evolve Hadoop. Hadoop 3 ³ was released in December of 2017 with many improvements, including better performance and significantly improved storage efficiency. ⁴

³For a list of features in Hadoop 3, see <https://hadoop.apache.org/docs/r3.0.0/>.

⁴ <https://www.datanami.com/2018/10/18/is-hadoop-officially-dead/>.

16.5.2 Summarizing Word Lengths in *Romeo and Juliet* via MapReduce

In the next several subsections, you'll create a cloud-based, multi-node cluster of computers using Microsoft Azure HDInsight. Then, you'll use the service's capabilities to demonstrate Hadoop MapReduce running on that cluster. The MapReduce task you'll define will determine the length of each word in `RomeoAndJuliet.txt` (from the “Natural Language Processing” chapter), then summarize how many words of each length there are. After defining the task's mapping and reduction steps, you'll submit the task to your HDInsight cluster, and Hadoop will decide how to use the cluster of computers to perform the task.

16.5.3 Creating an Apache Hadoop Cluster in Microsoft Azure HDInsight

Most major cloud vendors have support for Hadoop and Spark computing clusters that you can configure to meet your application's requirements. Multi-node cloud-based clusters typically are *paid* services, though most vendors provide free trials or credits so you can try out their services.

We want you to experience the process of setting up clusters and using them to perform tasks. So, in this Hadoop example, you'll use Microsoft Azure's HDInsight service to create cloud-based clusters of computers in which to test our examples. Go to

```
https://azure.microsoft.com/en-us/free
```

to sign up for an account. Microsoft requires a credit card for identity verification.

Various services are always free and some you can continue to use for 12 months. For information on these services see:

```
https://azure.microsoft.com/en-us/free/free-account-faq/
```

Microsoft also gives you a credit to experiment with their *paid* services, such as their HDInsight Hadoop and Spark services. Once your credits run out or 30 days pass (whichever comes first), you cannot continue using paid services unless you authorize Microsoft to charge your card.

Because you'll use your new Azure account's credit for these examples, ⁵ we'll discuss how to configure a low-cost cluster that uses less computing resources than Microsoft allocates by default. ⁶ **Caution: Once you allocate a cluster, it incurs costs whether you're using it or not. So, when you complete this case study, be sure to delete your cluster(s) and other resources, so you don't incur additional charges.** For more information, see:

⁵For Microsoft's latest free account features, visit <https://azure.microsoft.com/en-us/free/>.

⁶For Microsoft's recommended cluster configurations, see

<https://docs.microsoft.com/en-us/azure/hdinsight/hdinsight-component-versioning#default-node-configuration-andvirtual-machine->

izes-for-clusters. If you configure a cluster that's too small for a given scenario, when you try to deploy the cluster you'll receive an error.

```
https://docs.microsoft.com/en-us/azure/azure-resource-manager/resource-group-po
```

or Azure-related documentation and videos, visit:

- <https://docs.microsoft.com/en-us/azure/>—the Azure documentation.
- <https://channel9.msdn.com/>—Microsoft's Channel 9 video network.
- <https://www.youtube.com/user/windowsazure>—Microsoft's Azure channel on YouTube.

Creating an HDInsight Hadoop Cluster

The following link explains how to set up a cluster for Hadoop using the Azure HDInsight service:

```
https://docs.microsoft.com/en-us/azure/hdinsight/hadoop/apache-hadoop-linux-cre
```

While following their **Create a Hadoop cluster** steps, please note the following:

- In *Step 1*, you access the Azure portal by logging into your account at

```
https://portal.azure.com
```

- In *Step 2*, **Data + Analytics** is now called **Analytics**, and the HDInsight icon and icon color have changed from what is shown in the tutorial.
- In *Step 3*, you must choose a cluster name that does *not* already exist. When you enter your cluster name, Microsoft will check whether that name is available and display a message if it is not. You must create a password. For the **Resource group**, you'll also need to click **Create new** and provide a group name. Leave all other settings in this step as is.
- In *Step 5*: Under **Select a Storage account**, click **Create new** and provide a storage account name containing only lowercase letters and numbers. Like the cluster name, the storage account name must be unique.

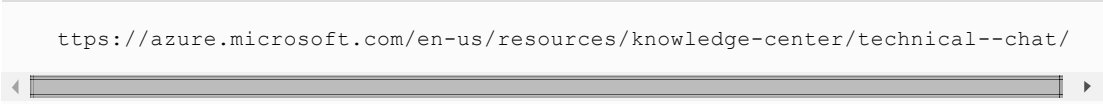
When you get to the **Cluster summary** you'll see that Microsoft initially configures the cluster as **Head (2 x D12 v2), Worker (4 x D4 v2)**. At the time of this writing, the estimated cost-per-hour for this configuration was \$3.11. This setup uses a total of 6 CPU nodes with 40 cores—far more than we need for demonstration purposes.

You can edit this setup to use fewer CPUs and cores, which also saves money. Let's change the configuration to a four-CPU cluster with 16 cores that uses less powerful computers. In the **Cluster summary**:

1. Click **Edit** to the right of **Cluster size**.
2. Change the **Number of Worker** nodes to 2.
3. Click **Worker node size**, then **View all**, select **D3 v2** (this is the minimum CPU size for Hadoop nodes) and click **Select**.
4. Click **Head node size**, then **View all**, select **D3 v2** and click **Select**.
5. Click **Next** and click **Next** again to return to the **Cluster summary**. Microsoft will validate the new configuration.
6. When the **Create** button is enabled, click it to deploy the cluster.

It takes 20–30 minutes for Microsoft to “spin up” your cluster. During this time, Microsoft is allocating all the resources and software the cluster requires.

After the changes above, our estimated cost for the cluster was \$1.18 per hour, based on *average* use for similarly configured clusters. Our actual charges were less than that. If you encounter any problems configuring your cluster, Microsoft provides HDInsight chat-based support at:



```
https://azure.microsoft.com/en-us/resources/knowledge-center/technical--chat/
```

16.5.4 Hadoop Streaming

For languages like Python that are not natively supported in Hadoop, you must use **Hadoop streaming** to implement your tasks. In Hadoop streaming, the Python scripts that implement the mapping and reduction steps use the **standard input stream** and **standard output stream** to communicate with Hadoop. Usually, the standard input stream reads from the keyboard and the standard output stream writes to the command line. However, these can be *redirected* (as Hadoop does) to read from other sources and write to other destinations. Hadoop uses the streams as follows:

- Hadoop supplies the input to the mapping script—called the **mapper**. This script reads its input from the standard input stream.
- The mapper writes its results to the standard output stream.
- Hadoop supplies the mapper’s output as the input to the reduction script—called the **reducer**—which reads from the standard input stream.
- The reducer writes its results to the standard output stream.
- Hadoop writes the reducer’s output to the Hadoop file system (HDFS).

The mapper and reducer terminology used above should sound familiar to you from our discussions of functional-style programming and filter, map and reduce in the “Sequences: Lists and Tuples” chapter.

16.5.5 Implementing the Mapper

In this section, you'll create a mapper script that takes lines of text as input from Hadoop and maps them to *key-value pairs* in which each key is a word, and its corresponding value is 1. The mapper sees each word individually so, as far as it is concerned, there's only one of each word. In the next section, the reducer will summarize these key-value pairs by key, reducing the counts to a single count for each key. By default, Hadoop expects the mapper's output and the reducer's input and output to be in the form of key-value pairs separated by a *tab*.

In the mapper script (`length_mapper.py`), the notation `#!` in line 1 tells Hadoop to execute the Python code using `python3`, rather than the default Python 2 installation. This line must come before all other comments and code in the file. At the time of this writing, Python 2.7.12 and Python 3.5.2 were installed. Note that because the cluster does not have Python 3.6 or higher, you cannot use f-strings in your code.

[lick here to view code image](#)

```
1 #!/usr/bin/env python3
2 # length_mapper.py
3 """Maps lines of text to key-value pairs of word lengths and 1."""
4 import sys
5
6 def tokenize_input():
7     """Split each line of standard input into a list of strings."""
8     for line in sys.stdin:
9         yield line.split()
10
11 # read each line in the the standard input and for every word
12 # produce a key-value pair containing the word, a tab and 1
13 for line in tokenize_input():
14     for word in line:
15         print(str(len(word)) + '\t1')
```

Generator function `tokenize_input` (lines 6–9) reads lines of text from the standard input stream and for each returns a list of strings. For this example, we are not removing punctuation or stop words as we did in the “atural Language Processing” chapter.

When Hadoop executes the script, lines 13–15 iterate through the lists of strings from `tokenize_input`. For each list (`line`) and for every string (`word`) in that list, line 15 outputs a key-value pair with the word's length as the key, a tab (`\t`) and the value 1, indicating that there is one word (so far) of that length. Of course, there probably are many words of that length. The MapReduce algorithm's reduction step will summarize these key-value pairs, reducing all those with the same key to a single key-value pair with the total count.

16.5.6 Implementing the Reducer

In the reducer script (`length_reducer.py`), function `tokenize_input` (lines 8–11) is a generator function that reads and splits the key-value pairs produced by the mapper. Again, the MapReduce algorithm supplies the standard input. For each line, `tokenize_input` strips any leading or trailing whitespace (such as the terminating newline) and yields a list containing the key and a value.

[lick here to view code image](#)

```

1 #!/usr/bin/env python3
2 # length_reducer.py
3 """Counts the number of words with each length."""
4 import sys
5 from itertools import groupby
6 from operator import itemgetter
7
8 def tokenize_input():
9     """Split each line of standard input into a key and a value."""
10    for line in sys.stdin:
11        yield line.strip().split('\t')
12
13 # produce key-value pairs of word lengths and counts separated by tabs
14 for word_length, group in groupby(tokenize_input(), itemgetter(0)):
15     try:
16         total = sum(int(count) for word_length, count in group)
17         print(word_length + '\t' + str(total))
18     except ValueError:
19         pass # ignore word if its count was not an integer

```

When the MapReduce algorithm executes this reducer, lines 14–19 use the **groupby** function from the **itertools module** to group all word lengths of the same value:

- The first argument calls `tokenize_input` to get the lists representing the key–value pairs.
- The second argument indicates that the key–value pairs should be grouped based on the element at index 0 in each list—that is the key.

Line 16 totals all the counts for a given key. Line 17 outputs a new key–value pair consisting of the word and its total. The MapReduce algorithm takes all the final word-count outputs and writes them to a file in HDFS—the Hadoop file system.

16.5.7 Preparing to Run the MapReduce Example

Next, you’ll upload files to the cluster so you can execute the example. In a Command Prompt, Terminal or shell, change to the folder containing your mapper and reducer scripts and the `RomeoAndJuliet.txt` file. We assume all three are in this chapter’s `ch16` examples folder, so be sure to copy your `RomeoAndJuliet.txt` file to this folder first.

Copying the Script Files to the HDInsight Hadoop Cluster

Enter the following command to upload the files. Be sure to replace *YourClusterName* with the cluster name you specified when setting up the Hadoop cluster and press *Enter* only after you’ve typed the entire command. The colon in the following command is required and indicates that you’ll supply your cluster password when prompted. At that prompt, type the password you specified when setting up the cluster, then press *Enter*:

[lick here to view code image](#)

```

scp length_mapper.py length_reducer.py RomeoAndJuliet.txt
sshuser@YourClusterName-ssh.azurehdinsight.net:

```

The first time you do this, you’ll be asked for security reasons to confirm that you trust the

target host (that is, Microsoft Azure).

Copying RomeoAndJuliet into the Hadoop File System

For Hadoop to read the contents of `RomeoAndJuliet.txt` and supply the lines of text to your mapper, you must first copy the file into Hadoop's file system. First, you must use `ssh` ⁷ to log into your cluster and access its command line. In a Command Prompt, Terminal or shell, execute the following command. Be sure to replace *YourClusterName* with your cluster name. Again, you'll be prompted for your cluster password:

⁷Windows users: If `ssh` does not work for you, install and enable it as described at <https://blogs.msdn.microsoft.com/powershell/2017/12/15/using-the-openssh-beta-in-windows-10-fall-creators-update-and-windows-server-1709/>. After completing the installation, log out and log back in or restart your system to enable `ssh`.

[lick here to view code image](#)

```
ssh sshuser@YourClusterName-ssh.azurehdinsight.net
```

For this example, we'll use the following Hadoop command to copy the text file into the already existing folder `/examples/data` that the cluster provides for use with Microsoft's Azure Hadoop tutorials. Again, press *Enter* only when you've typed the entire command:

[lick here to view code image](#)

```
hadoop fs -copyFromLocal RomeoAndJuliet.txt
/example/data/RomeoAndJuliet.txt
```

16.5.8 Running the MapReduce Job

Now you can run the MapReduce job for `RomeoAndJuliet.txt` on your cluster by executing the following command. For your convenience, we provided the text of this command in the file `yarn.txt` with this example, so you can copy and paste it. We reformatted the command here for readability:

[lick here to view code image](#)

```
yarn jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-streaming.jar
-D mapred.output.key.comparator.class=
    org.apache.hadoop.mapred.lib.KeyFieldBasedComparator
-D mapred.text.key.comparator.options=-n
-files length_mapper.py,length_reducer.py
-mapper length_mapper.py
-reducer length_reducer.py
-input /example/data/RomeoAndJuliet.txt
-output /example/wordlengthsoutput
```

The **yarn command** invokes the Hadoop's YARN ("yet another resource negotiator") tool to manage and coordinate access to the Hadoop resources the MapReduce task uses. The file `hadoop-streaming.jar` contains the Hadoop streaming utility that allows you to use Python to implement the mapper and reducer. The two `-D` options set Hadoop properties

that enable it to sort the final key–value pairs by key (`KeyFieldBasedComparator`) in descending order numerically (`-n`; the minus indicates descending order) rather than alphabetically. The other command-line arguments are:

- `-files`—A comma-separated list of file names. Hadoop copies these files to every node in the cluster so they can be executed locally on each node.
- `-mapper`—The name of the mapper’s script file.
- `-reducer`—The name of the reducer’s script file
- `-input`—The file or directory of files to supply as input to the mapper.
- `-output`—The HDFS directory in which the output will be written. If this folder already exists, an error will occur.

The following output shows some of the feedback that Hadoop produces as the MapReduce job executes. We replaced chunks of the output with `to save space` and bolded several lines of interest including:

- The total number of “input paths to process”—the 1 source of input in this example is the `RomeoAndJuliet.txt` file.
- The “number of splits”—2 in this example, based on the number of worker nodes in our cluster.
- The percentage completion information.
- File System Counters, which include the numbers of bytes read and written.
- Job Counters, which show the number of mapping and reduction tasks used and various timing information.
- Map-Reduce Framework, which shows various information about the steps performed.

[lick here to view code image](#)

```
ackageJobJar: []      [/usr/hdp/2.6.5.3004-13/hadoop-mapreduce/hadoop-streaming-2.
...
18/12/05 16:46:25 INFO  mapred.FileInputFormat: Total input paths to process :
18/12/05 16:46:26 INFO  mapreduce.JobSubmitter: number of splits:2
...
18/12/05 16:46:26 INFO mapreduce.Job: The url to track the  job:  http://hn0-p
...
18/12/05 16:46:35 INFO mapreduce.Job:  map    0% reduce 0%
18/12/05 16:46:43 INFO mapreduce.Job:  map    50% reduce 0%
18/12/05 16:46:44 INFO mapreduce.Job:  map   100% reduce 0%
18/12/05 16:46:48 INFO mapreduce.Job:  map   100% reduce 100%
18/12/05 16:46:50 INFO mapreduce.Job: Job    job_1543953844228_0025 completed suc
18/12/05 16:46:50 INFO mapreduce.Job: Counters: 49
      File System Counters
        FILE: Number of bytes   read=156411
        FILE: Number of bytes   written=813764
...
      Job Counters
```

```
Launched map tasks=2
Launched reduce tasks=1

...
Map-Reduce Framework
  Map input records=5260
  Map output records=25956
  Map output bytes=104493
  Map output materialized bytes=156417
  Input split bytes=346
  Combine input records=0
  Combine output records=0
  Reduce input groups=19
  Reduce shuffle bytes=156417
  Reduce input records=25956
  Reduce output records=19
  Spilled Records=51912
  Shuffled Maps =2
  Failed Shuffles=0
  Merged Map outputs=2
  GC time elapsed (ms)=193
  CPU time spent (ms)=4440
  Physical memory (bytes) snapshot=1942798336
  Virtual memory (bytes) snapshot=8463282176
  Total committed heap usage (bytes)=3177185280

...
18/12/05 16:46:50 INFO streaming.StreamJob: Output directory: /example/wordlengths
```

Viewing the Word Counts

Hadoop MapReduce saves its output into HDFS, so to see the actual word counts you must look at the file in HDFS within the cluster by executing the following command:

[click here to view code image](#)

```
hdfs dfs -text /example/wordlengthsoutput/part-00000
```

Here are the results of the preceding command:

[click here to view code image](#)

```
8/12/05 16:47:19 INFO lzo.GPLNativeCodeLoader: Loaded native gpl library
18/12/05 16:47:19 INFO lzo.LzoCodec: Successfully loaded & initialized native-
1      1140
2      3869
3      4699
4      5651
5      3668
6      2719
7      1624
8      1062
9       855
10     317
11     189
12     95
13     35
14     13
15     9
16     6
17     3
18     1
23     1
```

Deleting Your Cluster So You Do Not Incur Charges

Caution: Be sure to delete your cluster(s) and associated resources (like storage) so you don't incur additional charges. In the Azure portal, click **All resources** to see your list of resources, which will include the cluster you set up and the storage account you set up. Both can incur charges if you do not delete them. Select each resource and click the **Delete** button to remove it. You'll be asked to confirm by typing `yes`. For more information, see:

```
https://docs.microsoft.com/en-us/azure/azure-resource-manager/resource-group-po
```

6.6 SPARK

In this section, we'll overview Apache Spark. We'll use the Python **PySpark library** and Spark's functional-style filter/map/reduce capabilities to implement a simple word count example that summarizes the word counts in *Romeo and Juliet*.

16.6.1 Spark Overview

When you process truly big data, performance is crucial. Hadoop is geared to disk-based batch processing—reading the data from disk, processing the data and writing the results back to disk. Many big-data applications demand better performance than is possible with disk-intensive operations. In particular, fast streaming applications that require either real-time or near-real-time processing won't work in a disk-based architecture.

History

Spark was initially developed in 2009 at U. C. Berkeley and funded by DARPA (the Defense Advanced Research Projects Agency). Initially, it was created as a distributed execution engine for high-performance machine learning.⁸ It uses an **in-memory architecture** that “has been used to sort 100 TB of data 3X faster than Hadoop MapReduce on 1/10th of the machines”⁹ and runs some workloads up to 100 times faster than Hadoop.⁰ Spark's significantly better performance on batch-processing tasks is leading many companies to replace Hadoop MapReduce with Spark.^{1, 2 3}

⁸ <https://gigaom.com/2014/06/28/4-reasons-why-spark-could-jolt-hadoop-into-hyperdrive/>.

⁹ <https://spark.apache.org/faq.html>.

⁰ <https://spark.apache.org/>.

¹ <https://bigdata-madesimple.com/is-spark-better-than-hadoop-map-reduce/>.

² <https://www.datanami.com/2018/10/18/is-hadoop-officially-dead/>.

³ <https://blog.thecodeteam.com/2018/01/09/changing-face-data-analytics-fast-data-displaces-big-data/>.

Architecture and Components

Though it was initially developed to run on Hadoop and use Hadoop components like HDFS and YARN, Spark can run standalone on a single computer (typically for learning and testing purposes), standalone on a cluster or using various cluster managers and distributed storage systems. For resource management, Spark runs on Hadoop YARN, Apache Mesos, Amazon EC2 and Kubernetes, and it supports many distributed storage systems, including HDFS, Apache Cassandra, Apache HBase and Apache Hive. ⁴

⁴ <http://spark.apache.org/>.

At the core of Spark are **resilient distributed datasets (RDDs)**, which you'll use to process distributed data using functional-style programming. In addition to reading data from disk and writing data to disk, Hadoop uses replication for fault tolerance, which adds even more disk-based overhead. RDDs eliminate this overhead by remaining in memory—using disk only if the data will not fit in memory—and by not replicating data. Spark handles fault tolerance by remembering the steps used to create each RDD, so it can rebuild a given RDD if a cluster node fails. ⁵

⁵ <https://spark.apache.org/research.html>.

Spark distributes the operations you specify in Python to the cluster's nodes for parallel execution. Spark streaming enables you to process data as it's received. Spark `DataFrames`, which are similar to pandas `DataFrames`, enable you to view RDDs as a collection of named columns. You can use Spark `DataFrames` with Spark SQL to perform queries on distributed data. Spark also includes **Spark MLlib** (the Spark Machine Learning Library), which enables you to perform machine-learning algorithms, like those you learned in [chapters 14 and 15](#). We'll use RDDs, Spark streaming, `DataFrames` and Spark SQL in the next few examples.

Providers

Hadoop providers typically also provide Spark support. In addition to the providers listed in [section 16.5](#), there are Spark-specific vendors like Databricks. They provide a “zero-management cloud platform built around Spark.” ⁶ Their website also is an excellent resource for learning Spark. The paid Databricks platform runs on Amazon AWS or Microsoft Azure. Databricks also provides a free Databricks Community Edition, which is a great way to get started with both Spark and the Databricks environment.

⁶ <https://databricks.com/product/faq>.

16.6.2 Docker and the Jupyter Docker Stacks

In this section, we'll show how to download and execute a Docker stack containing Spark and the PySpark module for accessing Spark from Python. You'll write the Spark example's code in a Jupyter Notebook. First, let's overview Docker.

Docker

Docker is a tool for packaging software into **containers** (also called **images**) that bundle *everything* required to execute that software across platforms. Some software packages we use in this chapter require complicated setup and configuration. For many of these, there are

preexisting Docker containers that you can download for free and execute locally on your desktop or notebook computers. This makes Docker a great way to help you get started with new technologies quickly and conveniently.

Docker also helps with *reproducibility* in research and analytics studies. You can create custom Docker containers that are configured with the versions of every piece of software and every library you used in your study. This would enable others to recreate the environment you used, then reproduce your work, and will help you reproduce your results at a later time. We'll use Docker in this section to download and execute a Docker container that's preconfigured to run Spark applications.

Installing Docker

You can install Docker for Windows 10 Pro or macOS at:

```
https://www.docker.com/products/docker-desktop
```

On Windows 10 Pro, you must allow the "Docker for Windows.exe" installer to make changes to your system to complete the installation process. To do so, click **Yes** when Windows asks if you want to allow the installer to make changes to your system.⁷ Windows 10 Home users must use Virtual Box as described at:

⁷Some Windows users might have to follow the instructions under **Allow specific apps to make changes to controlled folders** at <https://docs.microsoft.com/en-us/windows/security/threat-protection/windows-defender-exploit-guard/customize-controlled-folders-exploit-guard>.

```
https://docs.docker.com/machine/drivers/virtualbox/
```

Linux users should install Docker Community Edition as described at:

```
https://docs.docker.com/install/overview/
```

For a general overview of Docker, read the **Getting started** guide at:

```
https://docs.docker.com/get-started/
```

Jupyter Docker Stacks

The Jupyter Notebooks team has preconfigured several Jupyter "Docker stacks" containers for common Python development scenarios. Each enables you to use Jupyter Notebooks to experiment with powerful capabilities without having to worry about complex software setup issues. In each case, you can open JupyterLab in your web browser, open a notebook in JupyterLab and start coding. JupyterLab also provides a **Terminal window** that you can use in your browser like your computer's Terminal, Anaconda Command Prompt or shell. Everything we've shown you in IPython to this point can be executed using IPython in JupyterLab's Terminal window.

We'll use the `jupyter/pyspark-notebook` Docker stack, which is preconfigured with

everything you need to create and test Apache Spark apps on your computer. When combined with installing other Python libraries we've used throughout the book, you can implement most of this book's examples using this container. For more about the available Docker stacks, visit:

```
https://jupyter-docker-stacks.readthedocs.io/en/latest/index.html
```

Run Jupyter Docker Stack

Before performing the next step, ensure that JupyterLab is not currently running on your computer. Let's download and run the `jupyter/pyspark-notebook` Docker stack. To ensure that you do not lose your work when you close the Docker container, we'll attach a local file-system folder to the container and use it to save your notebook—Windows users should replace `\` with `^.`:

[lick here to view code image](#)

```
docker run -p 8888:8888 -p 4040:4040 -it --user root \
-v fullPathToTheFolderYouWantToUse:/home/jovyan/work \
jupyter/pyspark-notebook:14fd9bf9cfc1 start.sh jupyter lab
```

The first time you run the preceding command, Docker will download the Docker container named:

[lick here to view code image](#)

```
jupyter/pyspark-notebook:14fd9bf9cfc1
```

The notation `":14fd9bf9cfc1"` indicates the specific `jupyter/pyspark-notebook` container to download. At the time of this writing, `14fd9bf9cfc1` was the newest version of the container. Specifying the version as we did here helps with *reproducibility*. Without the `":14fd9bf9cfc1"` in the command, Docker will download the *latest* version of the container, which might contain different software versions and might not be compatible with the code you're trying to execute. The Docker container is nearly 6GB, so the initial download time will depend on your Internet connection's speed.

Opening JupyterLab in Your Browser

Once the container is downloaded and running, you'll see a statement in your Command Prompt, Terminal or shell window like:

[lick here to view code image](#)

```
Copy/paste this URL into your browser when you connect for the first time, to l
```

```
http://(bb00eb337630 or 127.0.0.1):8888/?token=
9570295e90ee94ecef75568b95545b7910a8f5502e6f5680
```

Copy the long hexadecimal string (the string on your system will differ from this one):

```
9570295e90ee94ecef75568b95545b7910a8f5502e6f5680
```

then open `http://localhost:8888/lab` in your browser (localhost corresponds to `127.0.0.1` in the preceding output) and *paste* your token in the **Password or token** field. Click **Log in** to be taken to the JupyterLab interface. If you accidentally close your browser, go to `http://localhost:8888/lab` to continue your session.

When running in this Docker container, the `work` folder in the **Files** tab at the left side of JupyterLab represents the folder you attached to the container in the `docker run` command's `-v` option. From here, you can open the notebook files we provide for you. Any new notebooks or other files you create will be saved to this folder by default. Because the Docker container's `work` folder is connected to a folder on your computer, any files you create in JupyterLab will remain on your computer, even if you decide to delete the Docker container.

Accessing the Docker Container's Command Line

Each Docker container has a command-line interface like the one you've used to run IPython throughout this book. Via this interface, you can install Python packages into the Docker container and even use IPython as you've done previously.

Open a separate Anaconda Command Prompt, Terminal or shell and list the currently running Docker containers with the command:

```
docker ps
```

The output of this command is wide, so the lines of text will likely wrap, as in:

[lick here to view code image](#)

CONTAINER ID	IMAGE	STATUS	PORTS	COMMAND
CREATED				
NAMES				
f54f62b7e6d5	jupyter/pyspark-notebook:14fd9bf9cfc1	Up 2 minutes	0.0.0.0:8888->8888/tcp	"tini -g -- /bin/bash"
friendly_pascal				

In the last line of our system's output under the column head `NAMES` in the third line is the name that Docker randomly assigned to the running container—`friendly_pascal`—the name on your system will differ. To access the container's command line, execute the following command, replacing `container_name` with the running container's name:

```
docker exec -it container_name /bin/bash
```

The Docker container uses Linux under the hood, so you'll see a Linux prompt where you can enter commands.

The app in this section will use features of the NLTK and TextBlob libraries you used in the “Natural Language Processing” chapter. Neither is preinstalled in the Jupyter Docker stacks. To install NLTK and TextBlob enter the command:

```
conda install -c conda-forge nltk textblob
```

Stopping and Restarting a Docker Container

Every time you start a container with `docker run`, Docker gives you a new instance that does *not* contain any libraries you installed previously. For this reason, you should keep track of your container name, so you can use it from another Anaconda Command Prompt, Terminal or shell window to stop the container and restart it. The command

```
docker stop container_name
```

will shut down the container. The command

```
docker restart container_name
```

will restart the container. Docker also provides a GUI app called Kitematic that you can use to manage your containers, including stopping and restarting them. You can get the app from <https://kitematic.com/> and access it through the Docker menu. The following user guide overviews how to manage containers with the tool:

```
https://docs.docker.com/kitematic/userguide/
```

16.6.3 Word Count with Spark

In this section, we'll use Spark's filtering, mapping and reducing capabilities to implement a simple word count example that summarizes the words in *Romeo and Juliet*. You can work with the existing notebook named `RomeoAndJulietCounter.ipynb` in the `SparkWordCount` folder (into which you should copy your `RomeoAndJuliet.txt` file from the “ Natural Language Processing” chapter), or you can create a new notebook, then enter and execute the snippets we show.

Loading the NLTK Stop Words

In this app, we'll use techniques you learned in the “ Natural Language Processing” chapter to eliminate stop words from the text before counting the words' frequencies. First, download the NLTK stop words:

[click here to view code image](#)

```
[1]: import nltk
    nltk.download('stopwords')
[nltk_data] Downloading package stopwords to /home/jovyan/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[1]: True
```

Next, load the stop words:

[click here to view code image](#)

```
[2]: from nltk.corpus import stopwords
```

```
stop_words = stopwords.words('english')
```

Configuring a SparkContext

A **SparkContext** (from module `pyspark`) object gives you access to Spark's capabilities in Python. Many Spark environments create the `SparkContext` for you, but in the Jupyter `pyspark-notebook` Docker stack, you must create this object.

First, let's specify the configuration options by creating a **SparkConf** object (from module `pyspark`). The following snippet calls the object's `setAppName` method to specify the Spark application's name and calls the object's `setMaster` method to specify the Spark cluster's URL. The URL `'local[*]'` indicates that Spark is executing on your `local` computer (as opposed to a cloud-based cluster), and the asterisk indicates that Spark should run our code using the same number of *threads* as there are cores on the computer:

[lick here to view code image](#)

```
[3]: from pyspark import SparkConf
     configuration = SparkConf().setAppName('RomeoAndJulietCounter') \
         .setMaster('local[*]')
```

Threads enable a single node cluster to execute portions of the Spark tasks *concurrently* to simulate the parallelism that Spark clusters provide. When we say that two tasks are operating concurrently, we mean that they're both making progress at once—typically by executing a task for a short burst of time, then allowing another task to execute. When we say that two tasks are operating in *parallel*, we mean that they're executing simultaneously, which is one of the key benefits of Hadoop and Spark executing on cloud-based clusters of computers.

Next, create the `SparkContext`, passing the `SparkConf` as its argument:

[lick here to view code image](#)

```
[4]: from pyspark import SparkContext
     sc = SparkContext(conf=configuration)
```

Reading the Text File and Mapping It to Words

You work with a `SparkContext` using functional-style programming techniques, like filtering, mapping and reduction, applied to a **resilient distributed dataset (RDD)**. An RDD takes data stored throughout a cluster in the Hadoop file system and enables you to specify a series of processing steps to transform the data in the RDD. These processing steps are *lazy* (chapter 5)—they do not perform any work until you indicate that Spark should process the task.

The following snippet specifies three steps:

- `SparkContext` method `textFile` loads the lines of text from `RomeoAndJuliet.txt` and returns it as an **RDD** (from module `pyspark`) of strings that represent each line.
- RDD method `map` uses its `lambda` argument to remove all punctuation with `TextBlob`'s

`strip_punc` function and to convert each line of text to lowercase. This method returns a new RDD on which you can specify additional tasks to perform.

- RDD method `flatMap` uses its `lambda` argument to map each line of text into its words and produces a single list of words, rather than the individual lines of text. The result of `flatMap` is a new RDD representing all the words in *Romeo and Juliet*.

[lick here to view code image](#)

```
[5]: from textblob.utils import strip_punc
    tokenized = sc.textFile('RomeoAndJuliet.txt')\
        .map(lambda line: strip_punc(line, all=True).lower())\
        .flatMap(lambda line: line.split())
```

Removing the Stop Words

Next, let's use RDD method `filter` to create a new RDD with no stop words remaining:

[lick here to view code image](#)

```
[6]: filtered = tokenized.filter(lambda word: word not in stop_words)
```

Counting Each Remaining Word

Now that we have only the non-stop-words, we can count the number of occurrences of each word. To do so, we first `map` each word to a tuple containing the word and a count of 1. This is similar to what we did in Hadoop MapReduce. Spark will distribute the reduction task across the cluster's nodes. On the resulting RDD, we then call the method `reduceByKey`, passing the `operator` module's `add` function as an argument. This tells method `reduceByKey` to *add* the counts for tuples that contain the same word (the key):

[lick here to view code image](#)

```
[7]: from operator import add
    word_counts = filtered.map(lambda word: (word, 1)).reduceByKey(add)
```

Locating Words with Counts Greater Than or Equal to 60

Since there are hundreds of words in *Romeo and Juliet*, let's filter the RDD to keep only those words with 60 or more occurrences:

[lick here to view code image](#)

```
[8]: filtered_counts = word_counts.filter(lambda item: item[1] >= 60)
```

Sorting and Displaying the Results

At this point, we've specified all the steps to count the words. When you call RDD method `collect`, Spark initiates all the processing steps we specified above and returns a list containing the final results—in this case, the tuples of words and their counts. From your perspective, everything appears to execute on one computer. However, if the `SparkContext`

is configured to use a cluster, Spark will divide the tasks among the cluster's worker nodes for you. In the following snippet, sort in descending order (`reverse=True`) the list of tuples by their counts (`itemgetter(1)`).

The following snippet calls method `collect` to obtain the results and sorts those results in descending order by word count:

[lick here to view code image](#)

```
[9]: from operator import itemgetter
      sorted_items = sorted(filtered_counts.collect(),
                           key=itemgetter(1), reverse=True)
```

Finally, let's display the results. First, we determine the word with the most letters so we can right-align all the words in a field of that length, then we display each word and its count:

[lick here to view code image](#)

```
[10]: max_len = max([len(word) for word, count in sorted_items])
      for word, count in sorted_items:
          print(f'{word:>{max_len}}: {count}')
[10]: romeo: 298
      thou: 277
      juliet: 178
      thy: 170S
      nurse: 146
      capulet: 141
      love: 136
      thee: 135
      shall: 110
      lady: 109
      friar: 104
      come: 94
      mercutio: 83
      good: 80
      benvolio: 79
      enter: 75
      go: 75
      i'll: 71
      tybalt: 69
      death: 69
      night: 68
      lawrence: 67
      man: 65
      hath: 64
      one: 60
```

16.6.4 Spark Word Count on Microsoft Azure

As we said previously, we want to expose you to both tools you can use for free and real-world development scenarios. In this section, you'll implement the Spark word-count example on a Microsoft Azure HDInsight Spark cluster.

Create an Apache Spark Cluster in HDInsight Using the Azure Portal

The following link explains how to set up a Spark cluster using the HDInsight service:

<https://docs.microsoft.com/en-us/azure/hdinsight/spark/apache-spark-jupyter-spark>

While following the **Create an HDInsight Spark cluster** steps, note the same issues we listed in the Hadoop cluster setup earlier in this chapter and for the **Cluster type** select **Spark**.

Again, the default cluster configuration provides more resources than you need for our examples. So, in the **Cluster summary**, perform the steps shown in the Hadoop cluster setup to change the number of worker nodes to 2 and to configure the worker and head nodes to use **D3 v2** computers. When you click **Create**, it takes 20 to 30 minutes to configure and deploy your cluster.

Install Libraries into a Cluster

If your Spark code requires libraries that are not installed in the HDInsight cluster, you'll need to install them. To see what libraries are installed by default, you can use ssh to log into your cluster (as we showed earlier in the chapter) and execute the command:

```
/usr/bin/anaconda/envs/py35/bin/conda list
```

Since your code will execute on multiple cluster nodes, libraries must be installed on *every* node. Azure requires you to create a Linux shell script that specifies the commands to install the libraries. When you submit that script to Azure, it validates the script, then executes it on every node. Linux shell scripts are beyond this book's scope, and the script must be hosted on a web server from which Azure can download the file. So, we created an install script for you that installs the libraries we use in the Spark examples. Perform the following steps to install these libraries:

1. In the Azure portal, select your cluster.
2. In the list of items under the cluster's search box, click **Script Actions**.
3. Click **Submit new** to configure the options for the library installation script. For the **Script type** select **Custom**, for the **Name** specify `libraries` and for the **Bash script URI** use:

```
http://deitel.com/bookresources/IntroToPython/install_libraries.sh
```
4. Check both **Head** and **Worker** to ensure that the script installs the libraries on all the nodes.
5. Click **Create**.

When the cluster finishes executing the script, if it executed successfully, you'll see a green check next to the script name in the list of script actions. Otherwise, Azure will notify you that there were errors.

Copying `RomeoAndJuliet.txt` to the HDInsight Cluster

As you did in the Hadoop demo, let's use the `scp` command to upload to the cluster the `RomeoAndJuliet.txt` file you used in the "Natural Language Processing" chapter. In a Command Prompt, Terminal or shell, change to the folder containing the file (we assume this

chapter's `ch16` folder), then enter the following command. Replace *YourClusterName* with the name you specified when creating your cluster and press *Enter* only when you've typed the entire command. The colon is required and indicates that you'll supply your cluster password when prompted. At that prompt, type the password you specified when setting up the cluster, then press *Enter*:

```
scp RomeoAndJuliet.txt sshuser@YourClusterName-ssh.azurehdinsight.net:
```

Next, use `ssh` to log into your cluster and access its command line. In a Command Prompt, Terminal or shell, execute the following command. Be sure to replace *YourClusterName* with your cluster name. Again, you'll be prompted for your cluster password:

```
ssh sshuser@YourClusterName-ssh.azurehdinsight.net
```

To work with the `RomeoAndJuliet.txt` file in Spark, first use the `ssh` session to copy the file into the cluster's Hadoop's file system by executing the following command. Once again, we'll use the already existing folder `/examples/data` that Microsoft includes for use with HDInsight tutorials. Again, press *Enter* only when you've typed the entire command:

```
hadoop fs -copyFromLocal RomeoAndJuliet.txt
/example/data/RomeoAndJuliet.txt
```

Accessing Jupyter Notebooks in HDInsight

At the time of this writing, HDInsight uses the *old* Jupyter Notebook interface, rather than the newer JupyterLab interface shown earlier. For a quick overview of the old interface see:

```
https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Notebook%20
```

o access Jupyter Notebooks in HDInsight, in the Azure portal select **All resources**, then your cluster. In the **Overview** tab, select **Jupyter notebook** under **Cluster dashboards**. This opens a web browser window and asks you to log in. Use the username and password you specified when setting up the cluster. If you did not specify a username, the default is `admin`. Once you log in, Jupyter displays a folder containing `PySpark` and `Scala` subfolders. These contain Python and Scala Spark tutorials.

Uploading the `RomeoAndJulietCounter.ipynb` Notebook

You can create new notebooks by clicking **New** and selecting `PySpark3`, or you can upload existing notebooks from your computer. For this example, let's upload the previous section's `RomeoAndJulietCounter.ipynb` notebook and modify it to work with Azure. To do so, click the **Upload** button, navigate to the `ch16` example folder's `SparkWordCount` folder, select `RomeoAndJulietCounter.ipynb` and click **Open**. This displays the file in the folder with an **Upload** button to its right. Click that button to place the notebook in the current folder. Next, click the notebook's name to open it in a new browser tab. Jupyter will display a **Kernel not found** dialog. Select **PySpark3** and click **OK**. Do not run any cells yet.

Modifying the Notebook to Work with Azure

Perform the following steps, executing each cell as you complete the step:

1. The HDInsight cluster will not allow NLTK to store the downloaded stop words in NLTK's default folder because it's part of the system's protected folders. In the first cell, modify the call `nltk.download('stopwords')` as follows to store the stop words in the current folder (`'.'`):

```
nltk.download('stopwords', download_dir='.')
```

When you execute the first cell, Starting Spark application appears below the cell while HDInsight sets up a `SparkContext` object named `sc` for you. When this task is complete, the cell's code executes and downloads the stop words.

2. In the second cell, before loading the stop words, you must tell NLTK that they're located in the current folder. Add the following statement after the `import` statement to tell NLTK to search for its data in the current folder:

```
nltk.data.path.append('.')
```

3. Because HDInsight sets up the `SparkContext` object for you, the third and fourth cells of the original notebook are not needed, so you can delete them. To do so, either click inside it and select **Delete Cells** from Jupyter's **Edit** menu, or click in the white margin to the cell's left and type `dd`.
4. In the next cell, specify the location of `RomeoAndJuliet.txt` in the underlying Hadoop file system. Replace the string `'RomeoAndJuliet.txt'` with the string

```
'wasb:///example/data/RomeoAndJuliet.txt'
```

The notation `wasb:///` indicates that `RomeoAndJuliet.txt` is stored in a Windows Azure Storage Blob (WASB)—Azure's interface to the HDFS file system.

5. Because Azure currently uses Python 3.5.x, it does not support f-strings. So, in the last cell, replace the f-string with the following older-style Python string formatting using the string method `format`:

```
print('{:>{width}}: {}'.format(word, count, width=max_len))
```

You'll see the same final results as in the previous section.

Caution: Be sure to delete your cluster and other resources when you're done with them, so you do not incur charges. For more information, see:

<https://docs.microsoft.com/en-us/azure/azure-resource-manager/resource-group-po>

ote that when you delete your Azure resources, *your notebooks will be deleted as well*. You

can download the notebook you just executed by selecting **File > Download as > Notebook (.ipynb)** in Jupyter.

16.7 SPARK STREAMING: COUNTING TWITTER HASHTAGS USING THE PYSPARK-NOTEBOOK DOCKER STACK

In this section, you'll create and run a Spark streaming application in which you'll receive a stream of tweets on the topic(s) you specify and summarize the top-20 hashtags in a bar chart that updates every 10 seconds. For this purpose of this example, you'll use the Jupyter Docker container from the first Spark example.

There are two parts to this example. First, using the techniques from the “Data Mining Twitter” chapter, you'll create a script that streams tweets from Twitter. Then, we'll use Spark streaming in a Jupyter Notebook to read the tweets and summarize the hashtags.

The two parts will communicate with one another via networking **sockets**—a low-level view of *client/server networking* in which a *client* app communicates with a *server* app over a network using techniques similar to file I/O. A program can read from a socket or write to a socket similarly to reading from a file or writing to a file. The socket represents one endpoint of a connection. In this case, the *client* will be a Spark application, and the *server* will be a script that receives streaming tweets and sends them to the Spark app.

Launching the Docker Container and Installing Tweepy

For this example, you'll install the Tweepy library into the Jupyter Docker container. Follow section 16.6.2's instructions for launching the container and installing Python libraries into it. Use the following command to install Tweepy:

```
pip install tweepy
```

16.7.1 Streaming Tweets to a Socket

The script `starttweetstream.py` contains a modified version of the `TweetListener` class from the “Data Mining Twitter” chapter. It streams the specified number of tweets and sends them to a socket on the local computer. When the tweet limit is reached, the script closes the socket. You've already used Twitter streaming, so we'll focus only on what's new. Ensure that the file `keys.py` (in the `ch16` folder's `SparkHashtagSummarizer` subfolder) contains your Twitter credentials.

Executing the Script in the Docker Container

In this example, you'll use JupyterLab's Terminal window to execute `starttweetstream.py` in one tab, then use a notebook to perform the Spark task in another tab. With the Jupyter `pyspark-notebook` Docker container running, open

```
http://localhost:8888/lab
```

in your web browser. In JupyterLab, select **File > New > Terminal** to open a new tab containing a **Terminal**. This is a Linux-based command line. Typing the `ls` command and pressing *Enter* lists the current folder's contents. By default, you'll see the container's `work`

folder.

To execute `starttweetstream.py`, you must first navigate to the `SparkHashtagSummarizer` folder with the command ⁸:

⁸Windows users should note that Linux uses `/` rather than `\` to separate folders and that file and folder names are case sensitive.

```
cd work/SparkHashtagSummarizer
```

You can now execute the script with the command of the form

```
ipython starttweetstream.py number_of_tweets search_terms
```

where *number_of_tweets* specifies the total number of tweets to process and *search_terms* one or more space-separated strings to use for filtering tweets. For example, the following command would stream 1000 tweets about football:

```
ipython starttweetstream.py 1000 football
```

At this point, the script will display "Waiting for connection" and will wait until Spark connects to begin streaming the tweets.

`starttweetstream.py` import Statements

For discussion purposes, we've divided `starttweetstream.py` into pieces. First, we import the modules used in the script. The Python Standard Library's **socket module** provides the capabilities that enable Python apps to communicate via sockets.

[lick here to view code image](#)

```
1 # starttweetstream.py
2 """Script to get tweets on topic(s) specified as script argument(s)
3    and send tweet text to a socket for processing by Spark."""
4 import keys
5 import socket
6 import sys
7 import tweepy
8
```

Class `TweetListener`

Once again, you've seen most of the code in class `TweetListener`, so we focus only on what's new here:

- Method `__init__` (lines 12–17) now receives a `connection` parameter representing the socket and stores it in the `self.connection` attribute. We use this socket to send the hashtags to the Spark application.
- In method `on_status` (lines 24–44), lines 27–32 extract the hashtags from the Tweepy `Status` object, convert them to lowercase and create a space-separated string of the

hashtags to send to Spark. The key statement is line 39:

```
self.connection.send(hashtags_string.encode('utf-8'))
```

which uses the `connection` object's `send` method to send the tweet text to whatever application is reading from that socket. Method `send` expects as its argument a sequence of bytes. The string method call `encode('utf-8')` converts the string to bytes. Spark will automatically read the bytes and reconstruct the strings.

[lick here to view code image](#)

```
9 class TweetListener(tweepy.StreamListener):
10     """Handles incoming Tweet stream."""
11
12     def __init__(self, api, connection, limit=10000):
13         """Create instance variables for tracking number of tweets."""
14         self.connection = connection
15         self.tweet_count = 0
16         self.TWEET_LIMIT = limit # 10,000 by default
17         super().__init__(api) # call superclass's init
18
19     def on_connect(self):
20         """Called when your connection attempt is successful, enabling
21         you to perform appropriate application tasks at that point."""
22         print('Successfully connected to Twitter\n')
23
24     def on_status(self, status):
25         """Called when Twitter pushes a new tweet to you."""
26         # get the hashtags
27         hashtags = []
28
29         for hashtag_dict in status.entities['hashtags']:
30             hashtags.append(hashtag_dict['text'].lower())
31
32         hashtags_string = ' '.join(hashtags) + '\n'
33         print(f'Screen name: {status.user.screen_name}:')
34         print(f'Hashtags: {hashtags_string}')
35         self.tweet_count += 1 # track number of tweets processed
36
37         try:
38             # send requires bytes, so encode the string in utf-8 format
39             self.connection.send(hashtags_string.encode('utf-8'))
40         except Exception as e:
41             print(f'Error: {e}')
42
43         # if TWEET_LIMIT is reached, return False to terminate streaming
44         return self.tweet_count != self.TWEET_LIMIT
45
46     def on_error(self, status):
47         print(status)
48         return True
49
```

Main Application

Lines 50–80 execute when you run the script. You've connected to Twitter to stream tweets previously, so here we discuss only what's new in this example.

Line 51 gets the number of tweets to process by converting the command-line argument

`sys.argv[1]` to an integer. Recall that element 0 represents the script's name.

[lick here to view code image](#)

```
50 if __name__ == '__main__':  
51     tweet_limit = int(sys.argv[1]) # get maximum number of tweets
```

Line 52 calls the `socket` module's **socket function**, which returns a socket object that we'll use to wait for a connection from the Spark application.

[lick here to view code image](#)

```
52     client_socket = socket.socket() # create a socket  
53
```

Line 55 calls the socket object's **bind method** with a tuple containing the hostname or IP address of the computer and the port number on that computer. Together these represent where this script will wait for an initial connection from another app:

[lick here to view code image](#)

```
54     # app will use localhost (this computer) port 9876  
55     client_socket.bind(('localhost', 9876))  
56
```

Line 58 calls the socket's **listen method**, which causes the script to *wait* until a connection is received. This is the statement that prevents the Twitter stream from starting until the Spark application connects.

[lick here to view code image](#)

```
57     print('Waiting for connection')  
58     client_socket.listen() # wait for client to connect  
59
```

Once the Spark application connects, line 61 calls socket method **accept**, which accepts the connection. This method returns a tuple containing a new socket object that the script will use to communicate with the Spark application and the IP address of the Spark application's computer.

[lick here to view code image](#)

```
60     # when connection received, get connection/client address  
61     connection, address = client_socket.accept()  
62     print(f'Connection received from {address}')
```

Next, we authenticate with Twitter and start the stream. Lines 73–74 set up the stream, passing the socket object `connection` to the `TweetListener` so that it can use the socket to send hashtags to the Spark application.

[lick here to view code image](#)

```
64     # configure Twitter access
65     auth = tweepy.OAuthHandler(keys.consumer_key, keys.consumer_secret)
66     auth.set_access_token(keys.access_token, keys.access_token_secret)
67
68     # configure Tweepy to wait if Twitter rate limits are reached
69     api = tweepy.API(auth, wait_on_rate_limit=True,
70                      wait_on_rate_limit_notify=True)
71
72     # create the Stream
73     twitter_stream = tweepy.Stream(api.auth,
74                                   TweetListener(api, connection, tweet_limit))
75
76     # sys.argv[2] is the first search term
77     twitter_stream.filter(track=sys.argv[2:])
78
```

Finally, lines 79–80 call the `close` method on the socket objects to release their resources.

[lick here to view code image](#)

```
79     connection.close()
80     client_socket.close()
```

16.7.2 Summarizing Tweet Hashtags; Introducing Spark SQL

In this section, you'll use Spark streaming to read the hashtags sent via a socket by the script `starttweetstream.py` and summarize the results. You can either create a new notebook and enter the code you see here or load the `hashtagsummarizer.ipynb` notebook we provide in the `ch16 examples` folder's `SparkHashtagSummarizer` subfolder.

Importing the Libraries

First, let's import the libraries used in this notebook. We'll explain the `pyspark` classes as we use them. From `IPython`, we imported the `display` module, which contains classes and utility functions that you can use in Jupyter. In particular, we'll use the `clear_output` function to remove an existing chart before displaying a new one:

[lick here to view code image](#)

```
[1]: from pyspark import SparkContext
    from pyspark.streaming import StreamingContext
    from pyspark.sql import Row, SparkSession
    from IPython import display
    import matplotlib.pyplot as plt
    import seaborn as sns
    %matplotlib inline
```

This Spark application summarizes hashtags in 10-second batches. After processing each batch, it displays a Seaborn barplot. The `IPython` magic

```
%matplotlib inline
```


indicates that Matplotlib-based graphics should be displayed in the notebook rather than in their own windows. Recall that Seaborn uses Matplotlib.

We've used several IPython magics throughout the book. There are many magics specifically for use in Jupyter Notebooks. For the complete list of magics see:

```
https://ipython.readthedocs.io/en/stable/interactive/magics.html
```

Utility Function to Get the `SparkSession`

As you'll soon see, you can use **Spark SQL** to query data in resilient distributed datasets (RDDs). Spark SQL uses a Spark **DataFrame** to get a table view of the underlying RDDs. A **SparkSession** (module `pyspark.sql`) is used to create a `DataFrame` from an RDD.

There can be only one `SparkSession` object per Spark application. The following function, which we borrowed from the *Spark Streaming Programming Guide*,⁹ defines the correct way to get a `SparkSession` instance if it already exists or to create one if it does not yet exist:⁰

⁹ <https://spark.apache.org/docs/latest/streaming-programming-guide.html#dataframe-and-sql-operations>.

⁰Because this function was borrowed from the *Spark Streaming Programming Guides DataFrame and SQL Operations* section

(<https://spark.apache.org/docs/latest/streaming-programming-guide.html#dataframe-and-sql-operations>), we did not rename it to use Python's standard function naming style, and we did not use single quotes to delimit strings.

[lick here to view code image](#)

```
[2]: def getSparkSessionInstance(sparkConf):
    """Spark Streaming Programming Guide's recommended method
    for getting an existing SparkSession or creating a new one."""
    if ("sparkSessionSingletonInstance" not in globals()):
        globals()["sparkSessionSingletonInstance"] = SparkSession \
            .builder \
            .config(conf=sparkConf) \
            .getOrCreate()
    return globals()["sparkSessionSingletonInstance"]
```

Utility Function to Display a Barchart Based on a Spark DataFrame

We call function `display_barplot` after Spark processes each batch of hashtags. Each call clears the previous Seaborn barplot, then displays a new one based on the Spark `DataFrame` it receives. First, we call the Spark `DataFrame`'s `toPandas` method to convert it to a pandas `DataFrame` for use with Seaborn. Next, we call the `clear_output` function from the `IPython.display` module. The keyword argument `wait=True` indicates that the function should remove the prior graph (if there is one), but only once the new graph is ready to display. The rest of the code in the function uses standard Seaborn techniques we've shown previously. The function call `sns.color_palette('cool', 20)` selects twenty colors from the Matplotlib 'cool' color palette:

[lick here to view code image](#)

```
[3]: def display_barplot(spark_df, x, y, time, scale=2.0, size=(16, 9)):
    """Displays a Spark DataFrame's contents as a bar plot."""
    df = spark_df.toPandas()

    # remove prior graph when new one is ready to display
    display.clear_output(wait=True)
    print(f'TIME: {time}')

    # create and configure a Figure containing a Seaborn barplot
    plt.figure(figsize=size)
    sns.set(font_scale=scale)
    barplot = sns.barplot(data=df, x=x, y=y,
                          palette=sns.color_palette('cool', 20))

    # rotate the x-axis labels 90 degrees for readability
    for item in barplot.get_xticklabels():
        item.set_rotation(90)

    plt.tight_layout()
    plt.show()
```

Utility Function to Summarize the Top-20 Hashtags So Far

In Spark streaming, a **DStream** is a sequence of RDDs each representing a mini-batch of data to process. As you'll soon see, you can specify a function that is called to perform a task for every RDD in the stream. In this app, the function `count_tags` will summarize the hashtag counts in a given RDD, add them to the current totals (maintained by the `SparkSession`), then display an updated top-20 barplot so that we can see how the top-20 hashtags are changing over time. ¹ For discussion purposes, we've broken this function into smaller pieces. First, we get the `SparkSession` by calling the utility function `getSparkSessionInstance` with the `SparkContext`'s configuration information. Every RDD has access to the `SparkContext` via the `context` attribute:

¹When this function gets called the first time, you might see an exceptions error message display if no tweets with hashtags have been received yet. This is because we simply display the error message in the standard output. That message will disappear as soon as there are tweets with hashtags.

[lick here to view code image](#)

```
[4]: def count_tags(time, rdd):
    """Count hashtags and display top-20 in descending order."""
    try:
        # get SparkSession
        spark = getSparkSessionInstance(rdd.context.getConf())
```

Next, we call the RDD's `map` method to map the data in the RDD to **Row** objects (from the `pyspark.sql` package). The RDDs in this example contain tuples of hashtags and counts. The `Row` constructor uses the names of its keyword arguments to specify the column names for each value in that row. In this case, `tag[0]` is the hashtag in the tuple, and `tag[1]` is the total count for that hashtag:

[lick here to view code image](#)

```
# map hashtag string-count tuples to Rows
rows = rdd.map(
    lambda tag: Row(hashtag=tag[0], total=tag[1]))
```

The next statement creates a Spark `DataFrame` containing the `Row` objects. We'll use this with Spark SQL to query the data to get the top-20 hashtags with their total counts:

[lick here to view code image](#)

```
# create a DataFrame from the Row objects
hashtags_df = spark.createDataFrame(rows)
```

To query a Spark `DataFrame`, first create a *table view*, which enables Spark SQL to query the `DataFrame` like a table in a relational database. Spark `DataFrame` method **`createOrReplaceTempView`** creates a temporary table view for the `DataFrame` and names the view for use in the `from` clause of a query:

[lick here to view code image](#)

```
# create a temporary table view for use with Spark SQL
hashtags_df.createOrReplaceTempView('hashtags')
```

Once you have a table view, you can query the data using Spark SQL. ² The following statement uses the `SparkSession` instance's `sql` method to perform a Spark SQL query that selects the `hashtag` and `total` columns from the `hashtags` table view, orders the selected rows by `total` in descending (`desc`) order, then returns the first 20 rows of the result (`limit 20`). Spark SQL returns a new Spark `DataFrame` containing the results:

² or details of Spark SQLs syntax, see <https://spark.apache.org/sql/>.

[lick here to view code image](#)

```
# use Spark SQL to get top 20 hashtags in descending order
top20_df = spark.sql(
    """select hashtag, total
    from hashtags
    order by total, hashtag desc
    limit 20""")
```

Finally, we pass the Spark `DataFrame` to our `display_barplot` utility function. The `hashtags` and `totals` will be displayed on the *x*- and *y*-axes, respectively. We also display the time at which `count_tags` was called:

[lick here to view code image](#)

```
display_barplot(top20_df, x='hashtag', y='total', time=time)
except Exception as e:
    print(f'Exception: {e}')
```

Getting the SparkContext

The rest of the code in this notebook sets up Spark streaming to read text from the `starttweetstream.py` script and specifies how to process the tweets. First, we create the `SparkContext` for connecting to the Spark cluster:

```
[5]: sc = SparkContext()
```

Getting the StreamingContext

For Spark streaming, you must create a `StreamingContext` (module `pyspark.streaming`), providing as arguments the `SparkContext` and how often in seconds to process batches of streaming data. In this app, we'll process batches every 10 seconds—this is the *batch interval*:

[lick here to view code image](#)

```
[6]: ssc = StreamingContext(sc, 10)
```

Depending on how fast data is arriving, you may wish to shorten or lengthen your batch intervals. For a discussion of this and other performance-related issues, see the Performance Tuning section of the *Spark Streaming Programming Guide*:

<https://spark.apache.org/docs/latest/streaming-programming-guide.html#performan>

Setting Up a Checkpoint for Maintaining State

By default, Spark streaming does not maintain state information as you process the stream of RDDs. However, you can use Spark **checkpointing** to keep track of the streaming state. Checkpointing enables:

- fault-tolerance for restarting a stream in cases of cluster node or Spark application failures, and
- stateful transformations, such as summarizing the data received so far—as we're doing in this example.

`StreamingContext` method **checkpoint** sets up the checkpointing folder:

[lick here to view code image](#)

```
[7]: ssc.checkpoint('hashtagsummarizer_checkpoint')
```

For a Spark streaming application in a cloud-based cluster, you'd specify a location within HDFS to store the checkpoint folder. We're running this example in the local Jupyter Docker image, so we simply specified the name of a folder, which Spark will create in the current folder (in our case, the `ch16` folder's `SparkHashtagSummarizer`). For more details on checkpointing, see

Connecting to the Stream via a Socket

`StreamingContext` method `socketTextStream` connects to a socket from which a stream of data will be received and returns a `DStream` that receives the data. The method's arguments are the hostname and port number to which the `StreamingContext` should connect—these must match where the `starttweetstream.py` script is waiting for the connection:

[lick here to view code image](#)

```
[8]: stream = ssc.socketTextStream('localhost', 9876)
```

Tokenizing the Lines of Hashtags

We use functional-style programming calls on a `DStream` to specify the processing steps to perform on the streaming data. The following call to `DStream`'s `flatMap` method tokenizes a line of space-separated hashtags and returns a new `DStream` representing the individual tags:

[lick here to view code image](#)

```
[9]: tokenized = stream.flatMap(lambda line: line.split())
```

Mapping the Hashtags to Tuples of Hashtag-Count Pairs

Next, similar to the Hadoop mapper earlier in this chapter, we use `DStream` method `map` to get a new `DStream` in which each hashtag is mapped to a hashtag-count pair (in this case as a tuple) in which the count is initially 1:

[lick here to view code image](#)

```
[10]: mapped = tokenized.map(lambda hashtag: (hashtag, 1))
```

Totaling the Hashtag Counts So Far

`DStream` method `updateStateByKey` receives a two-argument lambda that totals the counts for a given key and adds them to the prior total for that key:

[lick here to view code image](#)

```
[11]: hashtag_counts = tokenized.updateStateByKey(  
    lambda counts, prior_total: sum(counts) + (prior_total or 0))
```

Specifying the Method to Call for Every RDD

Finally, we use `DStream` method `foreachRDD` to specify that every processed RDD should be passed to function `count_tags`, which then summarizes the top-20 hashtags so far and

displays a barplot:

[lick here to view code image](#)

```
[12]: hashtag_counts.foreachRDD(count_tags)
```

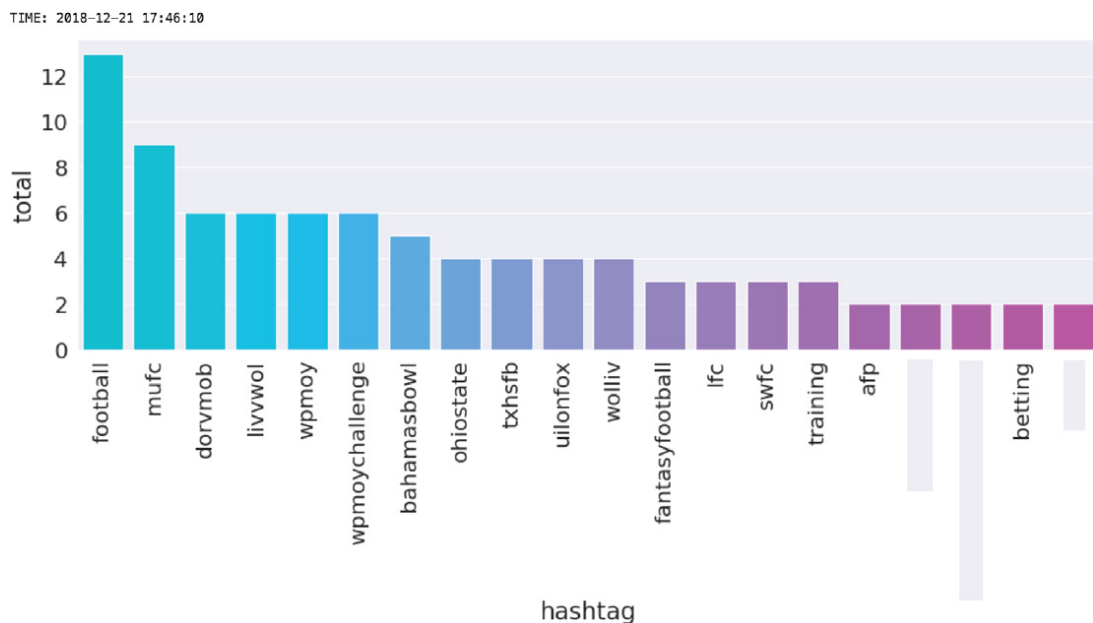
Starting the Spark Stream

Now, that we’ve specified the processing steps, we call the `StreamingContext`’s `start` method to connect to the socket and begin the streaming process.

[lick here to view code image](#)

```
[13]: ssc.start() # start the Spark streaming
```

The following shows a sample barplot produced while processing a stream of tweets about “football.” Because football is a different sport in the United States and the rest of the world the hashtags relate to both American football and what we call soccer—we grayed out three hashtags that were not appropriate for publication:



16.8 INTERNET OF THINGS AND DASHBOARDS

In the late 1960s, the Internet began as the ARPANET, which initially connected four universities and grew to 10 nodes by the end of 1970. ³ In the last 50 years, that has grown to billions of computers, smartphones, tablets and an enormous range of other device types connected to the Internet worldwide. *Any* device connected to the Internet is a “thing” in the Internet of Things (IoT).

³ <https://en.wikipedia.org/wiki/ARPANET#History..>

Each device has a unique Internet protocol address (IP address) that identifies it. The explosion of connected devices exhausted the approximately 4.3 billion available IPv4 (Internet Protocol version 4) addresses ⁴ and led to the development of IPv6, which supports approximately 3.4×10^{38} addresses (that’s a lot of zeros). ⁵

⁴ https://en.wikipedia.org/wiki/IPv4_address_exhaustion.

⁵ <https://en.wikipedia.org/wiki/IPv6>.

“Top research firms such as Gartner and McKinsey predict a jump from the 6 billion connected devices we have worldwide today, to 20–30 billion by 2020.” ⁶ Various predictions say that number could be 50 billion. Computer-controlled, Internet-connected devices continue to proliferate. The following is a small subset IoT device types and applications.

⁶ <https://www.pubnub.com/developers/tech/how-pubnub-works/>.

IoT devices		
activity trackers—		
Apple Watch, FitBit,		
Amazon Dash ordering buttons		smart home—lights, garage openers, video cameras, doorbells, irrigation controllers, security devices, smart locks, smart plugs, smoke detectors, thermostats, air vents
Amazon Echo (Alexa), Apple HomePod (Siri), Google Home (Google Assistant)	healthcare—blood glucose monitors for diabetics, blood pressure monitors, electrocardiograms (EKG/ECG), electroencephalograms (EEG), heart monitors, ingestible sensors, pacemakers, sleep trackers,	tsunami sensors
	sensors—chemical, gas, GPS, humidity, light, motion, pressure, temperature,	tracking devices
appliances—ovens, coffee makers, refrigerators,		wine cellar refrigerators
driverless cars		wireless network devices
earthquake sensors		

though there's a lot of excitement and opportunity in IoT, not everything is positive. There are many security, privacy and ethical concerns. Unsecured IoT devices have been used to perform distributed-denial-of-service (DDOS) attacks on computer systems.⁷ Home security cameras that you intend to protect your home could potentially be hacked to allow others access to the video stream. Voice-controlled devices are always “listening” to hear their trigger words. This leads to privacy and security concerns. Children have accidentally ordered products on Amazon by talking to Alexa devices, and companies have created TV ads that would activate Google Home devices by speaking their trigger words and causing Google Assistant to read Wikipedia pages about a product to you.⁸ Some people worry that these devices could be used to eavesdrop. Just recently, a judge ordered Amazon to turn over Alexa recordings for use in a criminal case.⁹

⁷ <https://threatpost.com/iot-security-concerns-peaking-with-no-end-in-sight/131308/>.

⁸ <https://www.symantec.com/content/dam/symantec/docs/security-center/whitepapers/istr-security-voice-activated-smart-speakers-en.pdf>.

⁹ <https://techcrunch.com/2018/11/14/amazon-echo-recordings-judge-order-case/>.

This Section's Examples

In this section, we discuss the **publish/subscribe model** that IoT and other types of applications use to communicate. First, without writing any code, you'll build a web-based dashboard using Freeboard.io and subscribe to a sample live stream from the PubNub service. Next, you'll simulate an Internet-connected thermostat which publishes messages to the free Dweet.io service using the Python module Dweeepy, then create a dashboard visualization of it with Freeboard.io. Finally, you'll build a Python client that subscribes to a sample live stream from the PubNub service and dynamically visualizes the stream with Seaborn and a Matplotlib `FuncAnimation`.

16.8.1 Publish and Subscribe

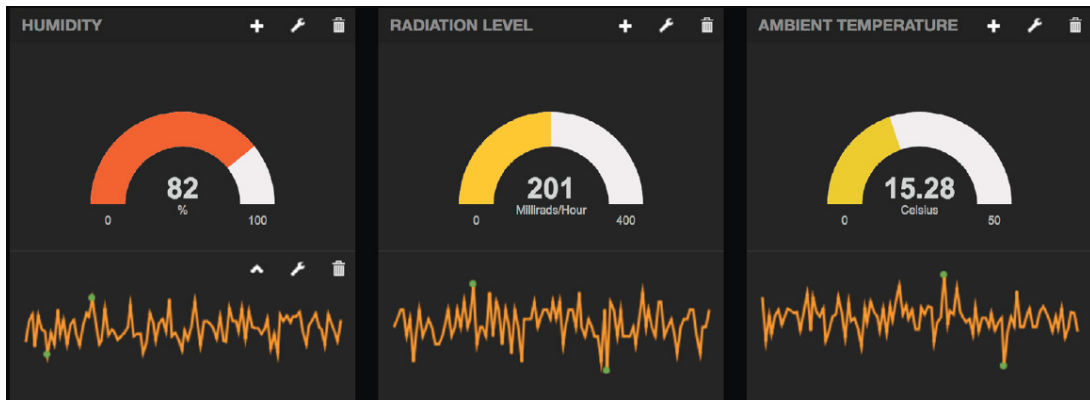
IoT devices (and many other types of devices and applications) commonly communicate with one another and with applications via **pub/sub (publisher/subscriber) systems**. A **publisher** is any device or application that sends a message to a cloud-based service, which in turn sends that message to all **subscribers**. Typically each publisher specifies a topic or channel, and each subscriber specifies one or more **topics** or **channels** for which they'd like to receive messages. There are many pub/sub systems in use today. In the remainder of this section, we'll use PubNub and Dweet.io. You also should investigate Apache Kafka—a Hadoop ecosystem component that provides a high-performance publish/subscribe service, real-time stream processing and storage of streamed data.

16.8.2 Visualizing a PubNub Sample Live Stream with a Freeboard Dashboard

PubNub is a pub/sub service geared to real-time applications in which any software and device connected to the Internet can communicate via small messages. Some of their common use-cases include IoT, chat, online multiplayer games, social apps and collaborative apps. PubNub provides several live streams for learning purposes, including one that simulates IoT sensors (Section 16.8.5 lists the others).

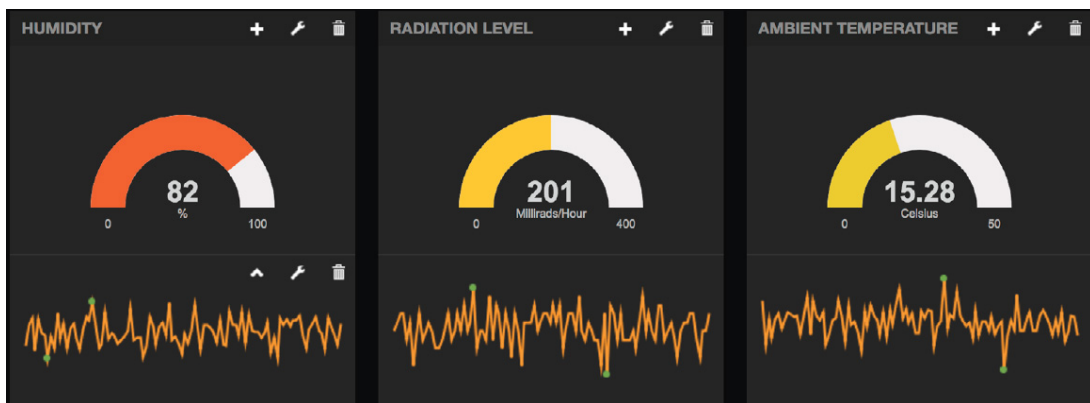
One common use of live data streams is visualizing them for monitoring purposes. In this section, you'll connect PubNub's live simulated sensor stream to a Freeboard.io web-based dashboard. A car's dashboard visualizes data from your car's sensors, showing information such as the outside temperature, your speed, engine temperature, the time and the amount of gas remaining. A web-based dashboard does the same thing for data from various sources, including IoT devices.

Freeboard.io is a cloud-based dynamic dashboard visualization tool. You'll see that, without writing any code, you can easily connect Freeboard.io to various data streams and visualize the data as it arrives. The following dashboard visualizes data from three of the four simulated sensors in the PubNub simulated IoT sensors stream:



For each sensor, we used a **Gauge** (the semicircular visualizations) and a **Sparkline** (the jagged lines) to visualize the data. When you complete this section, you'll see the **Gauges** and **Sparklines** frequently moving as new data arrives multiple times per second.

In addition to their paid service, Freeboard.io provides an open-source version (with fewer options) on GitHub. They also provide tutorials that show how to add *custom plug-ins*, so you can develop your own visualizations to add to their dashboards.



Signing up for Freeboard.io

For this example, register for a Freeboard.io 30-day trial at

```
https://freeboard.io/signup
```

Once you've registered, the **My Freeboards** page appears. If you'd like, you can click the **Try a Tutorial** button and visualize data from your smartphone.

Creating a New Dashboard

n the upper-right corner of the **My Freeboards** page, enter `Sensor Dashboard` in the **enter a name** field, then click the **Create New** button to create a dashboard. This displays the dashboard designer.

Adding a Data Source

If you add your data source(s) before designing your dashboard, you'll be able to configure each visualization as you add it:

1. Under **DATASOURCES**, click **ADD** to specify a new data source.
2. The **DATASOURCE** dialog's **TYPE** drop-down list shows the currently supported data sources, though you can develop plug-ins for new data sources as well. ^o Select **PubNub**. The web page for each PubNub sample live stream specifies the **Channel** and **Subscribe key**. Copy these values from PubNub's Sensor Network page at `https://www.pubnub.com/developers/realtime-data-streams/sensor-network/`, then insert their values in the corresponding **DATASOURCE** dialog fields. Provide a **NAME** for your data source, then click **SAVE**.

^o**Some of the listed data sources are available only via Freeboard.io, not the open source Freeboard on GitHub.**

Adding a Pane for the Humidity Sensor

A Freeboard.io dashboard is divided into panes that group visualizations. Multiple panes can be dragged to rearrange them. Click the + **Add Pane** button to add a new pane. Each pane can have a title. To set it, click the wrench icon on the pane, specify `Humidity` for the **TITLE**, then click **SAVE**.

Adding a Gauge to the Humidity Pane

A Freeboard.io dashboard is divided into panes that group visualizations. Multiple panes can be dragged to rearrange them. Click the + **Add Pane** button to add a new pane. Each pane can have a title. To set it, click the wrench icon on the pane, specify `Humidity` for the **TITLE**, then click **SAVE**.

Notice that the humidity value has four digits of precision to the right of the decimal point. PubNub supports JavaScript expressions, so you can use them to perform calculations or format data. For example, you can use JavaScript's function `Math.round` to round the humidity value to the closest integer. To do so, hover the mouse over the gauge and click its wrench icon. Then, insert `"Math.round("` before the text in the **VALUE** field and `")"` after the text, then click **SAVE**.

Adding a Sparkline to the Humidity Pane

A **sparkline** is a line graph without axes that's typically used to give you a sense of how a data value is changing over time. Add a sparkline for the humidity sensor by clicking the humidity pane's + button, then selecting **Sparkline** from the **TYPE** drop-down list. For the **VALUE**, once again select your data source and **humidity**, then click **SAVE**.

Completing the Dashboard

Using the techniques above, add two more panes and drag them to the right of the first.

Name them **Radiation Level** and **Ambient Temperature**, respectively, and configure each pane with a **Gauge** and **Sparkline** as shown above. For the **Radiation Level** gauge, specify `Millirads/Hour` for the **UNITS** and `400` for the **MAXIMUM**. For the **Ambient Temperature** gauge, specify `Celsius` for the **UNITS** and `50` for the **MAXIMUM**.

16.8.3 Simulating an Internet-Connected Thermostat in Python

Simulation is one of the most important applications of computers. We used simulation with dice rolling in earlier chapters. With IoT, it's common to use simulators to test your applications, especially when you do not have access to actual devices and sensors while developing applications. Many cloud vendors have IoT simulation capabilities, such as IBM Watson IoT Platform and IOTIFY.io.

Here, you'll create a script that simulates an Internet-connected thermostat publishing periodic JSON messages—called dweets—to `dweet.io`. The name “dweet” is based on “tweet”—a dweet is like a tweet from a device. Many of today's Internet-connected security systems include temperature sensors that can issue low-temperature warnings before pipes freeze or high-temperature warnings to indicate there might be a fire. Our simulated sensor will send dweets containing a location and temperature, as well as low- and high-temperature notifications. These will be `True` only if the temperature reaches 3 degrees Celsius or 35 degrees Celsius, respectively. In the next section, we'll use `freeboard.io` to create a simple dashboard that shows the temperature changes as the messages arrive, as well as warning lights for low- and high-temperature warnings.

Installing Dweepy

To publish messages to `dweet.io` from Python, first install the Dweepy library:

```
pip install dweepy
```

The library is straightforward to use. You can view its documentation at:

```
https://github.com/paddycarey/dweepy
```

Invoking the `simulator.py` Script

The Python script `simulator.py` that simulates our thermostat is located in the `ch16` example folder's `iot` subfolder. You invoke the simulator with two command-line arguments representing the number of total messages to simulate and the delay in seconds between sending dweets:

```
ipython simulator.py 1000 1
```

Sending Dweets

The `simulator.py` is shown below. It uses random-number generation and Python techniques that you've studied throughout this book, so we'll focus just on a few lines of code that publish messages to `dweet.io` via Dweepy. We've broken apart the script below for discussion purposes.

By default, `dweet.io` is a public service, so any app can publish or subscribe to messages. When publishing messages, ***you'll want to specify a unique name for your device.*** We used `'temperature-simulator-deitel-python'` (line 17). ¹ Lines 18–21 define a Python dictionary, which will store the current sensor information. Dweepy will convert this into JSON when it sends the dweet.

¹To truly guarantee a unique name, `dweet.io` can create one for you. The Dweepy documentation explains how to do this.

[lick here to view code image](#)

```
1 # simulator.py
2 """A connected thermostat simulator that publishes JSON
3 messages to dweet.io"""
4 import dweepy
5 import sys
6 import time
7 import random
8
9 MIN_CELSIUS_TEMP = -25
10 MAX_CELSIUS_TEMP = 45
11 MAX_TEMP_CHANGE = 2
12
13 # get the number of messages to simulate and delay between them
14 NUMBER_OF_MESSAGES = int(sys.argv[1])
15 MESSAGE_DELAY = int(sys.argv[2])
16
17 dweeter = 'temperature-simulator-deitel-python' # provide a unique name
18 thermostat = {'Location': 'Boston, MA, USA',
19               'Temperature': 20,
20               'LowTempWarning': False,
21               'HighTempWarning': False}
22
```

Lines 25–53 produce the number of simulated message you specify. During each iteration of the loop, we

- generate a random temperature change in the range -2 to $+2$ degrees and modify the temperature,
- ensure that the temperature remains in the allowed range,
- check whether the low- or high-temperature sensor has been triggered and update the thermostat dictionary accordingly,
- display how many messages have been generated so far,
- use Dweepy to send the message to `dweet.io` (line 52), and
- use the `time` module's `sleep` function to wait the specified amount of time before generating another message.

[lick here to view code image](#)

```
23 print('Temperature simulator starting')
24
```

```

25 for message in range(NUMBER_OF_MESSAGES):
26     # generate a random number in the range  -MAX_TEMP_CHANGE
27     # through MAX_TEMP_CHANGE and add it to the current temperature
28     thermostat['Temperature'] += random.randrange(
29         -MAX_TEMP_CHANGE, MAX_TEMP_CHANGE + 1)
30
31     # ensure that the temperature stays within range
32     if thermostat['Temperature'] < MIN_CELSIUS_TEMP:
33         thermostat['Temperature'] = MIN_CELSIUS_TEMP
34
35     if thermostat['Temperature'] > MAX_CELSIUS_TEMP:
36         thermostat['Temperature'] = MAX_CELSIUS_TEMP
37
38     # check for low temperature warning
39     if thermostat['Temperature'] < 3:
40         thermostat['LowTempWarning'] = True
41     else:
42         thermostat['LowTempWarning'] = False
43
44     # check for high temperature warning
45     if thermostat['Temperature'] > 35:
46         thermostat['HighTempWarning'] = True
47     else:
48         thermostat['HighTempWarning'] = False
49
50     # send the dweet to dweet.io via dweeepy
51     print(f'Messages sent: {message + 1}\r', end='')
52     dweeepy.dweet_for(dweeter, thermostat)
53     time.sleep(MESSAGE_DELAY)
54
55 print('Temperature simulator finished')

```

You do not need to register to use the service. On the first call to dweeepy's **dweet_for function** to send a dweet (line 52), `dweet.io` creates the device name. The function receives as arguments the device name (`dweeter`) and a dictionary representing the message to send (`thermostat`). Once you execute the script, you can immediately begin tracking the messages on the `dweet.io` site by going to the following address in your web browser:

```
https://dweet.io/follow/temperature-simulator-deitel-python
```

If you use a different device name, replace "temperature-simulator-deitel-python" with the name you used. The web page contains two tabs. The **Visual** tab shows you the individual data items, displaying a sparkline for any numerical values. The **Raw** tab shows you the actual JSON messages that Dweeepy sent to `dweet.io`.

16.8.4 Creating the Dashboard with Freeboard.io

The sites `dweet.io` and `freeboard.io` are run by the same company. In the `dweet.io` webpage discussed in the preceding section, you can click the **Create a Custom Dashboard** button to open a new browser tab, with a default dashboard already implemented for the temperature sensor. By default, `freeboard.io` will configure a data source named `Dweet` and auto-generate a dashboard containing one pane for each value in the dweet JSON. Within each pane, a text widget will display the corresponding value as the messages arrive.

If you prefer to create your own dashboard, you can use the steps in section 16.8.2 to create a data source (this time selecting `Dweeepy`) and create new panes and widgets, or you can you

modify the auto-generated dashboard.

Below are three screen captures of a dashboard consisting of four widgets:

- A **Gauge** widget showing the current temperature. For this widget's **VALUE** setting, we selected the data source's `Temperature` field. We also set the **UNITS** to `Celsius` and the **MINIMUM** and **MAXIMUM** values to `-25` and `45` degrees, respectively.
- A **Text** widget to show the current temperature in Fahrenheit. For this widget, we set the **INCLUDE SPARKLINE** and **ANIMATE VALUE CHANGES** to **YES**. For this widget's **VALUE** setting, we again selected the data source's `Temperature` field, then added to the end of the **VALUE** field

```
* 9 / 5 + 32
```

to perform a calculation that converts the Celsius temperature to Fahrenheit. We also specified `Fahrenheit` in the **UNITS** field.

- Finally, we added two **Indicator Light** widgets. For the first **Indicator Light**'s **VALUE** setting, we selected the data source's `LowTempWarning` field, set the **TITLE** to `Freeze Warning` and set the **ON TEXT** value to `LOW TEMPERATURE WARNING—ON TEXT` indicates the text to display when value is `true`. For the second **Indicator Light**'s **VALUE** setting, we selected the data source's `HighTempWarning` field, set the **TITLE** to `High Temperature Warning` and set the **ON TEXT** value to `HIGH TEMPERATURE WARNING`.



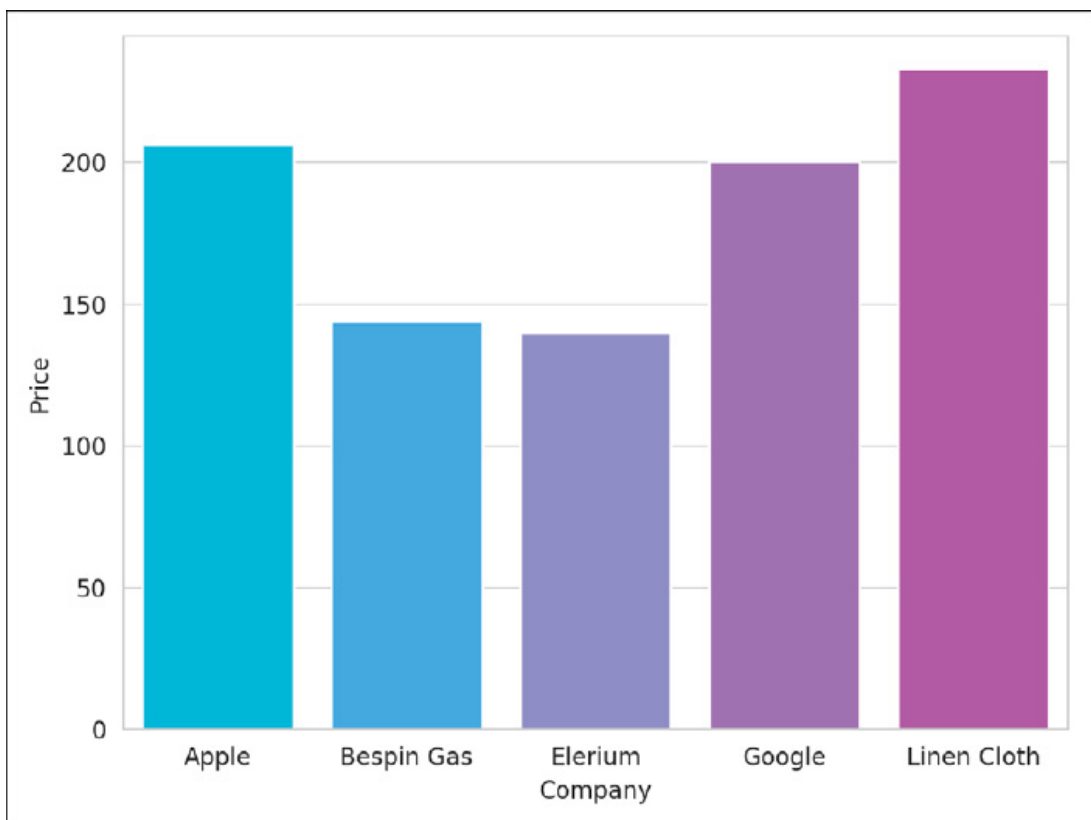
16.8.5 Creating a Python PubNub Subscriber

PubNub provides the `pubnub` Python module for conveniently performing pub/sub operations. They also provide seven sample streams for you to experiment with—four real-time streams and three simulated streams: ²

² <https://www.pubnub.com/developers/realtime-data-streams/>.

- Twitter Stream—provides up to 50 tweets-per-second from the Twitter live stream and does not require your Twitter credentials.
- Hacker News Articles—this site’s recent articles.
- State Capital Weather—provides weather data for the U.S. state capitals.
- Wikipedia Changes—a stream of Wikipedia edits.
- Game State Sync—simulated data from a multiplayer game.
- Sensor Network—simulated data from radiation, humidity, temperature and ambient light sensors.
- Market Orders—simulated stock orders for five companies.

In this section, you’ll use the **pubnub module** to subscribe to their simulated Market Orders stream, then visualize the changing stock prices as a Seaborn barplot, like:



Of course, you also can publish messages to streams. For details, see the pubnub module’s documentation at <https://www.pubnub.com/docs/python/pubnub-python-sdk>.

To prepare for using PubNub in Python, execute the following command to install the latest version of the pubnub module—the `'>=4.1.2'` ensures that at a minimum the 4.1.2 version of the pubnub module will be installed:

```
pip install "pubnub>=4.1.2"
```

The script `stocklistener.py` that subscribes to the stream and visualizes the stock prices is defined in the `ch16` folder’s `pubnub` subfolder. We break the script into pieces here for discussion purposes.

Message Format

The simulated Market Orders stream returns JSON objects containing five key–value pairs with the keys 'bid_price', 'order_quantity', 'symbol', 'timestamp' and 'trade_type'. For this example, we'll use only the 'bid_price' and 'symbol'. The PubNub client returns the JSON data to you as a Python dictionary.

Importing the Libraries

Lines 3–13 import the libraries used in this example. We discuss the PubNub types imported in lines 10–13 as we encounter them below.

[lick here to view code image](#)

```
1 # stocklistener.py
2 """Visualizing a PubNub live stream."""
3 from matplotlib import animation
4 import matplotlib.pyplot as plt
5 import pandas as pd
6 import random
7 import seaborn as sns
8 import sys
9
10 from pubnub.callbacks import SubscribeCallback
11 from pubnub.enums import PNStatusCategory
12 from pubnub.pnconfiguration import PNConfiguration
13 from pubnub.pubnub import PubNub
14
```

List and DataFrame Used for Storing Company Names and Prices

The list `companies` contains the names of the companies reported in the Market Orders stream, and the pandas DataFrame `companies_df` is where we'll store each company's last price. We'll use this DataFrame with Seaborn to display a bar chart.

[lick here to view code image](#)

```
15 companies = ['Apple', 'Bespun Gas', 'Elerium', 'Google', 'Linen Cloth']
16
17 # DataFrame to store last stock prices
18 companies_df = pd.DataFrame(
19     {'company': companies, 'price' : [0, 0, 0, 0, 0]})
20
```

Class `SensorSubscriberCallback`

When you subscribe to a PubNub stream, you must add a listener that receives status notifications and messages from the channel. This is similar to the Tweepy listeners you've defined previously. To create your listener, you must define a subclass of `SubscribeCallback` (module `pubnub.callbacks`), which we discuss after the code:

[lick here to view code image](#)

```
21 class SensorSubscriberCallback(SubscribeCallback):
22     """SensorSubscriberCallback receives messages from PubNub."""
23     def __init__(self, df, limit=1000):
```



```

24     """Create instance variables for tracking number of tweets."""
25     self.df = df # DataFrame to store last stock prices
26     self.order_count = 0
27     self.MAX_ORDERS = limit # 1000 by default
28     super().__init__() # call superclass's init
29
30     def status(self, pubnub, status):
31         if status.category == PNStatusCategory.PNConnectedCategory:
32             print('Connected to PubNub')
33         elif status.category == PNStatusCategory.PNAcknowledgmentCategory:
34             print('Disconnected from PubNub')
35
36     def message(self, pubnub, message):
37         symbol = message.message['symbol']
38         bid_price = message.message['bid_price']
39         print(symbol, bid_price)
40         self.df.at[companies.index(symbol), 'price'] = bid_price
41         self.order_count += 1
42
43         # if MAX_ORDERS is reached, unsubscribe from PubNub channel
44         if self.order_count == self.MAX_ORDERS:
45             pubnub.unsubscribe_all()
46

```

Class `SensorSubscriberCallback`'s `__init__` method stores the `DataFrame` in which each new stock price will be placed. The PubNub client calls overridden method `status` each time a new status message arrives. In this case, we're checking for the notifications that indicate that we've subscribed to or unsubscribed from a channel.

The PubNub client calls overridden method `message` (lines 36–45) when a new message arrives from the channel. Lines 37 and 38 get the company name and price from the message, which we print so you can see that messages are arriving. Line 40 uses the `DataFrame` method `at` to locate the appropriate company's row and its `'price'` column, then assign that element the new price. Once the `order_count` reaches `MAX_ORDERS`, line 45 calls the PubNub client's `unsubscribe_all` method to unsubscribe from the channel.

Function Update

This example visualizes the stock prices using the animation techniques you learned in chapter 6's Intro to Data Science section. Function `update` specifies how to draw one animation frame and is called repeatedly by the `FuncAnimation` we'll define shortly. We use Seaborn function `barplot` to visualize data from the `companies_df` `DataFrame`, using its `'company'` column values on the *x*-axis and `'price'` column values on the *y*-axis.

[lick here to view code image](#)

```

47 def update(frame_number):
48     """Configures bar plot contents for each animation frame."""
49     plt.cla() # clear old barplot
50     axes = sns.barplot(
51         data=companies_df, x='company', y='price', palette='cool')
52     axes.set(xlabel='Company', ylabel='Price')
53     plt.tight_layout()
54

```

Configuring the Figure

In the main part of the script, we begin by setting the Seaborn plot style and creating the Figure object in which the barplot will be displayed:

[lick here to view code image](#)

```
55 if __name__ == '__main__':
56     sns.set_style('whitegrid') # white background with gray grid lines
57     figure = plt.figure('Stock Prices') # Figure for animation
58
```

Configuring the FuncAnimation and Displaying the Window

Next, we set up the FuncAnimation that calls function update, then call Matplotlib's show method to display the Figure. Normally, this method blocks the script from continuing until you close the Figure. Here, we pass the block=False keyword argument to allow the script to continue so we can configure the PubNub client and subscribe to a channel.

[lick here to view code image](#)

```
59 # configure and start animation that calls function update
60 stock_animation = animation.FuncAnimation(
61     figure, update, repeat=False, interval=33)
62 plt.show(block=False) # display window
63
```

Configuring the PubNub Client

Next, we configure the PubNub subscription key, which the PubNub client uses in combination with the channel name to subscribe to the channel. The key is specified as an attribute of the PNConfiguration object (module pubnub.pnconfiguration), which line 69 passes to the new PubNub client object (module pubnub.pubnub). Lines 70–72 create the SensorSubscriberCallback object and pass it to the PubNub client's add_listener method to register it to receive messages from the channel. We use a command-line argument to specify the total number of messages to process.

[lick here to view code image](#)

```
64 # set up pubnub-market-orders sensor stream key
65 config = PNConfiguration()
66 config.subscribe_key = 'sub-c-4377ab04-f100-11e3-bffd-02ee2ddab7fe'
67
68 # create PubNub client and register a SubscribeCallback
69 pubnub = PubNub(config)
70 pubnub.add_listener(
71     SensorSubscriberCallback(df=companies_df,
72                             limit=int(sys.argv[1] if len(sys.argv) > 1 else 1000))
73
```

Subscribing to the Channel

The following statement completes the subscription process, indicating that we wish to receive messages from the channel named 'pubnub-market-orders'. The execute method starts the stream.

[lick here to view code image](#)

```
74     # subscribe to pubnub-sensor-network channel and begin streaming
75     pubnub.subscribe().channels('pubnub-market-orders').execute()
76
```

Ensuring the Figure Remains on the Screen

The second call to Matplotlib's `show` method ensures that the `Figure` remains on the screen until you close its window.

[lick here to view code image](#)

```
77     plt.show()     # keeps graph on screen until you dismiss its window
```

16.9 WRAP-UP

In this chapter, we introduced big data, discussed how large data is getting and discussed hardware and software infrastructure for working with big data. We introduced traditional relational databases and Structured Query Language (SQL) and used the `sqlite3` module to create and manipulate a `books` database in SQLite. We also demonstrated loading SQL query results into pandas `DataFrames`.

We discussed the four major types of NoSQL databases—key–value, document, columnar and graph—and introduced NewSQL databases. We stored JSON tweet objects as documents in a cloud-based MongoDB Atlas cluster, then summarized them in an interactive visualization displayed on a Folium map.

We introduced Hadoop and how it's used in big-data applications. You configured a multi-node Hadoop cluster using the Microsoft Azure HDInsight service, then created and executed a Hadoop MapReduce task using Hadoop streaming.

We discussed Spark and how it's used in high-performance, real-time big-data applications. You used Spark's functional-style filter/map/reduce capabilities, first on a Jupyter Docker stack that runs locally on your own computer, then again using a Microsoft Azure HDInsight multi-node Spark cluster. Next, we introduced Spark streaming for processing data in mini-batches. As part of that example, we used Spark SQL to query data stored in Spark `DataFrames`.

The chapter concluded with an introduction to the Internet of Things (IoT) and the publish/subscribe model. You used Freeboard.io to create a dashboard visualization of a live sample stream from PubNub. You simulated an Internet-connected thermostat which published messages to the free `dweet.io` service using the Python module `Dweeepy`, then used Freeboard.io to visualize the simulated device's data. Finally, you subscribed to a PubNub sample live stream using their Python module.

Thanks for reading *Python for Programmers*. We hope that you enjoyed the book and that you found it entertaining and informative. Most of all we hope you feel empowered to apply the technologies you've learned to the challenges you'll face in your career.



Photo courtesy of Paul Deitel

Expert-Led Video Training on Python, Data Science, AI, Big Data and the Cloud

From **PAUL DEITEL**

Python Fundamentals LiveLessons integrates coverage of the core Python language, Python Standard Library and key data science libraries with a range of data science and artificial intelligence case studies, including:

- Natural language processing
- Data mining Twitter® for sentiment analysis
- IBM® Watson™ and cognitive computing
- Machine learning with classification, regression and clustering
- Deep learning with convolutional and recurrent neural networks
- Big data: Hadoop®, Spark™ and NoSQL
- Internet of Things (IoT)

You'll receive a **hands-on** introduction to Python and data science with world-renowned trainer Paul Deitel as he presents 500+ real-world examples, including 40 larger scripts and case studies. Quickly master the latest Python coding idioms using the interactive **IPython interpreter** with code in **Jupyter Notebooks**, and benefit from additional insights and programming-language comparisons based on Paul's 38 years of programming and teaching experience in Java, C, C++, C++, Swift, iOS, Android and Internet and web programming.

Save 40%* —Use coupon code VIDEO40
informit.com/Deitel

*Discount code VIDEO40 confers a 40% discount off the list price of featured video when purchased on InformIT. Offer is subject to change.



Photo by izusek/gettyimages

Register Your Product at informit.com/register

Access additional benefits and **save 35%** on your next purchase

- Automatically receive a coupon for 35% off your next purchase, valid for 30 days. Look for your code in your InformIT cart or the Manage Codes section of your account page.
- Download available product updates.
- Access bonus material if available.*
- Check the box to hear from us and receive exclusive offers on new editions and related products.

**Registration benefits vary by product. Benefits will be listed on your account page under Registered Products.*

InformIT.com—The Trusted Technology Learning Source

InformIT is the online home of information technology brands at Pearson, the world's foremost education company. At InformIT.com, you can:

- Shop our books, eBooks, software, and video training
- Take advantage of our special offers and promotions (informit.com/promotions)
- Sign up for special offers and content newsletter (informit.com/newsletters)
- Access thousands of free chapters and video lessons

Connect with InformIT—Visit informit.com/community



informIT[®]
the trusted technology learning source

Addison-Wesley • Adobe Press • Cisco Press • Microsoft Press • Pearson IT Certification • Prentice Hall • Que • Sams • Peachpit Press